
Raven–MODFLOW 6

Groundwater Coupling

Manual chapter for the Raven Hydrological Modelling Framework
(based on Raven 4.15)

Rezgar Arabzadeh
Principal Scientist, Hazen and Sawyer

September 2026

Contents

1	Introduction and overview	1
1.1	The problem this solves	1
1.2	Two ways to use the coupling	1
1.3	How to read this manual	2
1.4	What the user does — and what Raven does	2
1.5	Main features	3
1.6	Glossary	3
1.7	The big picture	5
1.8	Scope and assumptions	7
1.9	When to use the coupling — and when not to	7
1.10	What changes for an existing Raven model	7
1.11	Files at a glance	7
2	Groundwater concepts	9
2.1	Head	9
2.2	Darcy’s law	9
2.3	Transmissivity and conductance	9
2.4	Storage	10
2.5	The groundwater flow equation	10
2.6	Head-dependent boundaries	10
2.7	Water table and potentiometric surface	11
2.8	Three kinds of boundary conditions	11
2.9	The Newton method in simple terms	11
2.10	A cell’s water account, with numbers	12
2.11	Units and conversions	12
3	Building the groundwater model	13
3.1	From HRUs to a groundwater grid	13
3.2	From one HRU to its cells: a complete example	16
3.3	Preparing the geometry	16
3.4	Grid types	17
3.5	Building the layers	21
3.6	Parameters used to build the model	23
3.7	What the generated files look like	24
3.8	Solver settings	24
3.9	How MODFLOW solves each time step	25
4	Coupling an existing MODFLOW 6 model	27
4.1	A complete example	28
4.2	Linking HRUs to the model’s cells	28
4.3	Time	30
4.4	Units	31
4.5	Packages	31
4.6	Safeguards	31
4.7	The model’s solver	33
4.8	Outputs and restarts	33
4.9	Worked example	33
4.10	Checking the calibration	34
4.11	Current limits	34

5	Water exchanges between Raven and MODFLOW	35
5.1	Recharge: from Raven to the water table	36
5.2	Seepage: groundwater returning to the surface	38
5.3	Rivers	40
5.4	Evaporation from a shallow water table	43
5.5	Wetlands and lakes held in Raven stores	44
5.6	Reservoirs	45
5.7	Wells and boundaries	47
6	Water accounting and time stepping	50
6.1	Keeping the books: water balance	50
6.2	A real day of water accounts	51
6.3	How to read the balance outputs	51
6.4	The coupled time step, start-up and restarts	51
7	Inputs and outputs	54
7.1	Input reference	54
7.2	Output reference	57
7.3	A complete example set-up	59
8	Applying the coupling	61
8.1	Tutorial: the Liard River model	61
8.2	Preparing the input data	63
8.3	Choosing a grid for the Liard model	65
8.4	Setting up your own model	66
8.5	Typical parameter values	66
8.6	Calibration	67
8.7	The Ostrich input file, line by line	68
8.8	Troubleshooting	68
8.9	Frequently asked questions	68
9	Verification and code audit	71
9.1	Models without groundwater are unchanged	71
9.2	Automated audit of every run	72
9.3	Behaviour tests	73
9.4	Independent checks	73
9.5	Randomized testing	77
9.6	Grid types	77
9.7	Coupling an existing model	79
9.8	Release audit	81
9.9	Code audit	83
9.10	Findings and corrections	83
10	Developer guide and limitations	88
10.1	Source files	88
10.2	Order of calls	89
10.3	MODFLOW 6 memory used	89
10.4	Changes to existing Raven files	89
10.5	Building	91
10.6	Getting MODFLOW 6	91
10.7	Adding a new exchange	91
10.8	Adding a grid type	92
10.9	Testing procedure	92
10.10	Limitations and future work	92

A	Reference tables	93
A.1	Parameter classes	93
A.2	Aquifer class parameters (:AquiferClasses, .rvp)	94
A.3	Typical values by material	94
A.4	Aquifer profile parameters (:AquiferProfileParameters, .rvp)	94
A.5	Layer types (:AquiferProfiles, .rvp)	95
A.6	Commands	96
A.7	Options	97
A.8	Exchange algorithms	98
A.9	Solver settings (sim.ims)	99
A.10	Output: GWBudget.csv columns	99
A.11	Hotstart blocks (solution.rvc)	100
A.12	Error messages	100
A.13	Parameter index	102
A.14	Notation	103
	Bibliography	104

CHAPTER 1

Introduction and overview

This chapter explains what the coupling does, how its parts fit together, and what a Raven user needs to know before starting.

1.1 The problem this solves

Raven [2] simulates the water cycle of a watershed: rain and snow, soil moisture, evaporation, runoff and flow in rivers and lakes. It divides a watershed into subbasins, and each subbasin into Hydrologic Response Units (HRUs) — areas that behave alike (for example, forest on till, or a wetland). Raven describes groundwater with simple “buckets” (linear reservoirs): water enters, and leaves at a rate proportional to what is stored. Buckets are fast and easy to calibrate, but they cannot say *where* groundwater flows, how deep the water table is at a well, how a pumping well lowers nearby streamflow, or whether a stream gains water from the aquifer or loses water to it.

MODFLOW 6 [7] is the U.S. Geological Survey’s groundwater model and the international standard for those questions. It divides the underground into a three-dimensional grid of cells and calculates the water level (the *head*) in every cell from the physics of flow through soil and rock. But MODFLOW has no snow, soil or evaporation processes of its own; it needs to be told how much water reaches the water table.

The coupling joins the two so that each does what it is good at: Raven calculates everything at and near the land surface; MODFLOW calculates the saturated groundwater below; and the two exchange water every day.

In plain words

Raven decides how much water soaks down to the water table. MODFLOW moves that water sideways and downwards through the aquifer. Wherever the aquifer overflows — into rivers, wetlands, lakes or back to the soil — MODFLOW hands the water back to Raven. Pumping wells and other stresses act on MODFLOW’s side. Both models keep exact accounts, so not a drop is created or lost at the boundary between them.

1.2 Two ways to use the coupling

Most users are in one of two situations (Figure 1.1).

- **You already have a MODFLOW 6 model** — perhaps one calibrated over years. Raven connects to it and uses it as it is: same grid, same layers, same wells and boundaries (Chapter 4).
- **You have a Raven model but no groundwater model.** Raven builds one for you from your HRUs and a short description of the soil and rock layers under them (Chapter 3).

Everything else — how water is exchanged, how the accounts are kept, the outputs and the tests — is the same in both cases.

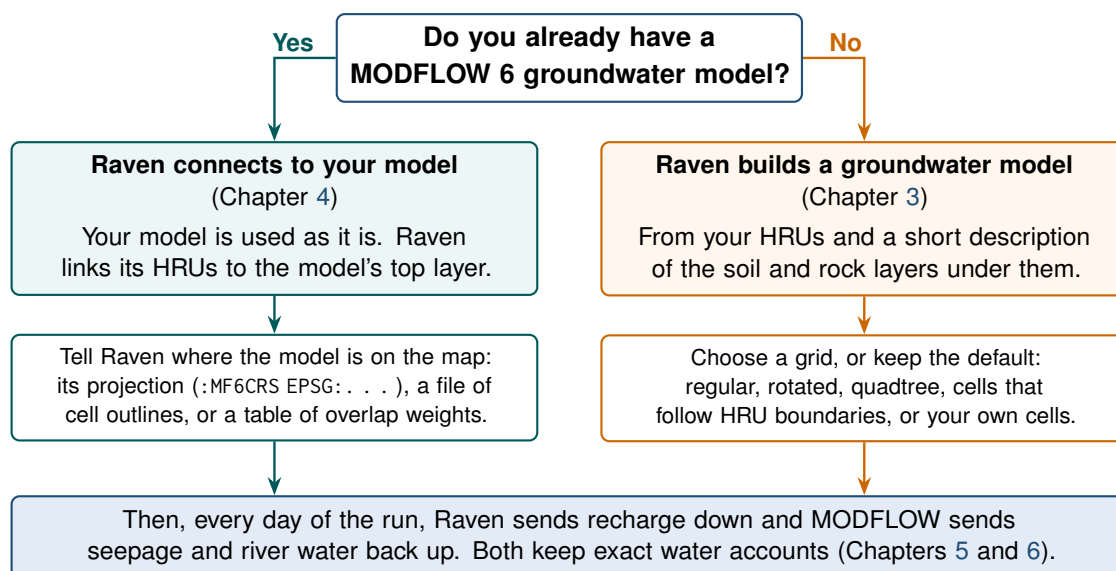


Figure 1.1: Which chapter to start from: connecting Raven to an existing MODFLOW 6 model, or letting Raven build one.

1.3 How to read this manual

Table 1.1: Where to find what.

If you want to...	Read
understand the idea	this chapter and Chapter 2 (groundwater in plain terms)
connect Raven to your MODFLOW model	Chapter 4; example <code>Liard_existing_model</code>
let Raven build a groundwater model	Chapter 3; example <code>Liard_groundwater</code>
know what water moves where, and how	Chapters 5 and 6
look up a command or an output column	Chapter 7 and Appendix A
set up, calibrate or fix a model	Chapter 8
see how the code was tested	Chapter 9
change the code	Chapter 10

Every chapter starts with a box *In plain words* that says what it covers without technical terms; technical words are explained in the glossary below.

1.4 What the user does — and what Raven does

The user works only with Raven’s familiar text files:

1. marks which HRUs have groundwater underneath (`AQUIFER_PROFILE` in the `.rvh` file);
2. describes the aquifer materials and their layering (`.rvp` file);
3. gives the map of HRUs (a GeoJSON file) and, optionally, land-surface and bedrock maps, rivers, wells, monitoring wells and boundaries (a new `.rvg` file);
4. switches the coupling on (`:GroundwaterModel MODFLOW6` in the `.rvi` file).

Raven then does everything else automatically: it lays a grid over the HRUs, works out how much of each HRU lies in each grid cell, builds the layers from the land surface down to bedrock, writes a standard MODFLOW 6 model, starts MODFLOW, exchanges water every time step, and writes results next to its usual outputs. MODFLOW is used as a *library* (a program part that another program can call); Raven talks to it through MODFLOW’s official programming interface [5]. No MODFLOW source code is changed.

1.5 Main features

- Any number of *aquifer profiles* (layer stacks), each with any number of layers; layer types aquifer, confined aquifer, aquitard (confining layer) and aquiclude.
- Land surface and bedrock from raster maps; layers that thin out against bedrock are handled automatically.
- Coupling to an existing MODFLOW 6 model: Raven links its HRUs to the model’s top active layer and leaves the model intact, with built-in map projections, unit conversion and safeguards against double counting (Chapter 4).
- Five grid options: the default regular grid, rotated and refined regular grids, quadtree grids, meshes whose cells follow HRU boundaries, and imported cells; layers can be connected by elevation (Section 3.4).
- Water exchanges: recharge (with optional delay), groundwater seepage at the land surface, rivers (gaining and losing), evaporation from a shallow water table, wetlands and lakes held in Raven stores, Raven reservoirs, pumping and injection wells, head-dependent and fixed-head boundaries.
- Cells can dry out and rewet; areas of aquifer can disconnect and reconnect by themselves.
- Monitoring wells: simulated heads are written out and scored against observations like streamflow, so the groundwater parameters can be calibrated (for example with Ostrich).
- Hotstart (continuing a run from where another ended), ensembles, NetCDF output of the whole head field.
- Exact water accounting on both sides, checked automatically by an audit tool supplied with the code.

1.6 Glossary

Table 1.2: Glossary.

Term	Meaning
Active cell	A cell that MODFLOW calculates. Cells outside the aquifer, or switched off in the model, are inactive.
API, library	MODFLOW 6 comes as a program library (<code>libmf6</code>) that Raven loads and controls step by step, instead of running it as a separate program.
Aquiclude	A layer that, for practical purposes, does not let water through.
Aquifer	A soil or rock layer that holds and passes on useful amounts of groundwater (gravel, sand).
Aquitard	A layer that lets water through only slowly (clay, till); water leaks through it.

Table 1.2: Glossary (continued).

Term	Meaning
Auxiliary variable	An extra column of numbers attached to each entry of a MODFLOW package — here, for example, the Raven subbasin that a river cell belongs to.
Bedrock	The base of the groundwater system in this model; below it no flow is simulated.
Cell	One box of the MODFLOW grid. A <i>column</i> is the stack of cells at one map location.
Conductance	How easily water passes a connection, in m^2/d ; flow = conductance $\times$ difference in water level.
Confined / unconfined	An unconfined aquifer has a free water table; a confined one is completely full and under pressure.
Converged	MODFLOW solves each day by repeated improvement; a day has <i>converged</i> when the answer stops changing within a small tolerance.
Coupled HRU	An HRU that has groundwater underneath it in the model and so exchanges water with MODFLOW.
DIS, DISV, DISU	MODFLOW’s three ways of describing a grid: rows and columns; layers of any-shaped cells; fully free cells.
EPSG code	A standard number for a map projection, e.g. EPSG:32610 is UTM zone 10 North. Used with :MF6CRS.
Existing-model mode	Using the coupling with a MODFLOW 6 model you already have (Chapter 4).
Grid	The set of cells MODFLOW calculates in: squares of one size by default, or other shapes (Section 3.4).
Head (h)	The level to which water rises in a well, in metres above sea level. Water flows from high to low head.
Hotstart, restart	Starting a run from the saved end state (solution.rvc) of an earlier run, so a long run can be split or continued.
HRU	Hydrologic Response Unit: Raven’s basic land unit, an area that behaves alike.
Hydraulic conductivity (K)	How easily water moves through a material, in m/d .
Leakance	How easily water passes a thin layer (a riverbed, the ground surface), per square metre, in $1/\text{d}$.
Linkage, overlap	How much of each HRU lies in each grid cell. It decides where each HRU’s water goes.
Newton method	The way MODFLOW solves its equations here; it copes well with cells drying out and filling up again.
Overlap weight	The share of a cell covered by an HRU (0 to 1); a table of them can replace the map of cells (:OverlapWeights).
Package	One component of a MODFLOW model, in its own file: wells (WEL), rivers (RIV), recharge (RCH) and so on.
Pass-through cell	A very thin cell where a layer is missing; water passes straight through it from the layer above to the layer below.
Profile	A stack of aquifer layers assigned to HRUs (used when Raven builds the model).
Projection	The rule that turns positions on the round Earth into flat map coordinates in metres; MODFLOW models use one, usually without saying which.

Table 1.2: Glossary (continued).

Term	Meaning
Quadtree, mesh	Grids with cells of different sizes: a quadtree cuts squares into four where detail is wanted; a mesh (here) has cells shaped to follow HRU boundaries.
Recharge	Water that reaches the water table from above.
Seepage	Groundwater flowing out at the land surface where the water table reaches it.
Specific storage (S_s)	Water released per m^3 of aquifer per metre of head fall when the aquifer stays full [1/m].
Specific yield (S_y)	Water released per m^2 of water table per metre of fall when the aquifer drains [-].
Stage	Water level in a river, lake or reservoir [m].
Steady state	A calculation as if conditions never changed — the long-term balance. Often used for the first period of a model.
Stress period	A stretch of time in a MODFLOW model (often a month) during which inputs such as pumping rates stay the same.
Time step	One step of the calculation; in the coupling, one Raven time step (usually a day).
Transmissivity (T)	K times saturated thickness, in m^2/d .
Water table	The top of the saturated zone in an unconfined aquifer.
Working copy	The copy of an existing MODFLOW model that Raven runs, in output/mf6/; your own files are never changed.
XT3D	An optional, more exact way for MODFLOW to calculate flow between cells of odd shapes; slower and off by default.

1.7 The big picture

1.7.1 Who owns which water

Every drop of water in the coupled system belongs to exactly one of the two models at any time:

- **Raven** owns snow, canopy, soil and vadose-zone stores, depressions and wetlands, rivers, lakes and reservoirs.
- **MODFLOW** owns the saturated groundwater below the *model top*. By default the model top is the land surface (as in GSFLOW [8]); users with physically based soil layers can lower it with SOIL_ZONE_DEPTH.

Water moves between the two only through the exchanges described in Chapter 5, and each exchange is recorded on both sides. In a coupled HRU, Raven’s GROUNDWATER store is a *pass-through*: whatever enters it during a day is handed to MODFLOW as recharge that day.

1.7.2 The pieces of the software

Figure 1.2 shows the parts. The *model builder* runs once at the start: it reads the inputs and writes a standard MODFLOW 6 model. The *exchange* runs every time step.

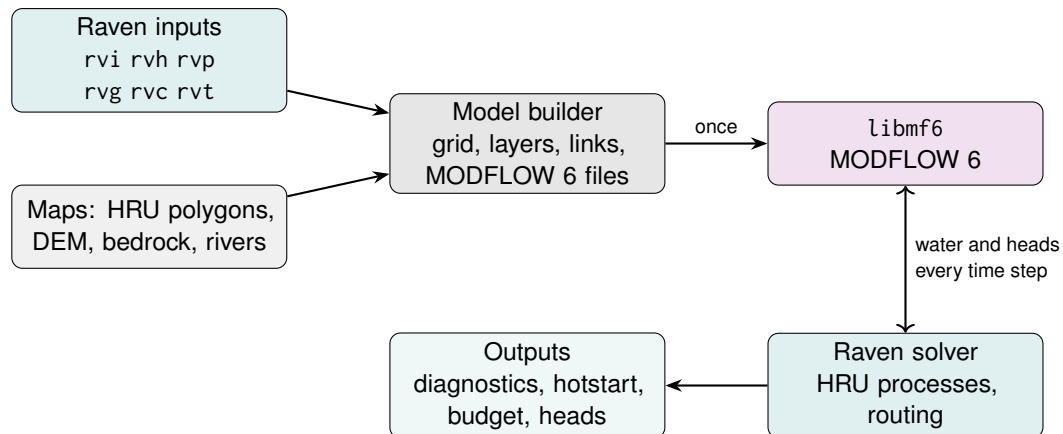


Figure 1.2: The software pieces. The builder runs once per simulation (or per ensemble member); the exchange runs every time step.

1.7.3 One day in the life of the coupled model

The following happens every time step (one day in the examples); Section 6.4 gives the exact algorithm.

1. **Raven’s land-surface day.** Raven calculates snowmelt, infiltration, evaporation and flow between its soil layers. In coupled HRUs, water that percolates past the soil ends up in GROUNDWATER.
2. **Recharge handed over.** The water in each HRU’s GROUNDWATER is spread over the grid cells under that HRU, in proportion to how much of the HRU lies in each cell.
3. **Stresses set.** River levels (from Raven’s rivers at the start of the day), well pumping rates, boundary heads, wetland and lake levels, and the evaporation demand not met by Raven are passed to MODFLOW.
4. **MODFLOW’s day.** MODFLOW calculates new heads everywhere, including the flows to and from rivers, drains, wells and boundaries.
5. **Water handed back.** Groundwater that seeps out at the land surface is added to a Raven store of the HRUs above; groundwater that discharges to rivers is added to the river inflow of the subbasin; river losses are taken from the river; wetland and lake exchanges change those Raven stores.
6. **Raven’s river day.** Raven routes water down the river network, now including the groundwater contributions. Reservoirs compute their seepage using the aquifer head beneath them.
7. **Accounts.** Both models update their water balances, and the groundwater outputs are written.

1.7.4 Design principles

1. **Raven is the single user interface.** All inputs are Raven files; MODFLOW files are generated.
2. **Standard MODFLOW only.** Only standard MODFLOW 6 packages are used; the generated model can be opened in FloPy or ModelMuse.
3. **Each fact is entered once.** Geometry comes from maps, hydrogeology from profiles, time-varying stresses from time series.

4. **Generic.** Nothing is specific to a basin; any Raven model can couple any subset of its HRUs.
5. **Exact accounting.** Every exchanged volume is computed once and applied identically on both sides.
6. **No surprise for existing users.** Models without groundwater give bit-for-bit the same results as Raven 4.15 (the version this coupling is built on); Raven still builds and runs without MODFLOW installed.

1.8 Scope and assumptions

- **What is simulated by MODFLOW:** saturated groundwater flow (Darcy flow, storage change) in a layered aquifer system under the coupled HRUs, including dry and rewetting cells.
- **What stays in Raven:** everything above the model top — snow, canopy, soil and unsaturated zone, surface runoff, depressions, rivers, lakes and reservoirs — and all HRUs without an aquifer profile.
- **Water density and temperature** are constant; there is no salt-water or heat transport.
- **The grid** is regular (square cells); the model area is the part of the rectangle around the coupled HRUs that those HRUs cover.
- **Time:** both models use Raven’s time step (daily or shorter). Exchanges are explicit between the models at the step level, but every head-dependent exchange is implicit inside MODFLOW.

1.9 When to use the coupling — and when not to

Use it when the question involves the *spatial* behaviour of groundwater: water-table depth at a location, effects of pumping on streams, gaining and losing reaches, groundwater flow between subbasins, wetlands fed by groundwater, drought recession of large aquifers, or when head measurements should constrain the model. Raven’s conceptual baseflow remains the better choice for fast screening, very large ensembles, or basins without a significant aquifer.

1.10 What changes for an existing Raven model

- Nothing, unless `:GroundwaterModel MODFLOW6` is added: results stay bit-for-bit identical.
- With the coupling on, coupled HRUs lose their conceptual deep-groundwater behaviour (their `GROUNDWATER` store becomes a pass-through), and their baseflow now comes from the aquifer via seepage and river exchange.
- Run times increase: the 20-year Liard run (3 252 active columns) takes about 15 minutes instead of 35 seconds on one processor core.

1.11 Files at a glance

Table 1.3: Input and output files of the coupling.

File	Role in the coupling
.rvi	switch on (:GroundwaterModel MODFLOW6); route recharge into GROUNDWATER
.rvh	AQUIFER_PROFILE per HRU; reservoirs with :SeepageParameters
.rvp	aquifer classes, profiles and profile parameters
.rvg	geometry (HRU polygons, maps, rivers), grid, wells, monitoring wells, boundaries, options
.rvt	well rates, boundary heads, observed heads
.rvc	initial heads; hotstart blocks written by Raven
GeoJSON, ASCII grids	HRU polygons, river lines, boundary shapes; DEM and bedrock maps
<output>/mf6/	generated MODFLOW 6 model and its own outputs

CHAPTER 2

Groundwater concepts

This chapter introduces the physical ideas and vocabulary used in the rest of the document. Readers familiar with groundwater hydrology can skim it; everyone else will find every later equation built from the few ideas presented here.

2.1 Head

The *head* h at a point is the elevation to which water would rise in a small well drilled to that point, in metres above a common datum (here, the same datum as the digital elevation model). Groundwater always flows from higher head to lower head. In an unconfined aquifer the head is the elevation of the water table; the *depth to water* is land-surface elevation minus head.

2.2 Darcy's law

The flow of water through a porous material is proportional to the drop in head and to the cross-section, and inversely proportional to the distance travelled:

$$Q = K A \frac{h_1 - h_2}{L}, \quad (2.1)$$

where Q is the flow [m^3/d], K the hydraulic conductivity [m/d], A the area the water passes through [m^2], $h_1 - h_2$ the head difference [m] and L the distance [m].

Worked example: Darcy's law

Gravel with $K = 50 \text{ m/d}$, a 2000 m wide and 10 m thick strip ($A = 20\,000 \text{ m}^2$), heads 101 m and 100 m, 2000 m apart: $Q = 50 \times 20\,000 \times 1/2000 = 500 \text{ m}^3/\text{d}$.

2.3 Transmissivity and conductance

Because K , A and L always appear together, it is convenient to combine them into a *conductance* $C = KA/L$ [m^2/d], so that $Q = C(h_1 - h_2)$. Every exchange in this document — between two grid cells, between an aquifer and a river, a drain, a wetland — is written in this form: **flow = conductance $\times$ head difference**. For a layer of saturated thickness b , the *transmissivity* is $T = Kb$ [m^2/d].

Between two neighbouring cells with different conductivities K_1, K_2 , the conductance is the series combination (harmonic mean), because water must pass through both halves in turn:

$$C_{12} = \frac{1}{\frac{L_1}{K_1 A} + \frac{L_2}{K_2 A}}. \quad (2.2)$$

Worked example: harmonic mean

Two 2000 m cells, 10 m thick and 2000 m wide ($A = 20\,000 \text{ m}^2$, $L_1 = L_2 = 1\,000 \text{ m}$ to the shared face), gravel $K_1 = 50 \text{ m/d}$ next to till $K_2 = 0.2 \text{ m/d}$: $C_{12} = 1/(1\,000/(50 \cdot 20\,000) + 1\,000/(0.2 \cdot 20\,000)) = 1/(0.001 + 0.25) = 3.98 \text{ m}^2/\text{d}$. The low-conductivity material controls the flow.

2.4 Storage

When the head in an aquifer falls, water is released from storage.

- In an *unconfined* aquifer the water table drops and pores drain. The water released per square metre per metre of fall is the *specific yield* S_y (typically 0.05–0.30).
- In a *confined* (full) aquifer, water is released only by the slight expansion of water and compression of the grains; per cubic metre of aquifer per metre of head fall this is the *specific storage* S_s (typically 10^{-6} – 10^{-4} 1/m).

Worked example: storage change

A 4 km² cell of gravel ($S_y = 0.25$) whose water table falls 0.1 m in a day releases $4 \times 10^6 \times 0.25 \times 0.1 = 100\,000$ m³. The same fall in a 20 m thick confined sand ($S_s = 10^{-5}$ 1/m) releases $4 \times 10^6 \times 20 \times 10^{-5} \times 0.1 = 80$ m³ — over a thousand times less.

2.5 The groundwater flow equation

Combining Darcy’s law with conservation of water (what flows in, minus what flows out, equals the change in storage) gives the equation MODFLOW solves. For one cell n with neighbours m :

$$\sum_m C_{nm} (h_m - h_n) + \sum_b q_{b,n} = S_n \frac{h_n^{t+\Delta t} - h_n^t}{\Delta t}, \quad (2.3)$$

where all heads are those at the end of the step, $t + \Delta t$ (MODFLOW’s implicit scheme, which is stable for any step length), the first sum is flow from the neighbours, $q_{b,n}$ are the boundary flows of the cell (recharge, rivers, drains, wells, ...; positive into the aquifer), and S_n is the storage capacity of the cell (S_y times area when unconfined, S_s times volume when confined). Writing this for every cell gives a large set of equations for the unknown heads, which MODFLOW solves every time step.

In plain words

Each cell is a small water account: inflows from neighbours and boundaries, minus outflows, equals the change of water stored. MODFLOW finds the heads that make every cell’s account balance at the same time.

2.6 Head-dependent boundaries

Many exchanges depend on the head: a river gives water to the aquifer when the river is higher than the water table, and receives water when it is lower. MODFLOW writes such a boundary as

$$q_b = C_b (h_b - h_n), \quad (2.4)$$

where h_b is the boundary’s water level. Because q_b depends on the unknown head h_n , MODFLOW solves for the boundary flow together with the heads; this keeps the solution stable even with large conductances. In MODFLOW’s internal notation each boundary contributes two numbers per cell, HCOF and RHS, and its flow is

$$q_b = \text{HCOF} \cdot h_n - \text{RHS}. \quad (2.5)$$

The coupling uses exactly this formula, with MODFLOW’s own HCOF and RHS, to compute every exchanged flow. It is the same formula MODFLOW uses for its own budget, so the numbers exchanged with Raven are exactly the numbers MODFLOW accounts for.

2.7 Water table and potentiometric surface

In an unconfined aquifer the head is the water-table elevation and the pores above it are partly empty. In a confined aquifer, sealed above by a low-permeability layer, water rises in a well above the top of the aquifer; the imaginary surface joining these levels is the *potentiometric surface*. The same aquifer can be confined in one place and unconfined in another, or change over time as heads fall; MODFLOW’s *convertible* layers handle this switch automatically by using S_s while full and S_y once the head falls below the top of the layer.

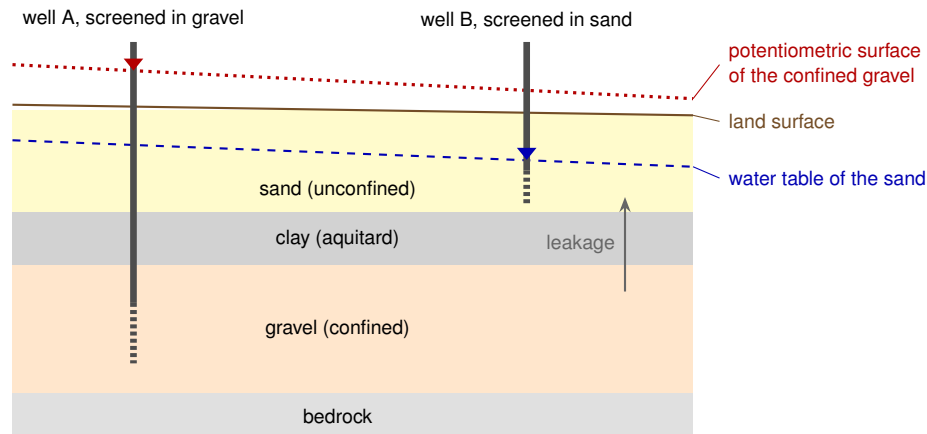


Figure 2.1: The same place can have two different heads: the water table of the upper sand and the higher potentiometric surface of the confined gravel below the clay. Water leaks upward through the clay because the gravel head is higher.

2.8 Three kinds of boundary conditions

Table 2.1: Three kinds of boundary conditions.

Kind	Meaning	In this coupling
Specified flow	the flow is known (it does not depend on the head)	recharge, wells, reservoir seepage
Specified head	the head is known; the flow is whatever is needed	fixed-head boundaries (CHD)
Head-dependent flow	flow = conductance $\times$ (boundary level – head)	rivers, seepage drains, wetlands, lakes, general-head boundaries, water-table ET

Head-dependent boundaries are the heart of a coupled model: they let the aquifer and the surface water “negotiate” the exchange according to the water levels on both sides.

2.9 The Newton method in simple terms

The flow equations are non-linear: in an unconfined layer, transmissivity depends on the saturated thickness, which depends on the unknown head. MODFLOW therefore solves them iteratively: guess heads, compute the imbalance of every cell, correct the heads, and repeat until the corrections are smaller than a tolerance (here 0.00001 m, i.e. 0.01 mm). The Newton–Raphson method computes each correction from the slope of the imbalance with respect to the heads, which converges quickly even when cells are nearly dry. Each repetition is an *outer iteration*; GWBudget.csv reports how many were needed each step (typically 3–10 in the Liard example).

2.10 A cell's water account, with numbers

Worked example: one cell for one day

A 4 km² gravel cell receives 8 000 m³ recharge; it gets 1 500 m³ from an up-gradient neighbour and passes 2 500 m³ to a down-gradient one; its drain removes 4 000 m³. Inflow – outflow = 8 000 + 1 500 – 2 500 – 4 000 = 3 000 m³ are stored, so with $S_y = 0.25$ the water table rises by $3\,000 / (4 \times 10^6 \times 0.25) = 3$ mm. MODFLOW finds heads for which every cell's account balances like this one at the same time.

2.11 Units and conversions

Table 2.2: Units and conversions.

Quantity	Conversion
depth of water in an HRU to volume	$V [\text{m}^3] = x [\text{mm}] \times A [\text{km}^2] \times 1000$
flow	$1 \text{ m}^3/\text{s} = 86\,400 \text{ m}^3/\text{d}$
recharge rate	$1 \text{ mm}/\text{d over } 1 \text{ km}^2 = 1\,000 \text{ m}^3/\text{d}$
hydraulic conductivity	$1 \text{ m}/\text{d} \approx 1.16 \times 10^{-5} \text{ m}/\text{s}$
leakance	conductance per area, d^{-1} ; $C = L \times \text{area}$

CHAPTER 3

Building the groundwater model

Raven's HRUs are irregular areas; MODFLOW needs a regular three-dimensional grid. This chapter follows the model builder step by step. It shows which HRUs are coupled, how their polygons are placed on a map and covered with a grid, and how much of each HRU falls in each cell. It then shows how the layers are built from the land surface down to bedrock, which MODFLOW files are written, and how MODFLOW solves each step.

3.1 From HRUs to a groundwater grid

3.1.1 Which HRUs are coupled

An HRU is *coupled* when its `AQUIFER_PROFILE` in the `.rvh` file names a profile defined in the `.rvp` file. HRUs with `[NONE]` (or an empty entry) keep Raven's usual behaviour. Disabled HRUs are never coupled (Raven prints a warning). We write $\mathcal{K}$ for the set of coupled HRUs and $p(k)$ for the profile of HRU k .

Raven checks two rules at start-up:

- No Raven process may take water *out of* GROUNDWATER in a coupled HRU (MODFLOW owns that water); Raven stops with an error naming the process.
- If a process sends water *into* GROUNDWATER in an *uncoupled* HRU, that water leaves the model; Raven warns so the user can restrict the process with `:->Conditional`.

3.1.2 Map projection

The HRU polygons are read from a GeoJSON file. GeoJSON coordinates are longitude λ and latitude ϕ in degrees, which cannot be used directly for areas and distances. Raven converts them to metres with the *Lambert azimuthal equal-area* projection [9], centred on the average centroid (λ_0, ϕ_0) of the coupled HRUs, on a sphere of radius $R = 6\,371\,007.181$ m:

$$k' = \sqrt{\frac{2}{1 + \sin \phi_0 \sin \phi + \cos \phi_0 \cos \phi \cos(\lambda - \lambda_0)}}, \quad (3.1)$$

$$x = R k' \cos \phi \sin(\lambda - \lambda_0), \quad (3.2)$$

$$y = R k' [\cos \phi_0 \sin \phi - \sin \phi_0 \cos \phi \cos(\lambda - \lambda_0)]. \quad (3.3)$$

The reverse conversion (used to look up map values at cell locations) is

$$\rho = \sqrt{x^2 + y^2}, \quad c = 2 \arcsin \frac{\rho}{2R}, \quad (3.4)$$

$$\phi = \arcsin \left(\cos c \sin \phi_0 + \frac{y \sin c \cos \phi_0}{\rho} \right), \quad (3.5)$$

$$\lambda = \lambda_0 + \text{atan2}(x \sin c, \rho \cos \phi_0 \cos c - y \sin \phi_0 \sin c). \quad (3.6)$$

In plain words

This projection keeps areas correct, which is what matters for water volumes: one square kilometre on the map is one square kilometre on the ground, anywhere in the model area.

3.1.3 The grid

By default a square grid with cell size Δ (:GridCellSize, default 1 000 m) covers the rectangle around all coupled HRUs. Rows are numbered from north to south and columns from west to east. The plan-view cell (a whole column of layers) is indexed i ; its lower-left corner is $(x_0 + c\Delta, y_0 + (n_r - 1 - r)\Delta)$ for row r and column c . Rotated grids, grids refined around rivers and wells, quadtree grids, meshes that follow HRU boundaries and imported cells are described in Section 3.4; everything below applies to all of them, with cell i a polygon of area A_i .

3.1.4 Polygon areas

The area of a polygon with corners $(x_1, y_1), \dots, (x_n, y_n)$ is given by the shoelace formula

$$A = \frac{1}{2} \sum_{j=1}^n (x_j y_{j+1} - x_{j+1} y_j), \quad (x_{n+1}, y_{n+1}) = (x_1, y_1). \quad (3.7)$$

The result is positive when the corners run counter-clockwise. Raven orients outer boundaries counter-clockwise and holes clockwise, so the area of a polygon with holes is simply the sum over all its rings.

Worked example: shoelace formula

A square from (0,0) to (2,2) km: $\frac{1}{2}[(0 \cdot 0 - 2 \cdot 0) + (2 \cdot 2 - 2 \cdot 0) + (2 \cdot 2 - 0 \cdot 2) + (0 \cdot 0 - 0 \cdot 2)] = \frac{1}{2}(0 + 4 + 4 + 0) = 4 \text{ km}^2$.

3.1.5 How much of each HRU lies in each cell

For every HRU polygon P_k and every cell Ω_i it touches, Raven cuts the polygon to the cell rectangle with the Sutherland–Hodgman clipping algorithm [10] and measures the area of what remains:

$$A_{k,i} = |P_k \cap \Omega_i|. \quad (3.8)$$

For rectangular cells (regular and quadtree grids) the polygon is clipped by the cell rectangle; for convex cells (HRU meshes) by the cell polygon; a concave imported cell is split into triangles and clipped by each. The clipping works one cell edge at a time: walking around the polygon, it keeps the corners that are inside the edge and adds a new corner wherever the polygon crosses the edge. After four edges, what is left is the part of the polygon inside the cell; the shoelace formula gives its area. (This gives the correct area even for concave polygons.)

Worked example: overlap areas

An HRU of 10 km² lies over three 2 km cells, covering 4.0, 3.5 and 2.5 km² of them. Its overlaps are $A_{k,1} = 4.0$, $A_{k,2} = 3.5$, $A_{k,3} = 2.5 \text{ km}^2$, and the fractions of the HRU in each cell are 0.40, 0.35 and 0.25. Every flow from this HRU to the grid is split 40/35/25.

3.1.6 Active cells and their profile

A column is *active* when the coupled HRUs cover at least a fraction $f_{\min}$ (:MinCellCoverage, default 0.25) of it:

$$a_i = \sum_{k \in \mathcal{K}} A_{k,i} \geq f_{\min} \Delta^2. \quad (3.9)$$

An active column takes the profile of the HRU with the largest overlap (the *dominant* HRU), $p(i) = p(\arg \max_k A_{k,i})$. Fluxes still use the exact overlaps, so a column shared by two HRUs receives water from both.

The table of overlaps $A_{k,i}$ over active columns is called the *linkage*. It is the single source of truth for moving water between HRUs and cells in both directions (Sections 5.1–5.5).

Watch out

The polygon area of an HRU need not equal its AREA in the .rvh file (Raven prints a warning when they differ by more than 5%). This does not break the water balance: volumes are always computed from the .rvh area and split by overlap *fractions*.

3.1.7 The linkage cache

Computing the linkage requires reading and clipping every polygon. The result is saved in <output>/mf6/linkage.cache (or the file named by :LinkageCache). The cache is labelled with a fingerprint (an FNV-1a hash) of the GeoJSON file together with the ID field, the cell size and the list of coupled HRUs; if any of these change, Raven recomputes the linkage. Numbers are written with 17 significant digits, so a run that reads the cache gives exactly the same results as one that computed it.

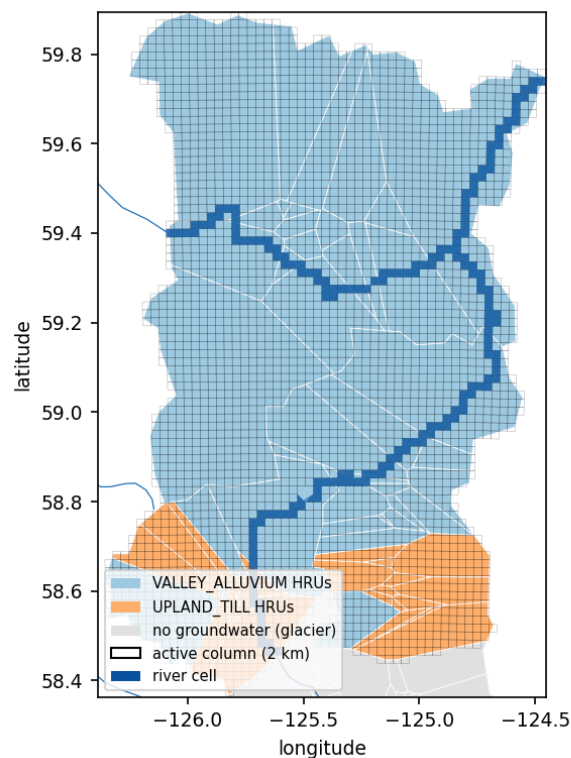


Figure 3.1: Liard example: coupled HRUs (colour = aquifer profile), active 2 km columns and river cells.

3.2 From one HRU to its cells: a complete example

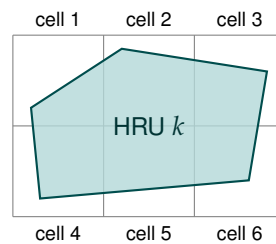


Figure 3.2: An HRU polygon over six grid cells. Clipping gives its area in each cell.

Worked example: an HRU through the whole chain

Suppose the HRU in Figure 3.2 has an `.rvh` area of 3.2 km^2 , 2 km cells, and clipping gives overlaps (km^2) 0.45, 0.55, 0.30 in cells 1–3 and 0.60, 0.85, 0.45 in cells 4–6 (sum 3.20).

1. *Activation.* If other coupled HRUs fill the rest of these cells, all six have coverage above 25% and are active.
2. *Recharge.* Raven’s percolation puts 1.5 mm into GROUNDWATER: $V = 0.0015 \times 3.2 \times 10^6 = 4800 \text{ m}^3$. Cell 5 receives $4800 \times 0.85/3.20 = 1275 \text{ m}^3$, i.e. $1275/4 \times 10^6 = 0.32 \text{ mm}$ over the whole cell (plus whatever other HRUs deliver).
3. *Seepage back.* If cell 5 seeps $10\,000 \text{ m}^3$ in the day and our HRU covers 0.85 of the 4 km^2 covered by all HRUs in that cell, the HRU receives $10\,000 \times 0.85/4.0 = 2\,125 \text{ m}^3$, i.e. $2\,125/3.2 \times 10^6 \times 1000 = 0.66 \text{ mm}$ in its lowest soil layer.
4. *Water-table depth in the outputs.* `GWHRUState.csv` reports the overlap-weighted mean head of the six cells.

3.3 Preparing the geometry

3.3.1 What the HRU GeoJSON must contain

A FeatureCollection of Polygon or MultiPolygon features in longitude/latitude (EPSG:4326), each with a property holding the HRU ID of the `.rvh` file. Holes are allowed. Several features may share an ID (their areas add up).

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "properties": { "HRU_ID": 2736 },
      "geometry": {
        "type": "Polygon",
        "coordinates": [
          [
            [ -124.91, 59.22 ],
            [ -124.80, 59.22 ],
            [ -124.80, 59.30 ],
            [ -124.91, 59.30 ],
            [ -124.91, 59.22 ]
          ]
        ]
      }
    }
  ]
}
```

Only HRUs with an `AQUIFER_PROFILE` need polygons; others may be present and are ignored.

3.3.2 Raster maps

ESRI ASCII grids in longitude/latitude, for example:

```
ncols 125
nrows 115
xllcorner -126.6
yllcorner 58.1
cellsize 0.02
NODATA_value -9999
612.3 615.9 ...
```

Values are interpolated bilinearly between raster cell centres, ignoring no-data cells; each model cell takes the mean of 5×5 such samples. Points more than one raster cell outside the map count as no-data.

3.3.3 Choosing the cell size

The number of active columns is roughly the coupled area divided by Δ^2 : for the Liard example's 13 000 km², $\Delta = 2$ km gives about 3 300 columns, $\Delta = 1$ km about 13 000, $\Delta = 500$ m about 52 000. Run time grows roughly in proportion. Choose Δ so that the narrowest valleys and the spacing of wells and monitoring wells span at least two or three cells. The water balance is exact for any cell size (verified for 1, 2 and 4 km, Chapter 9).

3.4 Grid types

In plain words

The grid is the set of boxes (cells) in which MODFLOW calculates. By default Raven uses squares of one size. It can instead turn the grid to follow a valley, use smaller cells near rivers and wells, shape the cells to follow your HRU boundaries, or take cells made in another program. Smaller cells give more detail where it matters, at the cost of longer runs. Figure 3.3 shows the same area on four grids.

The grid need not be a regular array of squares. Five options cover most needs (Table 3.1, Figure 3.3). All of them are built from the same inputs and keep the water accounts exact, and each can be combined with every exchange, boundary, output and restart.

Table 3.1: Grid options.

Option	MODFLOW	XT3D default	Use it when
Regular (default)	DIS	no	a first model; uniform cells are enough
Regular, rotated and/or refined	DIS	no	a valley has a clear direction; finer cells near rivers or wells are wanted and whole rows and columns may be refined
QUADTREE	DISV	no	fine cells only where they matter (rivers, wells, areas of interest) with few cells elsewhere
HRU_MESH	DISU	no	cells should follow HRU boundaries, so each cell belongs to one HRU
FILE (imported cells)	DISU	no	a grid made elsewhere (FloPy, gridgen, ModelMuse, QGIS) should be used
:LayerConnection ELEVATION (any grid)	DISU	no	layers of neighbouring profiles should connect where they are at the same elevation, not by layer number

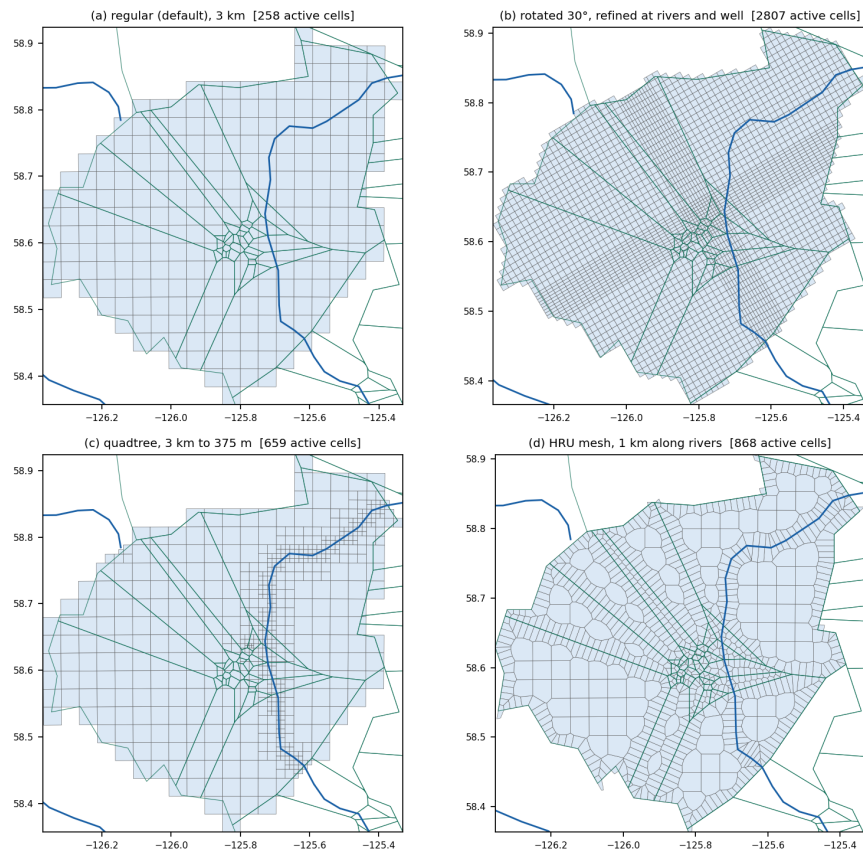


Figure 3.3: The same model (Liard subbasin 52, HRU boundaries green, rivers blue) on four grids: (a) the default regular grid; (b) rotated 30° and refined along rivers and around a well; (c) a quadtree refined along rivers and around the well; (d) an HRU mesh whose cells follow HRU boundaries, with cells along the rivers.

3.4.1 Regular grids: rotation and variable spacing

:GridRotation θ turns the grid θ degrees counterclockwise, for example to align columns with a valley. The grid is built in its own rotated frame, in which every cell is still a rectangle; HRU polygons, rivers, wells and boundaries are rotated into that frame, and MODFLOW receives the rotation as ANGROT. With :GridRefinement the grid gets variable spacing. Every base column and row that comes within the target size of a refinement feature is divided into equal parts no larger than the target. Neighbouring columns and rows are then halved until the widths change by at most a factor of two from one to the next. Refinement therefore always spans whole columns and rows.

3.4.2 Quadtree grids

A quadtree starts from large squares and cuts a square into four smaller ones wherever finer detail is wanted — along a river, say — repeating until the cells are small enough. With :GridType QUADTREE a square is split into four while it is larger than the target size and a refinement feature lies within half the target size of it. Splitting stops after :QuadtreeMaxLevel halvings (by default, enough to reach the smallest target). The tree is then balanced so that neighbouring cells differ by one level at most. Where a large cell meets two small ones its side carries the extra corner of the small cells (a hanging vertex), so MODFLOW’s DISV input finds every shared face. Because the line joining the centres of a large and a small cell is not perpendicular to their shared face, MODFLOW’s XT3D correction (the full conductance tensor) could be used. It is off by default, for two reasons found in the tests of Chapter 9. The balanced quadtree was as accurate without it (Thiem test). And XT3D — even in its right-hand-side form — slowed the Newton method badly where cells dry and rewet: on the Liard model it took 32 instead of 7 iterations per day, and some days failed. :FlowCorrection XT3D_RHS or XT3D switch it on.

3.4.3 Meshes that follow HRU boundaries

Picture a dot for every cell: each cell is the area closer to its own dot than to any other dot (a *Voronoi* cell). If two dots sit on opposite sides of an HRU boundary, at the same distance from it, the edge between their cells falls exactly on that boundary. Raven places its dots this way, so the cells follow the HRU boundaries. In detail, with :GridType HRU_MESH the seed points are:

1. along every HRU boundary, pairs of seeds mirrored across the boundary at spacing $s_b = \min(\Delta/2, \text{smallest line refinement size (RIVERS, LINES)})$, or the LAKES/POLYGONS size where the boundary passes within one such size of a refinement polygon, and $0.3 s_b$ from it — the face between the two seeds of a pair lies on the boundary;
2. seeds along refinement lines (rivers run through cell centres), in rings around refinement points (wells), and on a staggered lattice of the target size inside and within one size around refinement polygons (lakes); seeds outside the model’s box (e.g. a monitoring well outside the model) are left out;
3. interior seeds on a lattice of spacing Δ , kept only where no other seed is closer than about half the spacing.

Each cell is the part of the plane closer to its seed than to any other. Raven builds it by cutting away, for each neighbouring seed, the half of the plane that is closer to that neighbour. Each face remembers the seed on its other side, so the connections between cells are exact. The line joining two seeds crosses their shared face at right angles, which is the arrangement MODFLOW’s standard method assumes, so no XT3D is needed. Where boundaries meet at sharp corners a cell may cover slivers of two HRUs. The overlaps are computed exactly, as for any other grid, so the water accounts are unaffected. GWModelSummary.txt reports how much of each active cell’s area lies in its main HRU (typically above 99%).

3.4.4 Imported cells

With `:GridType FILE` and `:GridFile cells.geojson ID_FIELD` the cells are the polygons of a GeoJSON file in longitude/latitude, one single-ring polygon per cell, numbered by the ID field (default `CELL_ID`). Cells may be concave, and a cell side may be shared with several smaller cells (T-junctions): neighbours are found wherever edges of two cells lie on the same line — within 0.1% of the edge length, because a straight edge drawn in one map projection bends slightly in another — and overlap. A face made of several pieces gets their total length and their average position and direction. Repeated vertices, which GIS exports often contain, are removed first. MODFLOW receives the grid as DISU with the connection lengths, face widths and face directions computed from the polygons. Imported cells need not be orthogonal; for strongly non-orthogonal cells consider `:FlowCorrection XT3D_RHS` and check the convergence in `GWBudget.csv`.

3.4.5 Connecting layers by elevation

In a layered grid a cell connects sideways only to the cell with the same layer number. When neighbouring columns have different profiles — a gravel layer thinning out next to a till, say — the same layer number can mean different materials at different elevations. `:LayerConnection ELEVATION` writes the grid as DISU and connects every pair of cells of neighbouring columns whose vertical intervals overlap, as staggered horizontal connections (MODFLOW's IHC 2), so water flows between the layers that actually touch. Pass-through cells are removed and the cells above and below them connected directly.

3.4.6 Refinement

```
:GridRefinement RIVERS 250 # along :RiverGeometry lines
:GridRefinement WELLS 100 # around pumping and monitoring wells
:GridRefinement LAKES 250 # in and around every reservoir's lake HRU
:GridRefinement LINES faults.geojson 200 # along any lines
:GridRefinement POLYGONS wetlands.geojson 250 # inside and around polygons
```

Sizes must be smaller than `:GridCellSize`. Refinement applies to regular (variable spacing), quadtree and HRU-mesh grids; an imported grid is used as it is.

LAKES needs no extra file: the outline of the lake HRU of every Raven reservoir (`:Reservoir . . . :HRUID`) is taken from `:HRUGeometry`. Fine cells around a lake place its seepage (`:ReservoirExchange`) and the exchange along its shore more precisely. On a regular grid the rows and columns through the lake are refined; a quadtree splits the squares in and near the lake; an HRU mesh places cells of the target size inside the lake and in a band one cell wide around it, and shortens the spacing of its cells along the shore. POLYGONS works the same way for any outline in a file.

3.4.7 Rivers and boundaries on any grid

On the default grid river and boundary lines are cut into cells as before. On every other grid each piece of a line is cut exactly at the edges of the cells it crosses (rectangles along their sides, other cells along their edges or triangle by triangle). River lengths per cell are therefore exact and do not depend on the cell size. A stretch lying exactly on a shared cell edge is counted once. A boundary line may be drawn along the edge of the modelled area, where the edge cells can be inactive because too little of them is covered. Such a line is attached to the nearest active cell within one cell size, so a boundary drawn on the watershed outline is never lost.

3.4.8 Choosing a grid

Start with the default grid and a cell size of a few kilometres. Refine where heads and exchanges change quickly: along rivers that gain or lose much water, around pumping wells, around lakes and reservoirs that leak to the aquifer, and in wetlands. A quadtree gives the most accuracy per cell; an HRU mesh gives the cleanest HRU–cell linkage; a rotated regular grid is simplest when a valley has one direction. XT3D (:FlowCorrection XT3D_RHS or XT3D) is off by default; use it only for strongly non-orthogonal imported cells, and check that the model still converges well.

3.4.9 Grid outputs

Every run writes `mf6/grid_cells.geojson`: each cell as a longitude/latitude polygon with its number (as in MODFLOW and in the hotstart file), whether it is active, its profile, its dominant HRU and its area — ready to be joined with heads in QGIS. For grids other than regular, `GWHeads.nc` has a `cell` dimension instead of rows and columns, and the hotstart block is written as `:GWHeads nlay 1 ncells`. The grid type, cell sizes, MODFLOW format and HRU fidelity are reported in `GWModelSummary.txt`. Setting the environment variable `RAVEN_GW_PROFILE=1` prints the time taken by each stage of building the model, useful for large grids.

3.5 Building the layers

3.5.1 Land surface, model top and bedrock

For each active column Raven needs three elevations:

Land surface $z_{\text{land},i}$: the average of 25 points (5×5) of the DEM over the cell (:LandSurface); without a DEM, the area-weighted ELEVATION of the HRUs in the cell.

Model top $z_{\text{top},i} = z_{\text{land},i} - d_{\text{sz},p}$, where d_{sz} is the profile's SOIL_ZONE_DEPTH (default 0, so the top is the land surface).

Bedrock $z_{\text{br},i}$: the average of 25 points of the bedrock map (:BedrockSurface); without a map, $z_{\text{land},i} - D_p$ with D_p the profile's DEFAULT_BEDROCK_DEPTH.

A column whose bedrock lies less than 0.5 m below its top has no room for an aquifer and is made inactive.

3.5.2 Stacking the layers

A profile lists its layers from the top down, each with a material (*aquifer class*), a thickness b_ℓ (or TO_BEDROCK) and a type. The bottom of layer ℓ is

$$z_{\text{bot},i}^{(\ell)} = \begin{cases} z_{\text{br},i} & \text{for a TO_BEDROCK layer,} \\ \max(z_{\text{bot},i}^{(\ell-1)} - b_\ell, z_{\text{br},i}) & \text{otherwise,} \end{cases} \quad z_{\text{bot},i}^{(0)} \equiv z_{\text{top},i}. \quad (3.10)$$

Layers never extend below bedrock. Two special cases:

- A layer (other than the first) cut to less than 0.5 m by bedrock becomes a *pass-through* cell: it is given a nominal thickness of 0.01 m and MODFLOW's IDOMAIN = −1, which removes it from the calculation while connecting the cells above and below it directly.
- The model has as many layers as the thickest profile. Where a profile has fewer layers, the missing ones are pass-through cells.

Worked example: one column

Land surface 600 m, bedrock 555 m, profile VALLEY_ALLUVIUM = gravel 15 m, clay till 5 m, sand TO_BEDROCK. Top = 600 m. Layer 1 (gravel): bottom $\max(600 - 15, 555) = 585$ m. Layer 2 (till): bottom $\max(585 - 5, 555) = 580$ m. Layer 3 (sand): bottom 555 m (to bedrock), thickness 25 m. If bedrock were at 582 m instead, layer 2 would end at $\max(580, 582) = 582$ m (3 m thick), and layer 3, which runs to bedrock, would span 582 to 582 m — zero thickness, so it becomes a pass-through cell.

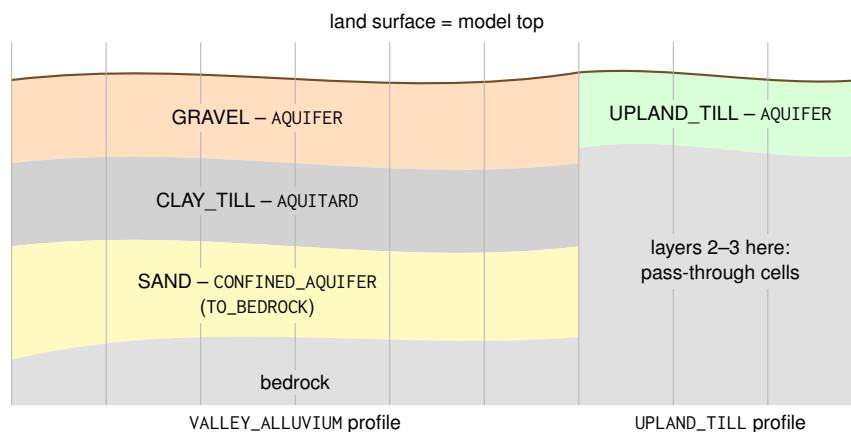


Figure 3.4: Layers are stacked from the model top and clipped at bedrock; the one-layer upland profile is padded with pass-through cells.

3.5.3 Layer types

Table 3.2: Layer types.

Type in the profile	MODFLOW 6 setting	What it means
AQUIFER (UNCONFINED_AQUIFER)	ICELLTYPE=1, ICONVERT=1	water table can be inside the layer: storage by S_y , transmissivity from the saturated thickness
CONFINED_AQUIFER	ICELLTYPE=0, ICONVERT=0	always full: constant transmissivity, storage by S_s
AQUITARD, CONFINING_LAYER	as AQUIFER, with the low K of its class	slow leaky layer, simulated explicitly
AQUICLUDE	IDOMAIN=0	no flow at all through this layer (not allowed as the top layer)

3.5.4 Material properties

Each layer takes its properties from its aquifer class (:AquiferClasses): horizontal conductivity K_h (K_HORIZ), vertical conductivity K_v (K_VERT, default $K_h/10$), S_s (SPEC_STORAGE, default 10^{-5}), S_y (SPEC_YIELD, default 0.1) and porosity (POROSITY, default 0.3; stored for future transport calculations; MODFLOW's flow calculation does not use it).

Watch out

MODFLOW connects layer ℓ of one column sideways only to layer ℓ of its neighbours. Where two profiles meet, list their layers in the same geological order, otherwise, for example, a gravel layer may connect to a clay layer.

3.5.5 Initial heads

The starting head of every cell in a column is

$$h_i^0 = \max(z_{\text{land},i} - d_0, z_{\text{low},i} + 0.1), \quad (3.11)$$

where d_0 is the profile's INITIAL_HEAD_DEPTH (or the value given by :InitialGWHeads in the .rvc file), and $z_{\text{low},i}$ is the bottom of the lowest active cell. With :InitialGWHeads ELEVATION z , z replaces $z_{\text{land},i} - d_0$ (still at least 0.1 m above the lowest active cell bottom). A hotstart (:GWHeads in the .rvc) overrides all of this cell by cell. Optionally, a steady-state calculation replaces these first guesses (Section 6.4.2).

3.5.6 The generated MODFLOW files

Raven writes a standard MODFLOW 6 simulation to <output>/mf6/: mfsim.nam (simulation), sim.tdis (time steps), sim.ims (solver), gwf.nam (model), gwf.dis (grid), gwf.npf (conductivities and layer types), gwf.sto (storage), gwf.ic (initial heads), gwf.oc (output control) and one file per exchange package (gwf.rch, gwf.drn, gwf.riv, gwf.evt, gwf.wel, gwf.ghb, gwf.chd, gwf.swx, gwf.res). Grid arrays are written layer by layer, each grid row starting on a new line, which is how MODFLOW reads them. These files can be opened in FloPy or ModelMuse; they never need editing.

3.6 Parameters used to build the model

The grid and layers are built from the following parameters; the class column says where each is set, the reference column where its default and allowed range are listed.

Table 3.3: Parameters used to build the model.

Symbol	Parameter	Class (where it is set)	Units	Reference
Δ	:GridCellSize	.rvg option	m	Sec. A.6
$f_{\min}$	:MinCellCoverage	.rvg option	–	Sec. A.6
–	AQUIFER_PROFILE	HRU attribute (:HRUs, .rvh)	–	Sec. A.6
P_k	HRU polygon	:HRUGeometry (.rvg)	–	Sec. A.6
z_{land}	land surface	:LandSurface (.rvg); else HRU ELEVATION	m	Sec. A.6
z_{br}	bedrock	:BedrockSurface (.rvg); else DEFAULT_BEDROCK_DEPTH (aquifer profile)	m	Sec. A.4
d_{sz}	SOIL_ZONE_DEPTH	aquifer profile	m	Sec. A.4
b_ℓ	layer thickness, type	aquifer profile (:AquiferProfiles)	m	Sec. A.5
K_h, K_v	K_HORIZ, K_VERT	aquifer class of the layer	m/d	Sec. A.2
S_s, S_y	SPEC_STORAGE, SPEC_YIELD	aquifer class of the layer	1/m, –	Sec. A.2
d_0	INITIAL_HEAD_DEPTH; :InitialGWHeads	aquifer profile; .rvc	m	Sec. A.4

3.7 What the generated files look like

The model name file lists the packages Raven uses:

```
BEGIN OPTIONS
  NEWTON
END OPTIONS
BEGIN PACKAGES
  DIS6 gwf.dis DIS
  NPF6 gwf.npf NPF
  ST06 gwf.sto STO
  IC6 gwf.ic IC
  RCH6 gwf.rch RCH_RAVEN
  DRN6 gwf.drn DRN_RAVEN
  RIV6 gwf.riv RIV_RAVEN
  OC6 gwf.oc OC
END PACKAGES
```

The grid file records the projection in its first line, so the grid can be placed on a map:

```
# Lambert azimuthal equal-area projection, origin lat=58.81662353 lon=-125.4914706 (sphere R
    =6371007.181 m)
BEGIN DIMENSIONS
  NLAY 3
  NROW 85
  NCOL 54
END DIMENSIONS
```

Boundary packages contain one line per cell with placeholder values; Raven replaces the values in memory every step, so the files show the grid positions of recharge, drains, rivers, wells and boundaries but not their daily values.

3.8 Solver settings

Why such a small head tolerance? MODFLOW stops iterating when no head changes by more than the tolerance, but a flow is a conductance times a head difference. The seepage drains have very large conductances (one 1 km cell with a leakance of 1 d^{-1} has $10^6 \text{ m}^2/\text{d}$), so a head tolerance of 1 cm could still leave about $10^4 \text{ m}^3/\text{d}$ of imbalance in such a cell on a difficult day. With 0.01 mm the imbalance is a thousand times smaller than with 1 cm. The price is more Newton iterations — in the 20-year Liard run about 10 per day on average instead of 7–8 — and a longer run (about 15 instead of 8 minutes). The value was chosen from the randomized tests: at 0.1 mm a few extreme set-ups (drain conductance $2 \times 10^7 \text{ m}^2/\text{d}$ per cell) still showed a 0.2% residual on single steps; at 0.01 mm none did.

Two-tier convergence. A very tight tolerance can occasionally stall the Newton iterations on a hard step. The target is the tolerance above. If a step has not reached it after 100 iterations (or half the iteration limit, if that is smaller), the coupler raises MODFLOW's own tolerance to ten times the target (0.1 mm) for the rest of that step only, and restores it for the next step. MODFLOW's own test always decides whether a step has converged. A step accepted at the relaxed tolerance is therefore as reliable as a step of a model run with that tolerance. Only if even that fails within the iteration limit is the step reported as not converged. `GWBudget.csv` records which case applied (*converged* = 1 target reached, 2 accepted at the relaxed tolerance, 0 not converged). The water exchanged between Raven and MODFLOW is exact at any tolerance — both sides use the same computed flows — so the tolerance only controls MODFLOW's own small numerical residual.

Table 3.4: MODFLOW solver settings (sim.ims).

IMS setting	Value	Meaning
COMPLEXITY	COMPLEX	robust defaults for non-linear problems
OUTER_DVCLOSE	0.00001 m	Newton iterations stop when no head changes by more than this (:SolverHeadTolerance)
OUTER_MAXIMUM	500	iteration limit (:SolverMaxOuterIterations)
UNDER_RELAXATION	DBD	damps oscillating head changes (theta 0.7, kappa 0.07, gamma 0.1)
BACKTRACKING_NUMBER	20	retries with a smaller step when the imbalance grows
INNER_MAXIMUM	500	linear-solver iteration limit
INNER_DVCLOSE, INNER_RCLOSE	$0.1 \times \text{outer}, 1 \text{ m}^3/\text{d}$	linear-solver tolerances
LINEAR_ACCELERATION	BICGSTAB	linear solver
PRECONDITIONER_LEVELS	5	ILU fill level

3.9 How MODFLOW solves each time step

3.9.1 Time steps

MODFLOW’s time steps are generated from Raven’s :StartDate, :Duration and :TimeStep: one transient stress period of $N = \text{round}(T/\Delta t)$ steps of length Δt , preceded by a one-day steady-state period when :GWInitialization STEADY_STATE is used. MODFLOW therefore always takes exactly one step for each Raven step. If Raven would run more steps than MODFLOW holds, Raven stops with a clear message.

3.9.2 Cells that dry and rewet: the Newton method

An unconfined cell whose water table falls below its bottom is *dry*. Older MODFLOW versions switched such cells off, which often made the solution fail when they needed to switch back on. MODFLOW 6’s Newton–Raphson method, which the coupling always uses, keeps dry cells in the calculation. The conductance between two cells is scaled smoothly by how full the upstream cell is, and falls to zero as that cell empties. The cell fills again naturally when water arrives. The solver settings written by Raven are:

- outer (Newton) iterations up to 500 (:SolverMaxOuterIterations), head-change tolerance 0.00001 m (:SolverHeadTolerance);
- delta-bar-delta under-relaxation (damps oscillating head changes) and backtracking (retries a smaller change when the residual grows);
- inner linear solver BiCGSTAB with an ILU preconditioner of level 5, residual tolerance $1.0 \text{ m}^3/\text{d}$.

The Newton option is written without its own UNDER_RELAXATION keyword, which resets a head that falls below the cell bottom. With MODFLOW 6.6 that reset made the Newton change of a drying cell repeat unchanged at every iteration, so the step never converged (on the Liard model this happened on most days once the thin aquitard under the gravel began to dry). Without it, MODFLOW 6.6.3 and 6.8.1 converge in the same few iterations. Raven runs the Newton iterations itself through MODFLOW’s interface (prepare, repeated solve, finalize), so it knows whether each step converged. If a step does not converge, MODFLOW continues (option CONTINUE), the step is flagged in GWBudget.csv, and a safeguard limits what that step can return to Raven (Section 6.4.5).

3.9.3 Connectivity between aquifer areas

Because dry cells carry no flow, parts of an aquifer can become disconnected when the water table falls and join again when it rises, without any action by the user. Streams disconnect from a falling water table in the same way (Section 5.3). To show this, `GWConnectivity.csv` reports each step:

- the fraction of top-layer cells of each profile that are *wet*, meaning $h_i > z_{\text{bot},i}^{(1)} + 10^{-3} \text{ m}$;
- the number of separate wet areas, where two wet cells belong to the same area if they share an edge (4-neighbour connectivity; counted by a flood-fill of the wet cells).

This is a diagnostic only; it does not affect the calculation.

3.9.4 Heads of dry cells

Under the Newton method a dry cell carries no flow, and MODFLOW leaves its head at whatever value the solver reaches; it can be far below the cell bottom (values of -10^8 m have been seen in a pumped, isolated cell). MODFLOW's own flows are unaffected, but such a head is not a water level. Wherever the coupling needs a real water level it therefore uses the *water table* of the column: the head of the highest cell whose head is above its bottom, or the bottom of the lowest active cell if all are dry. This is used for the reservoir's local groundwater level, in `GWHRUState.csv`, and for monitoring wells (a well screened in a dry cell reports a missing value, `-1.2345`). The MODFLOW head file and the hotstart heads keep MODFLOW's raw values, so a restart continues exactly.

CHAPTER 4

Coupling an existing MODFLOW 6 model

In plain words

You give Raven your MODFLOW 6 model and tell it three things: where the model lies on the map, on what calendar date the model's time starts, and which of the model's packages Raven should replace — usually recharge and evaporation, which Raven now supplies. Raven copies the model, makes the few changes shown in Figure 4.1, and runs the copy together with the Raven model, one day at a time. Your own files are never changed.

Many users already have a calibrated MODFLOW 6 model and want to connect it to a Raven model. In this mode Raven does not build a groundwater model. It takes the user's model as it is: the grid, the layers, the conductivities, storage, wells, boundaries, stress periods and solver settings. Raven links its HRUs to the model's top active layer — the highest layer that holds water at each location — and exchanges water there.

Only three things change, and only in a copy of the model (Figure 4.1):

1. the stress periods are split into Raven time steps;
2. the packages that Raven takes over (usually recharge and evapotranspiration) are switched off;
3. two packages are added: RCH_RAVEN (recharge from Raven) and DRN_RAVEN (seepage back to Raven).

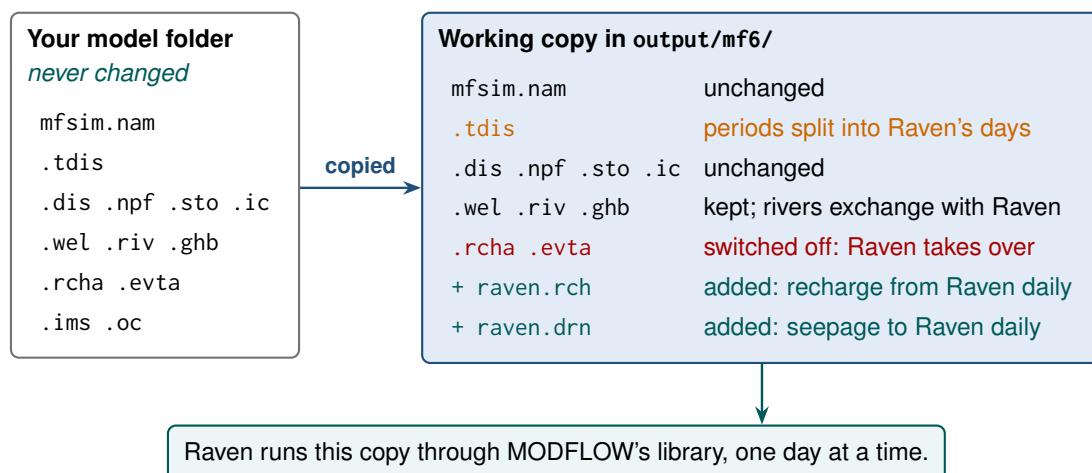


Figure 4.1: Raven runs a working copy of the model. The copy differs from the original in three ways only; the original folder is never changed.

The approach follows L. Scantlebury's MODFLOW-USG coupling, which linked the top layer of an existing model to HRUs through a table of overlap weights. It works for any grid, because only the top layer is linked and the model's cells are found through MODFLOW's own memory. Regular (DIS) and polygon (DISV) grids are supported.

4.1 A complete example

```
# Liard.rvg - Raven coupled to an existing MODFLOW 6 model
:HRUGeometry Liard_HRUs.geojson HRU_ID # HRU polygons (lon/lat)
:MF6Library /opt/mf6/libmf6.so
:MF6Simulation model/mfsim.nam # the existing simulation
:MF6CRS EPSG:32610 # projection of the model's coordinates
:MF6StartDate 1985-10-01 # date of the model's time zero
:RavenTakesOver RCH_MODEL EVT_MODEL # packages Raven replaces
:StreamPackage RIV_STREAMS AUX SUBBASIN # river cells linked to Raven's reaches
:SeepageToRaven ON # groundwater seeping out returns to Raven
```

HRUs with an `AQUIFER_PROFILE` in the `.rvh` file are coupled, as in the generated mode; with an existing model the profile's layers are not used, only its seepage parameters (`SEEPAGE_LEAKANCE`, `SEEPAGE_SMOOTHING_DEPTH`). Raven's recharge must reach the `GROUNDWATER` store, exactly as in Section 3.6.

4.2 Linking HRUs to the model's cells

Raven needs to know which grid cells lie under each HRU. MODFLOW model files do not say where on Earth the model is, so the user supplies that information in one of three ways (give exactly one). Each coupled HRU is then linked to the top active cell of every column it overlaps.

:MF6CRS (recommended). The user names the model's map projection, usually as a standard EPSG code. Raven reads the cell outlines from the model itself (column and row widths, origin and rotation for DISV grids; the cell corners for DISV grids) and places them on the map with that projection (Figure 4.2). Built-in projections need no external software: Transverse Mercator

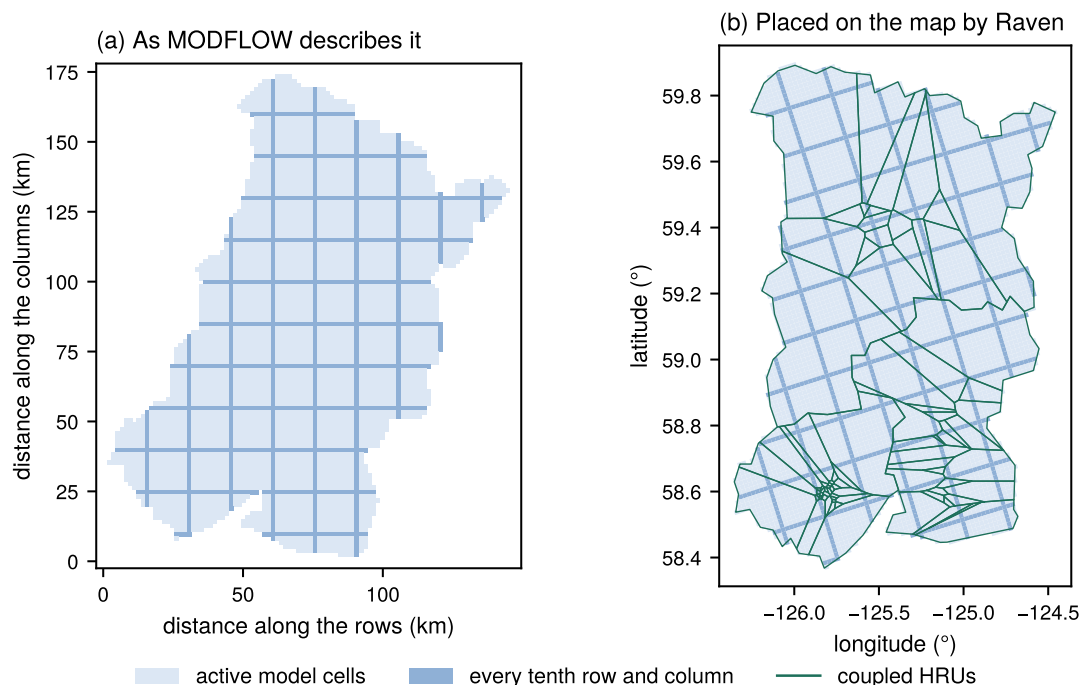


Figure 4.2: The synthetic Liard model's cells: (a) as MODFLOW describes them, in the model's own rotated coordinates; (b) placed on the map by Raven from the projection EPSG: 32610, under the Raven HRUs.

(every UTM zone, north and south), Albers equal-area and Lambert conformal conic, on the WGS84 or GRS80 (NAD83) ellipsoid. Common codes are recognised directly:

Code	Projection
EPSG:32601–32660, EPSG:32701–32760	WGS84 / UTM zones, north and south
EPSG:26901–26923	NAD83 / UTM zones 1–23 north
EPSG:3005	NAD83 / BC Albers
ESRI:102001 (or EPSG:102001)	Canada Albers equal-area conic (an ESRI code)
EPSG:5070	NAD83 / CONUS Albers
EPSG:3978, EPSG:3347	NAD83 / Canada Atlas Lambert; Statistics Canada Lambert

Any other projection of these three kinds is given by its parameters:

```
:MF6CRS TMERC lat0 lon0 k0 FE FN {WGS84|GRS80}
:MF6CRS ALBERS lat1 lat2 lat0 lon0 FE FN {WGS84|GRS80}
:MF6CRS LCC lat1 lat2 lat0 lon0 FE FN {WGS84|GRS80}
```

The built-in formulas (Karney’s series for Transverse Mercator, Snyder’s for the conics) agree with the PROJ library to a few micrometres (Section 9.7). The small datum difference between NAD83 and WGS84 (about a metre) is ignored.

:MF6GridFile. A GeoJSON file of the top-layer cells in longitude/latitude, numbered by an ID field (default CELL_ID) from 1 to the number of cells in a layer, as exported by GIS software. Raven checks that there is one polygon per cell and that the polygons are the model’s cells in the model’s order. A map projection scales areas and turns directions slightly, so Raven allows one common scale and one common rotation, and nothing else:

- every polygon’s area, divided by the area MODFLOW gives the cell, must be the same for all cells within 5%, and this common ratio must lie between 0.8 and 1.25 (a wrong length unit gives about 0.09 or 10.8);
- for every pair of neighbouring cells MODFLOW connects, the two polygons must lie as far apart and in the same direction as MODFLOW’s cells, within 20% and 15° after the common scale and rotation. A shuffled numbering changes the distances; a mirrored one (for example rows numbered from the south: MODFLOW’s row 1 is the northern row) turns directions around.

A file that fails either check stops the run with a message naming the first cell that does not fit. GWModelSummary.txt reports the common area ratio.

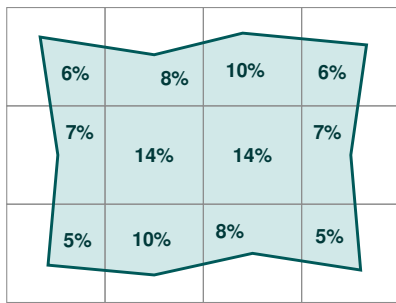
:OverlapWeights. The table of the MODFLOW-USG coupling, prepared outside Raven:

```
:OverlapWeights
# [HRU ID] [cell] [w = overlap area / cell area]
2736 1204 0.35
2736 1205 1.0
:EndOverlapWeights
```

No HRU polygons are needed. Monitoring wells cannot be located with this option.

No water is lost on the way. Each HRU’s recharge is shared among the cells it covers, in proportion to how much of the HRU lies in each cell (Figure 4.3a). The shares always add up to 100%, so all the water Raven sends arrives in MODFLOW — even when the HRU polygons or the weights do not quite match the HRU areas in the .rvh file. A mismatch changes only *where* inside the HRU the water lands, not *how much* (Figure 4.3b). In formula form, cell i receives $V_{k \rightarrow i} = R_k A_k A_{ki} / \sum_j A_{kj}$ from HRU k (Section 3.1.5).

(a) The HRU's share in each cell



One HRU (green) over 12 grid cells.
The shares add up to 100%.

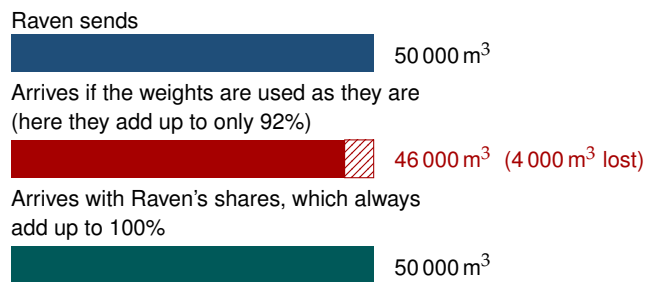
(b) 10 mm of recharge on a 5 km² HRU

Figure 4.3: Sharing an HRU's water among cells. (a) The HRU's share in each cell, computed from the shapes; the shares add up to 100%. (b) Weights that add up to only 92% would lose 8% of the water; Raven's shares always add up to 100%, so every cubic metre arrives.

In the older MODFLOW-USG coupling the weights were used as given, so the recharge volume depended on how carefully the HRUs were drawn. Rates passed to MODFLOW use MODFLOW's own cell areas. `GWModelSummary.txt` reports how much of the coupled HRUs' area lies on active cells; a low value usually means a wrong projection or HRUs outside the model.

4.3 Time

MODFLOW models divide time into *stress periods*: stretches of time, often a month, during which inputs such as pumping rates stay the same. Raven works in days. Raven keeps the model's stress periods and their dates, and splits each period into Raven days (Figure 4.4), so the model's own changes — a well pumping harder in summer, a boundary level changing — still happen on their dates.

`:MF6StartDate` is the calendar date on which the model's time starts. Each stress period must contain a whole number of Raven time steps, and Raven's run may not go past the end of the model; otherwise the run stops with a message. Steady-state periods (solved as the long-term balance) stay steady-state.

Steady-state models. Many calibrated models are steady state: no storage package (or one that marks every period steady) and a single stress period whose length has no meaning (often one second or one day). Raven stretches that period over its run, so every Raven step is a steady-state solution with that step's recharge: the aquifer responds at once, without storage. `GWModelSummary.txt` says so, and the storage column of `GWBudget.csv` stays zero (the balance residual is reported as MODFLOW's error). Because such a model has no time of its own, Raven may start on any date from `:MF6StartDate` on.

Starting later: restarts. Raven may start later than the model's time zero — when a run is continued from `solution.rvc`, or begins part-way through the model. Raven then removes the stress periods already simulated from the working copy, shortens the period that holds Raven's start, and renumbers the periods of every package; the new first period takes the inputs that were in force on Raven's start date (Figure 4.4, bottom). This is safe for packages whose period entries replace one another (WEL, RIV, GHB, CHD, DRN, list-based RCH and EVT, STO, OC, the water mover MVR), and for array-based recharge and ET and the advanced packages (SFR, LAK, MAW, UZF) when at most one period entry comes before Raven's start. Such a package with several earlier entries, whose settings build on one another, stops such a run, as do time-varying

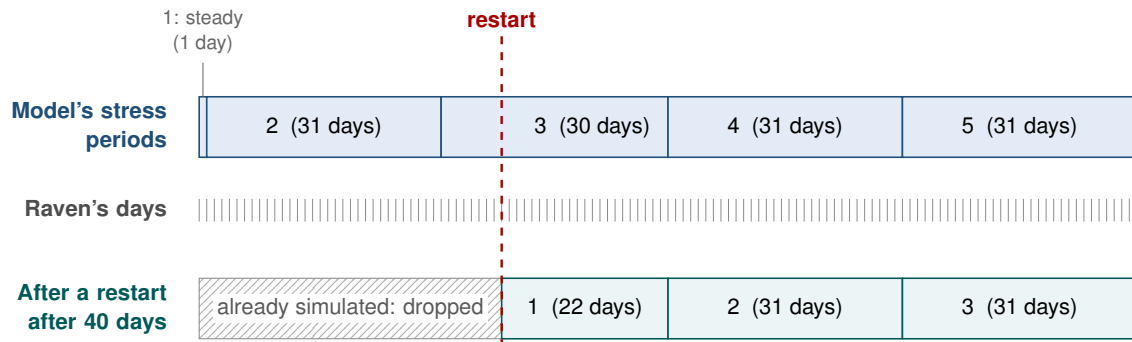


Figure 4.4: Top: a model with a one-day steady period followed by periods of 31, 30, 31 and 31 days, and Raven's days. Bottom: after a restart 40 days into the model, the time already simulated is dropped, period 3 keeps its last 22 days, and the periods are renumbered; each keeps its own well rates and boundary levels.

conductivity or storage (TVK6, TVS6) and a simulation holding more than one model (only the coupled model's periods would be renumbered); Raven must then start at :MF6StartDate. A package that uses time series stops any later start, even one inside the first period, because the times of a series count from the model's own start. A time of day in Raven's :StartDate counts in the offset, and the working copy's START_DATE_TIME is set to Raven's start.

4.4 Units

Length units (metres, feet, centimetres) are read from the discretization package and time units (seconds to years) from the time discretization. Heads, flows, rates, conductances and coordinates are converted at the interface; Raven's outputs are always in metres and days.

4.5 Packages

4.6 Safeguards

Every one of these stops the run with a message that names the problem and the fix:

- a recharge, ET or UZF package not declared taken over or kept; :RavenTakesOver naming another kind of package; a stream package that is not RIV, DRN or GHB, or one that sends water through the mover; a package name not in the model name file; a mover whose period data sit in another file, when it names a package Raven takes over;
- several groundwater models in the simulation and no :MF6Model;
- a Raven start before the model's time zero (:MF6StartDate); a later start that the model's packages cannot follow (Section 4.3); a stress period that is not a whole number of Raven steps; a Raven run longer than the model;
- a cell file whose polygons are not the model's cells (count, numbering or area);
- a stream cell whose subbasin does not exist or is disabled;
- options of the generated mode left in the .rvg file (grid options, :GridCellSize, :LandSurface, :BedrockSurface, the solver settings, :Wells, head boundaries, :RiverGeometry, :GWInitialization), or none or several ways of linking HRUs;
- a simulation with more than one solution (for example a transport model with its own IMS); Raven drives one groundwater-flow solution;

Table 4.1: How the model’s packages are treated.

Package	Treatment
Recharge, evapotranspiration, UZF	must be named in :RavenTakesOver (removed; Raven supplies recharge) or in :KeepPackages (they stay). An undeclared one stops the run, so recharge is never counted twice
Stream package (:StreamPackage)	a RIV, DRN or GHB package whose flows go to Raven’s reaches: to the subbasin in an auxiliary variable (AUX name, default SUBBASIN), or to the subbasin of the dominant HRU of the cell (DOMINANT_HRU). Losing reaches take the water from the channel; a loss that cannot be supplied is carried to the next step
DRN_RAVEN (:SeepageToRavenON)	added at the top of the top active cell of coupled columns; the discharge returns to Raven (Section 5.2)
RCH_RAVEN	added at the top active cell of every coupled column. MODFLOW ignores recharge on specified-head cells, so columns whose top cell is specified-head (read from the model each step, as the stress periods may change them) take none: each HRU’s recharge is spread over its other cells, and recharge of an HRU lying only on such cells is reported in GWBudget.csv (<i>recharge onto CHD cells</i>)
Water mover (MVR)	kept; movers from or to a package Raven takes over are removed from the working copy (MODFLOW would stop on a package that is no longer there), and GWModelSummary.txt reports how many. A stream package that sends water through the mover is refused, because Raven would take the same water from the aquifer a second time
All other packages	kept unchanged: wells, head boundaries, drains, lakes, streamflow routing and so on. Their simulated flows, including the water they pass to the mover, enter the balance check (GWBudget.csv, column <i>other model packages</i>)

- adaptive time stepping (ATS6 in the TDIS options): Raven needs fixed time steps;
- a Raven calendar other than Gregorian (the dates of the two models are compared day by day);
- :WriteNetCDFHeads with :OverlapWeights (a weight table has no coordinates);
- a simulation that lies inside Raven’s own working folder (output/mf6/), which Raven overwrites.

A solver tolerance looser than 0.01 m is reported in GWModelSummary.txt.

4.7 The model’s solver

The model’s own solver settings (.ims) are used. On a step that needs more than 100 outer iterations (or half the model’s iteration limit, if that is smaller) the coupler relaxes MODFLOW’s head tolerance tenfold for the rest of that step (as for generated models, Section 3.9), in the model’s units. The first coupled step is often the hardest — a steady-state period solved from the model’s starting heads with Raven’s recharge — so allow at least 500 outer iterations (OUTER_MAXIMUM). A step that does not converge is flagged in GWBudget.csv and the exchange on it is limited, as described in Section 3.9. For this, the working copy’s mfsim.nam always has the CONTINUE option: without it, MODFLOW would stop the whole simulation at the first step that does not converge.

With MODFLOW versions before 6.8, the UNDER_RELAXATION keyword of the model’s NEWTON option can make a step in which a cell dries repeat the same Newton change at every iteration and never converge (the synthetic Liard model failed on every day with MODFLOW 6.6.3 and converged on every day with 6.8.1). Raven reads the library’s version, writes it to GWModelSummary.txt, and warns when an older version meets that keyword: use MODFLOW 6.8 or newer, or remove the keyword from the model’s name file (on the synthetic model the results with 6.8.1 are identical either way).

4.8 Outputs and restarts

All outputs of Chapter 7 are written. Raven copies the whole folder that holds the simulation file (the simulation may read files from anywhere in it), so keep the model in a folder of its own; old results (.hds, .cbc, .lst, .grb, .bud, .cbb) are not copied. The simulation file may have any name: the working copy always holds it as mfsim.nam, the name MODFLOW’s library opens. In GWHRState.csv the water-table depth is measured from the top of the model’s top active cell, which with an existing model stands for the land surface. mf6/ holds the working copy of the simulation, with Raven’s packages (raven.rch, raven.drn); mf6/grid_cells.geojson shows the linked cells (not with :OverlapWeights). Restarts store the heads in the model’s layout (:GWHeads nlay nrow ncol for DIS, nlay 1 ncells for DISV); on a restart Raven writes them as the initial heads of the working copy (raven.ic) and renumbers the stress periods as described above. A restarted run reproduces a continuous one (Section 9.7).

4.9 Worked example

examples/Liard_existing_model/ holds a MODFLOW 6 model of the Liard coupled area built independently of Raven: a regular grid of 1.5 km cells rotated 15° in UTM zone 10N, four layers with spatially varying conductivity, a one-day steady-state period followed by 24 monthly periods, two wells with seasonal pumping, 239 river cells carrying their subbasin ID in an auxiliary variable, a head boundary at the outlet, and its own recharge and ET packages. The .rvg file of Section 4 couples it to the Liard Raven model; the two-year run takes about four minutes.

4.10 Checking the calibration

Replacing a calibrated recharge package with Raven’s recharge changes the model’s water budget. Compare the coupled heads and budget with a run of the original model before using the results: large differences mean Raven’s recharge, or the recharge the model was calibrated with, needs attention.

4.11 Current limits

DISU models, water-table ET from Raven (the model’s own ET package can be kept), surface-store and reservoir exchanges, and the conductance limit of generated river cells are not available in this mode; the stream package’s stage comes from the model. Restarts after the model’s time zero need daily or longer Raven time steps.

CHAPTER 5

Water exchanges between Raven and MODFLOW

Raven and MODFLOW exchange water in eight ways (Table 5.1). Each section below explains what the exchange represents and gives its equations. It then says what Raven does with the result, which safeguards keep the water accounts exact, which inputs control it, where to see it in the outputs, and gives a worked example.

In plain words

Water crosses between the two models in eight places. The two main ones are *recharge* — water soaking down from Raven’s soil to the water table — and *seepage* — groundwater coming back up where the water table reaches the ground. Rivers, lakes and reservoirs, wetlands, wells and evaporation from a shallow water table make up the rest. For each one this chapter gives the formula, a worked example with numbers, and the checks that make sure no water is created or lost on the way.

Table 5.1: The eight exchanges between Raven and MODFLOW.

Exchange	Direction	MODFLOW package	Raven side	Section
Recharge	Raven → aquifer	RCH	GROUNDWATER	5.1
Seepage	aquifer → Raven	DRN	lowest soil layer	5.2
Rivers	both	RIV	reach inflow	5.3
Water-table ET	aquifer → air	EVT	unmet PET	5.4
Wetlands, lakes	both	RIV	HRU store	5.5
Reservoirs	both	WEL	reservoir seepage	5.6
Wells	out (or in)	WEL	time series	5.7
Boundaries	both	GHB, CHD	time series	5.7

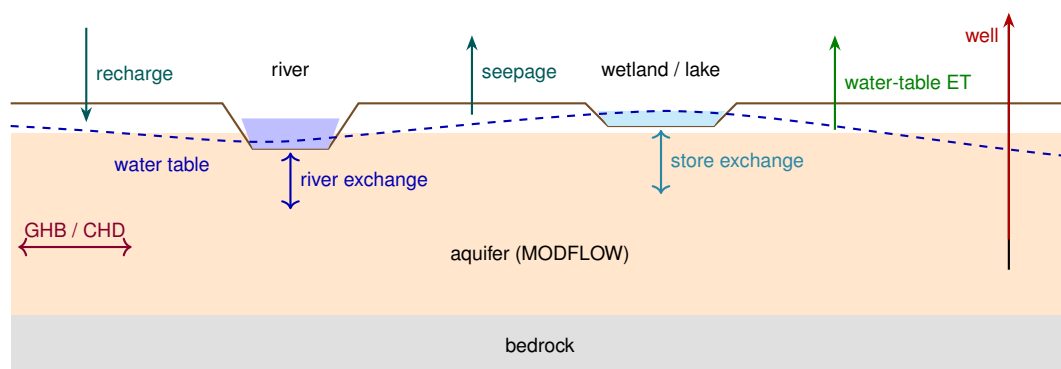


Figure 5.1: The exchanges between Raven (above the water table) and MODFLOW (below). Reservoir seepage works like the store exchange, through Raven’s reservoir seepage law.

5.1 Recharge: from Raven to the water table

5.1.1 Where recharge comes from

Any Raven process that moves water into GROUNDWATER in a coupled HRU produces recharge — no special process is needed. For example, in the Liard model the percolation from the fast reservoir is sent to GROUNDWATER for coupled HRUs:

```
:Percolation PERC_CONSTANT FAST_RESERVOIR GROUNDWATER
:-->Conditional HRU_GROUP IS AquiferHRUs
```

At the end of a day, the GROUNDWATER store of HRU k holds r_k mm; this is the recharge of that day. Raven counts it as water leaving Raven (its balance, Section 6.1), and it is empty again at the start of the next day.

5.1.2 Volume of recharge

$$V_k = \frac{r_k}{1000} A_k \quad [\text{m}^3], \quad (5.1)$$

with A_k the HRU area from the .rvh file in m^2 .

5.1.3 Optional recharge delay

Where the water table is deep, water takes time to cross the unsaturated zone. With the aquifer profile parameter RECHARGE_DELAY $\tau > 0$ (Section A.4) (days) the recharge passes through a linear reservoir S_k before reaching the water table:

$$S_k^* = S_k^n + V_k, \quad V_k^{\text{out}} = S_k^* \left(1 - e^{-\Delta t/\tau}\right), \quad S_k^{n+1} = S_k^* - V_k^{\text{out}}. \quad (5.2)$$

With $\tau = 0$ the delay is off and $V_k^{\text{out}} = V_k$. The delay store is reported in GWBudget.csv, saved in solution.rvc (:GWRechargeStore) and read back on hotstart.

Worked example: recharge delay

$\tau = 60$ d, $\Delta t = 1$ d: each day $1 - e^{-1/60} = 1.65\%$ of the store is released. A single $1\,000\text{ m}^3$ pulse reaches the water table as 16.5 m^3 on the first day, then less each day; after 60 days 63% of it has arrived (Figure 5.2a).

5.1.4 Spreading recharge over the cells

The released volume is shared among the HRU's cells in proportion to overlap. Cells whose top cell holds a fixed head (Section 5.7) are skipped, because MODFLOW ignores recharge there:

$$Q_i^{\text{rch}} = \sum_{k \in \mathcal{K}} \frac{V_k^{\text{out}}}{\Delta t} \frac{A_{k,i}}{\sum_{j \notin \mathcal{C}} A_{k,j}} \quad [\text{m}^3/\text{d}], \quad R_i = \frac{Q_i^{\text{rch}}}{\Delta^2} \quad [\text{m}/\text{d}], \quad (5.3)$$

where $\mathcal{C}$ are the fixed-head columns. R_i is the recharge rate given to MODFLOW's RCH package, applied to the top cell of the column (the top layer of an active column is always active). The fractions for one HRU always add up to one, so exactly V_k^{out} enters MODFLOW. If an HRU lies entirely on fixed-head cells, its recharge passes to that boundary and is reported in the column *recharge onto CHD cells*.

Worked example: spreading recharge

The 10 km^2 HRU of the Section 3.1 example receives 2 mm in a day: $V = 0.002 \times 10^7 = 20\,000\text{ m}^3$. The three cells get 8 000, 7 000 and 5 000 m^3/d ; for 4 km^2 cells the recharge rates are 2.0, 1.75 and 1.25 mm/d (plus any recharge from other HRUs in the same cells).

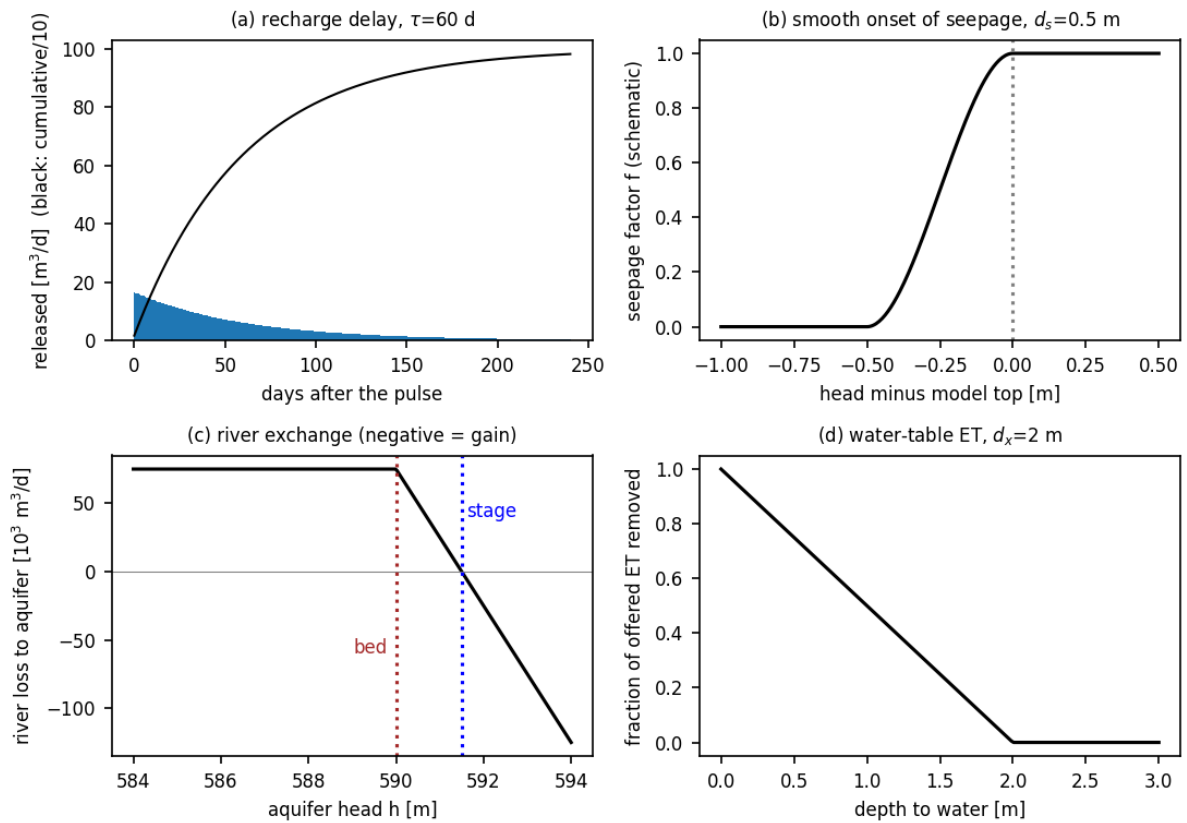


Figure 5.2: Response curves of four exchanges: (a) recharge delay reservoir after a $1\,000\text{ m}^3$ pulse ($\tau = 60$ d); (b) seepage factor near the model top ($d_s = 0.5$ m); (c) river flow against aquifer head; (d) water-table ET against depth ($d_x = 2$ m).

Reference card: recharge

Sources/sinks. Moves water from GROUNDWATER of each coupled HRU to the aquifer (MODFLOW RCH package RCH_RAVEN, top cell of each active column), optionally through the recharge-delay store (internal to the coupling, saved in solution.rvc).

Constraints/notes. In a coupled HRU GROUNDWATER is a pass-through: no Raven process may draw water from it (start-up error). Recharge is never negative. The volume of each HRU is conserved exactly; recharge of HRUs lying wholly on fixed-head cells is reported separately.

Command syntax

```
# .rvi -- any process that moves water into GROUNDWATER in coupled HRUs, e.g.
:Percolation PERC_CONSTANT FAST_RESERVOIR GROUNDWATER
  :-->Conditional HRU_GROUP IS AquiferHRUs
# .rvp -- optional recharge delay, per aquifer profile
:AquiferProfileParameters
  :Attributes, RECHARGE_DELAY
  :Units, d
  VALLEY_ALLUVIUM, 30.0
:EndAquiferProfileParameters
```

Parameters

Symbol	Parameter	Class (where it is set)	Units	Reference
r_k	GROUNDWATER	Raven state variable of HRU k (filled by the chosen process)	mm	Raven state var.
A_k	AREA	HRU (:HRUs, .rvh)	km ²	Raven manual
τ	RECHARGE_DELAY	aquifer profile (:AquiferProfileParameters)	d	Sec. A.4
$A_{k,i}$	overlap area	computed from :HRUGeometry (.rvg)	m ²	Sec. 3.1
Δ	:GridCellSize	.rvg option	m	Sec. A.6

Outputs. GWBudget.csv: *Raven recharge, MF6 recharge, recharge delay storage, recharge onto CHD cells*; Raven’s GROUNDWATER storage shows each day’s recharge (mm) of every HRU.

5.1.5 Common questions

Can recharge be negative? Only if a Raven process removes water from GROUNDWATER, which is forbidden in coupled HRUs; recharge is therefore never negative. *Does recharge depend on the water table?* Not directly — Raven decides it. If the water table reaches the surface, the extra water returns immediately as seepage (Section 5.2), which acts as “rejected recharge”.

5.2 Seepage: groundwater returning to the surface

Where the water table reaches the land surface, groundwater flows out: springs, wet meadows, seepage faces. Every active column has a drain (MODFLOW’s DRN package) at the model top that removes water whenever the water table is close to the top, and hands it back to Raven.

5.2.1 Equation

$$Q_i^{\text{seep}} = C_i^{\text{dm}} f(h_i) (h_i - z_{\text{top},i} + d_s) \quad \text{for } h_i > z_{\text{top},i} - d_s, \quad C_i^{\text{dm}} = L_s \Delta^2, \quad (5.4)$$

where L_s [1/d] is the aquifer profile parameter SEEPAGE_LEAKANCE and d_s [m] the aquifer profile parameter SEEPAGE_SMOOTHING_DEPTH (Section A.4). The factor f rises smoothly from 0 at d_s below the top to 1 at the top (Figure 5.2b). Where the top cell is thinner than d_s , d_s is limited to its thickness (MODFLOW does not accept a smoothing interval below the cell). Without this smoothing the drain would switch on abruptly, which makes the Newton iterations harder; with it, seepage starts gently as the water table approaches the surface.

5.2.2 Back to Raven

Each column's seepage is shared among the HRUs above it by overlap:

$$V_k^{\text{ret}} = \sum_i Q_i^{\text{seep}} \Delta t \frac{A_{k,i}}{\sum_m A_{m,i}}, \quad (5.5)$$

and added to the Raven store :SeepageReturnTo of HRU k (default: the lowest soil layer) as $1000 V_k^{\text{ret}} / A_k$ mm. HRUs without soil layers (lake, glacier or rock profiles) receive it in SURFACE_WATER instead. From there Raven's own processes take it on — for example, the base-flow process of the lowest soil layer carries it to the river.

Worked example: seepage

A 4 km² cell with $L_s = 1 \text{ d}^{-1}$ has $C = 4 \times 10^6 \text{ m}^2/\text{d}$. With the water table 0.1 m above the top and the factor $f = 1$, seepage is $4 \times 10^6 \times (0.1 + 0.5) \approx 2.4 \times 10^6 \text{ m}^3/\text{d}$ — a large number, which is why in practice the water table settles very close to the surface wherever the aquifer is full: the drain is a strong outlet.

Reference card: seepage

Sources/sinks. Moves water from the aquifer (MODFLOW DRN package DRN_RAVEN at the model top of every active column) to the Raven store named by :SeepageReturnTo in each overlapping HRU — by default the lowest soil layer SOIL[m], and SURFACE_WATER for HRUs without soil layers.

Constraints/notes. Seepage occurs only when the head is within d_s of the model top; it is returned to Raven by overlap, exactly. In a step that does not converge, seepage (with river gains and store discharge) is limited to the step's inflow.

Command syntax

```
# .rvp
:AquiferProfileParameters
:Attributes, SEEPAGE_LEAKANCE, SEEPAGE_SMOOTHING_DEPTH, SOIL_ZONE_DEPTH
:Units, 1/d, m, m
  VALLEY_ALLUVIUM, 1.0, 0.5, 0.0
:EndAquiferProfileParameters
# .rvg -- optional
:SeepageReturnTo SOIL[2]
```

Parameters				
Symbol	Parameter	Class (where it is set)	Units	Reference
L_s	SEEPAGE_LEAKANCE	aquifer profile	1/d	Sec. A.4
d_s	SEEPAGE_SMOOTHING_DEPTH	aquifer profile	m	Sec. A.4
z_{top}	land surface — SOIL_ZONE_DEPTH	:LandSurface (.rvg); aquifer profile	m	Sec. A.4
—	:SeepageReturnTo	.rvg option (a Raven storage state variable)	—	Sec. A.6

Outputs. GWBudget.csv: *seepage to Raven* and *returned to Raven* (equal); the receiving store in Raven’s outputs.

5.2.3 Choosing the leakance

A large leakance makes the drain a strong outlet: the water table then stays within a few centimetres of the surface wherever the aquifer is full, and seepage follows recharge closely. A small leakance lets the water table rise above the surface (ponding in the model’s sense) and delays seepage. Values of 0.1–10 d^{−1} cover most cases; 1 d^{−1} is the default.

5.3 Rivers

Rivers exchange water with the aquifer in both directions: a *gaining* river receives groundwater (baseflow); a *losing* river leaks into the aquifer. The coupling uses MODFLOW’s RIV package in every cell crossed by a river line (:RiverGeometry), with river levels taken from Raven’s own river routing.

5.3.1 River cells

River lines (GeoJSON, lon/lat) are cut into short pieces (1/20 of a cell) and each piece is assigned to the cell under its midpoint. For each subbasin p and cell i Raven records the river length $L_{p,i}$ and the lowest DEM value along the line in that cell. The subbasin of a piece comes from an ID field in the GeoJSON, or, if none is given, from the subbasin of the dominant HRU of the cell.

5.3.2 Riverbed elevation and river level

The DEM usually records the water surface or floodplain along a river, so the bed lies one bankfull depth below it:

$$z_{\text{bed}} = \min_{\text{line in cell}} z_{\text{DEM}} - d_{\text{bf},p}, \quad (5.6)$$

where $d_{\text{bf},p}$ is the depth of Raven’s channel at its reference flow (or the value of :RiverBedDepth); z_{bed} is never placed above the model top. The bed is never placed above the model top nor below the column: if it would lie below every active cell, it is set 0.1 m above the lowest active cell bottom. The river cell is placed in the highest active layer whose bottom is below the bed. Each day the river level (*stage*) is

$$s = \min(z_{\text{bed}} + d_p(t), z_{\text{land},i}), \quad (5.7)$$

with $d_p(t)$ Raven’s channel depth at the start of the day. Above bankfull the extra water is on the floodplain, which is not part of this exchange, so the stage is capped at the land surface.

5.3.3 Exchange equation

$$q^{\text{riv}} = \begin{cases} C^{\text{riv}} (s - h) & h > z_{\text{bed}}, \\ C^{\text{riv}} (s - z_{\text{bed}}) & h \leq z_{\text{bed}}, \end{cases} \quad C^{\text{riv}} = \frac{K_b W_p(t) L_{p,i}}{M_b}, \quad (5.8)$$

with K_b [m/d] the riverbed conductivity (aquifer profile parameter RIVERBED_K), M_b [m] its thickness (aquifer profile parameter RIVERBED_THICKNESS; Section A.4) and $W_p(t)$ Raven's channel top width. Positive q is a losing river. When the water table drops below the bed, the river is *disconnected*: it leaks at a constant maximum rate, whatever the depth of the water table (Figure 5.2c).

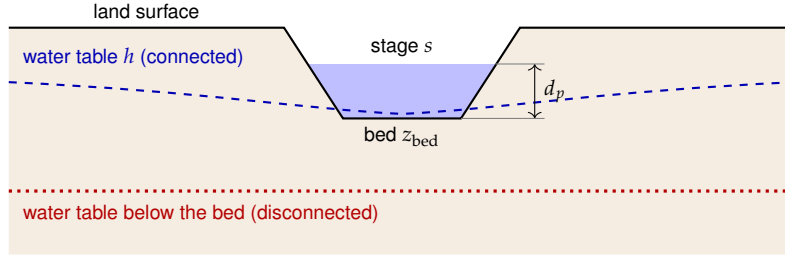


Figure 5.3: River cell: stage $s = \text{bed} + \text{Raven channel depth}$. With the water table above the bed, flow is $C(s - h)$ (either direction); below the bed the river is disconnected and leaks at $C(s - z_{\text{bed}})$.

5.3.4 Never taking water the river does not have

A losing river cannot give more than it carries. The conductance is therefore limited each day so that the largest possible loss — the disconnected rate $C(s - z_{\text{bed}})$ — is no more than the cell's share (by river length) of the flow in the channel reach at the start of the day (the channel flow, upstream of any reservoir at the subbasin outlet):

$$C^{\text{riv}} \leq \frac{Q_p^{\text{out}} (L_{p,i}/L_p)}{s - z_{\text{bed}}}, \quad (5.9)$$

with Q_p^{out} in m^3/d and L_p the subbasin's total river length on active cells. (Using the reservoir release instead would cut off every river cell above a lake whenever the lake is not spilling.) Rivers of disabled subbasins get no river cells.

5.3.5 Back to Raven's rivers

For each subbasin the net aquifer discharge is $G_p = -\sum_i q_{p,i}^{\text{riv}}$ [m^3/d]. A gain is added to the subbasin's in-catchment inflow for the day (so it passes Raven's catchment routing like other runoff). A loss is taken first from that in-catchment inflow and then, inside Raven's routing loop, from the flow entering the reach from upstream. A loss that still cannot be supplied in the day — rare, because of the limit above — is *carried* to the next day and taken from the river first thing then, so no water is created:

$$V_{p,n}^{\text{applied}} = G_{p,n} \Delta t - K_{p,n-1} + K_{p,n}, \quad (5.10)$$

where $K_{p,n}$ [m^3] is the loss carried out of day n . Summed over a run, the water applied to Raven equals MODFLOW's net river exchange minus whatever is still carried at the end (written to solution.rvc as :GWRiverLossCarry and taken on the first day of a restart).

Worked example: river exchange

$K_b = 0.5$ m/d, $M_b = 1$ m, width 40 m, 2 500 m of river in the cell: $C = 0.5 \times 40 \times 2\,500/1 = 50\,000$ m²/d. Bed 590 m, depth 1.5 m, stage 591.5 m. Water table at 590.8 m: $q = 50\,000 \times 0.7 = 35\,000$ m³/d lost to the aquifer. Water table at 592.0 m: $q = -25\,000$ m³/d, a gain for the river. Water table at 585 m (below the bed): the river leaks $50\,000 \times 1.5 = 75\,000$ m³/d.

Reference card: rivers

Sources/sinks. Exchanges water in both directions between the aquifer (MODFLOW RIV package RIV_RAVEN in every river cell) and Raven’s reaches: gains are added to the subbasin’s in-catchment inflow; losses are taken from it and then from the reach inflow.

Constraints/notes. The conductance is limited by the channel flow at the start of the step; a loss that cannot be supplied is carried to the next step. The stage never exceeds the land surface; the bed is never above the model top. Disabled subbasins get no river cells.

Command syntax

```
# .rvg
:RiverGeometry my_rivers.geojson SubId # river lines; subbasin ID field optional
:RiverBedDepth REFERENCE_FLOW # or a depth [m] below the DEM
# .rvp
:AquiferProfileParameters
:Attributes, RIVERBED_K, RIVERBED_THICKNESS
:Units, m/d, m
VALLEY_ALLUVIUM, 0.5, 1.0
:EndAquiferProfileParameters
```

Parameters

Symbol	Parameter	Class (where it is set)	Units	Reference
K_b	RIVERBED_K	aquifer profile (of the column)	m/d	Sec. A.4
M_b	RIVERBED_THICKNESS	aquifer profile	m	Sec. A.4
W_p, d_p	channel top width, depth	subbasin channel (:ChannelProfile, .rvp); Raven routing state	m	Raven manual
$d_{bf,p}$	:RiverBedDepth or depth at Q_REFERENCE	.rvg option; subbasin parameter (.rvh)	m	Sec. A.6
$L_{p,i}$	river length in a cell	computed from :RiverGeometry	m	Sec. 5.3
Q_p^{out}	channel flow	Raven routing state (subbasin)	m ³ /s	Raven state var.

Outputs. GWBudget.csv: *river leakage to aquifer, aquifer discharge to rivers, river exchange applied to Raven, river loss carried to next step*; streamflow in Hydrographs.csv includes the exchange.

5.3.6 Common questions

Why is the stage from the start of the step? Raven routes rivers after MODFLOW has solved; using the start-of-step depth avoids iterating between the two models, and at daily steps the difference is small. *What if a subbasin has no channel?* Its depth and width are zero, so its conductance and stage give no exchange, and any groundwater reaching the surface there leaves through the seepage drains. *Headwater reaches with little flow* cannot lose much water, because the supply limit scales the conductance with the reach outflow.

5.4 Evaporation from a shallow water table

Plants and bare soil can draw water directly from a water table close to the surface. With the aquifer profile parameter `EXTINCTION_DEPTH` (Section A.4) $d_x > 0$, each active column gets MODFLOW’s EVT package. Raven first satisfies evaporation from its own stores; only the potential evaporation it could *not* meet is offered to the water table:

$$E_i = \frac{1}{\Delta^2} \sum_k A_{k,i} \frac{\max(\text{PET}_k \Delta t - \text{AET}_k, 0)}{1000 \Delta t} \quad [\text{m/d}], \quad (5.11)$$

where PET_k [mm/d] is the HRU’s potential evaporation and AET_k [mm] what Raven already evaporated that day. Only HRUs whose own profile has an extinction depth contribute. The unmet demand of those HRUs in the cell (a volume) is spread over the whole cell area Δ^2 , because MODFLOW multiplies the rate by the full cell area. The most MODFLOW can remove is then exactly that unmet volume, even where coupled HRUs cover only part of the cell. MODFLOW removes

$$Q_i^{\text{evt}} = E_i \Delta^2 \min\left(1, \max\left(0, 1 - \frac{z_{\text{land},i} - h_i}{d_x}\right)\right), \quad (5.12)$$

the full rate when the water table is at the surface, falling linearly to zero at depth d_x (Figure 5.2d). This water leaves the system as evaporation.

Worked example: water-table ET

Unmet PET 2 mm/d over a 4 km² cell fully covered by coupled HRUs, $d_x = 2$ m, water table 0.5 m deep: $Q = 0.002 \times 4 \times 10^6 \times (1 - 0.5/2) = 6\,000 \text{ m}^3/\text{d}$. If coupled HRUs covered only 1 km² of the cell, the unmet volume would be 2 000 m³, $E = 2\,000/4 \times 10^6 = 0.5 \text{ mm/d}$ over the cell, and $Q = 1\,500 \text{ m}^3/\text{d}$.

Reference card: water-table evaporation

Sources/sinks. Moves water from the aquifer (MODFLOW EVT package `EVT_RAVEN`, top cell) to the atmosphere. It leaves the system and is not included in Raven’s AET.

Constraints/notes. Only HRUs whose aquifer profile has an extinction depth contribute; MODFLOW can never remove more than their unmet evaporative demand; the rate falls linearly to zero at the extinction depth.

Command syntax

```
# .rvp
:AquiferProfileParameters
:Attributes, EXTINCTION_DEPTH
:Units, m
  VALLEY_ALLUVIUM, 2.0
:EndAquiferProfileParameters
```

Parameters

Symbol	Parameter	Class (where it is set)	Units	Reference
d_x	<code>EXTINCTION_DEPTH</code>	aquifer profile (0 = off)	m	Sec. A.4
PET_k	potential evapotranspiration	Raven forcing function of HRU k (:Evaporation method, .rvi)	mm/d	Raven manual
AET_k	AET	Raven state variable of HRU k	mm	Raven state var.

Outputs. `GWBudget.csv`: *water-table ET*. Total evaporation is Raven’s AET plus this column.

5.5 Wetlands and lakes held in Raven stores

Wetlands and small lakes are often represented in Raven by a store in their HRUs: DEPRESSION for wetlands, or LAKE_STORAGE for lakes. :SurfaceWaterExchange SV group LEAKANCE L connects such a store, in the HRUs of an HRU group, to the aquifer below.

5.5.1 The boundary

In each column holding HRUs of the group, the exchange is a river-type boundary (MODFLOW RIV package) whose bed is the model top and whose water level is the top plus the average depth of water in the store:

$$\bar{s}_i = \frac{\sum_k A_{k,i} s_k}{\sum_k A_{k,i}}, \quad h_b = z_{\text{top},i} + \bar{s}_i, \quad q = \begin{cases} C (h_b - h) & h > z_{\text{top},i} \\ C \bar{s}_i & h \leq z_{\text{top},i} \end{cases} \quad (5.13)$$

with s_k the store depth of HRU k in metres (1/1000 of the store in mm). Positive q is leakage from the store to the aquifer; negative q is groundwater feeding the store. Because leakage stops growing once the water table is below the store's bed, it can never exceed $C \bar{s}_i$.

5.5.2 Never draining more than the store holds

The water available in a cell is the store's real volume, split among the HRU's cells by overlap and fixed at the start of the day:

$$V_{k,i} = s_k A_k \frac{A_{k,i}}{\sum_j A_{k,j}}, \quad V_i = \sum_k V_{k,i}. \quad (5.14)$$

The conductance is limited so that the largest possible leakage in the day is no more than this water:

$$C = \min \left(L \sum_k A_{k,i}, \frac{V_i}{\Delta t \bar{s}_i} \right), \quad C = 0 \text{ when the stores are empty.} \quad (5.15)$$

After MODFLOW solves, leakage is removed from each HRU's store in proportion to $V_{k,i}$, and groundwater discharge is added in proportion to area. Because the shares come from the same fixed volumes used for the limit, no store can be over-drawn and the volumes on both sides match exactly. A store of an HRU may be connected by only one :SurfaceWaterExchange; Raven stops if two exchanges would draw on the same store. Groundwater that reaches the surface under an *empty* store is still returned to Raven by the seepage drain (Section 5.2).

Worked example: wetland leakage

Group HRUs cover 3 km² of a cell, $L = 0.05 \text{ d}^{-1}$: $C \leq 0.05 \times 3 \times 10^6 = 150\,000 \text{ m}^2/\text{d}$. The wetland holds 20 mm ($\bar{s} = 0.02 \text{ m}$, $V = 60\,000 \text{ m}^3$ in this cell). The supply limit is $60\,000 / (1 \times 0.02) = 3 \times 10^6 \text{ m}^2/\text{d}$, so $C = 150\,000$. With the water table 1 m below the top (disconnected), leakage is $150\,000 \times 0.02 = 3\,000 \text{ m}^3/\text{d}$, i.e. 1 mm/d over the 3 km².

Reference card: wetlands and lakes

Sources/sinks. Exchanges water in both directions between a Raven storage state variable (e.g. DEPRESSION, LAKE_STORAGE, SOIL[0]) of the HRUs in an HRU group and the aquifer (MODFLOW RIV package SWX_RAVEN, top cell).

Constraints/notes. The conductance is limited by the water stored, so a store can never be over-drawn; an empty store does not leak. A store of an HRU may be connected by one exchange only.

Command syntax

```
# .rvg
:SurfaceWaterExchange DEPRESSION WetlandHRUs LEAKANCE 0.01
# .rvh
:HRUGroup WetlandHRUs
    101, 102, 157
:EndHRUGroup
```

where DEPRESSION is the Raven store, WetlandHRUs an HRU group and 0.01 the leakance [1/d].

Parameters

Symbol	Parameter	Class (where it is set)	Units	Reference
L	LEAKANCE	:SurfaceWaterExchange argument (.rvg)	1/d	Sec. A.6
s_k	store depth	Raven state variable (the named store) of HRU k	mm	Raven state var.
–	HRU group	:HRUGroup (.rvh)	–	Raven manual

Outputs. GWBudget.csv: *surface-store leakage to aquifer, aquifer discharge to surface stores, surface-store exchange applied to Raven*; the store itself in Raven’s outputs.

5.6 Reservoirs

Raven reservoirs can already lose water to groundwater with their :SeepageParameters k [m³/s/m] and a fixed local groundwater level h_{loc} :

$$V^{\text{res}} = k \left(\frac{1}{2} (h_{\text{res}}^t + h_{\text{res}}^{t+\Delta t}) - h_{\text{loc}} \right) \cdot 86\,400 \Delta t \quad [\text{m}^3], \quad (5.16)$$

using the average of the stages at the start and end of the step. :ReservoirExchange connects this to the aquifer for every reservoir that has :SeepageParameters and whose lake HRU (:HRUID) has an AQUIFER_PROFILE.

5.6.1 How the exchange works

1. After MODFLOW solves a day, the local groundwater level of the reservoir is set to the average head under its lake HRU, weighted by overlap:

$$h_{\text{loc}} = \sum_i w_i \max(z_i^{\text{wt}}, \min(z_{\text{top},i}, h_{\text{res}})), \quad w_i = \frac{A_{k,i}}{\sum_{j \notin C} A_{k,j}}, \quad (5.17)$$

where the sums run over the lake HRU’s cells that do not hold a fixed head (C), as for recharge, and z_i^{wt} is the water table of column i (the head of its highest wet cell; Section 3.9). A water table below the lake bed (the model top) means the lake is disconnected: its seepage then depends on the bed, not on how deep the water table is — exactly as for a river. If the lake stands below the model top (for example a DEM that recorded a higher water surface), the true bed is below the lake level, so the bed is taken at the lake level h_{res} : a deep water table then gives no exchange, where the model top would have given a false inflow to the lake. (Using raw heads here was an error: a dry cell’s head can be far below the cell and produced impossible seepage.)

2. Raven routes the day; the reservoir calculates its seepage V^{res} with this level and counts it as a reservoir loss (water leaving Raven).
3. The next day, this volume is injected into the aquifer cells under the lake (MODFLOW WEL package RES_RAVEN): $Q_i = V^{\text{res}} w_i / \Delta t$. A negative volume (groundwater feeding

the reservoir) is extracted. If a cell is nearly dry, MODFLOW extracts less than asked (AUTO_FLOW_REDUCE); the lake has already received that water, so the shortfall is taken back from the water reaching the reservoir’s subbasin in the same step, like a river loss (GWBudget.csv: *reservoir gain not supplied by MODFLOW*). No water is created.

The reservoir’s stages must be elevations on the aquifer’s datum (:AbsoluteCrestHeight): a stage below the bottom of the whole aquifer under the lake — relative stages, with the crest at 0 m, or another datum — stops the run. For a large lake, fine cells in and around the lake HRU place this seepage more precisely: :GridRefinement LAKES (Section 3.4). The one-day delay is exact bookkeeping, not a loss. The volume in transit is reported every day (*reservoir seepage awaiting delivery*), saved in solution.rvc (:GWReservoirSeepage), and delivered on the first day of a restarted run.

Without an initial condition in the .rvh, a reservoir with absolute stages starts at its crest height — the same state that stage 0 means for a reservoir with relative stages. (Raven previously started it at 0 m, far below the lake bed, which created a large one-time error in its water balance; this was corrected.) Stages count as absolute when :AbsoluteCrestHeight is given or when a stage–storage table starts above 0 m; a table measured from the reservoir bottom that happens to start above 0 m therefore also starts at the crest, so give such reservoirs an :InitialReservoirStage.

Watch out

The reservoir stage and the aquifer heads must use the same vertical datum. Give reservoirs absolute elevations (for simple lakes, :AbsoluteCrestHeight).

Worked example: reservoir seepage

$k = 0.2 \text{ m}^3/\text{s}/\text{m}$, average stage 1 136 m, aquifer head below the lake 1 132 m: seepage $0.2 \times 4 \times 86\,400 = 69\,120 \text{ m}^3$ that day, injected into the aquifer the next day.

Reference card: reservoirs

Sources/sinks. Exchanges water between a Raven reservoir (subbasin outlet) and the aquifer under its lake HRU (MODFLOW WEL package RES_RAVEN); Raven books the seepage as a reservoir loss.

Constraints/notes. One-step lag, exactly accounted (volume in transit reported and saved). The local head is never taken below the lake bed. Stages must be absolute. A reservoir that dries out books only the water that can actually leave.

Command syntax

```
# .rvh
:Reservoir Lake1
:SubBasinID 43
:HRUID 2736 # lake HRU, with an AQUIFER_PROFILE
:Type RESROUTE_STANDARD
:WeirCoefficient 0.6
:CrestWidth 100.0
:MaxDepth 10.0
:LakeArea 5.0E7
:AbsoluteCrestHeight 1133.0 # absolute stages (required)
:SeepageParameters 0.2 1128 # k [m3/s/m]; the head is replaced by the aquifer head
:EndReservoir
# .rvg
:ReservoirExchange
```

Parameters				
Symbol	Parameter	Class (where it is set)	Units	Reference
k	:SeepageParameters (first value)	reservoir (:Reservoir, .rvh)	m ³ /s/m	Raven manual
h_{res}	stage	Raven reservoir state; :AbsoluteCrestHeight	m	Raven state var.
h_{loc}	local groundwater head	computed each step from MODFLOW (replaces the second value)	m	Sec. 5.6
–	:ReservoirExchange	.rvg option	–	Sec. A.6

Outputs. GWBudget.csv: *reservoir seepage sent, delivered, awaiting delivery, reservoir gain not supplied by MODFLOW*; Raven’s reservoir outputs show the seepage as a loss.

5.7 Wells and boundaries

5.7.1 Pumping and injection wells

Wells are listed in :Wells with their location and screen (top and bottom elevation); pumping rates come from a time series :WellRate in the .rvt file (m³/d, negative for pumping). A screen that crosses several layers is split among them by transmissivity, so more water comes from the more permeable layers:

$$Q_{w,\ell} = Q_w \frac{T_\ell}{\sum_m T_m}, \quad T_\ell = K_\ell \left[\min(z_{\text{st}}, z_{\text{bot}}^{(\ell-1)}) - \max(z_{\text{sb}}, z_{\text{bot}}^{(\ell)}) \right]_+, \quad (5.18)$$

where $[x]_+ = \max(x, 0)$ and $z_{\text{st}}, z_{\text{sb}}$ are the screen top and bottom. If the screen misses the aquifer, the nearest active layer is used (with a warning). MODFLOW’s AUTO_FLOW_REDUCE lowers the pumping rate as a cell runs dry, so a well cannot pump water that is not there. A well placed in a fixed-head cell has no effect, and Raven warns about it. GWBudget.csv reports both the specified and the actual rates. Pumped water leaves the system.

5.7.2 Head-dependent boundaries (GHB)

A general-head boundary exchanges water with a water body outside the model — for example a large lake or the continuation of the aquifer beyond the model:

$$q = C_g (h_g - h), \quad (5.19)$$

with C_g the conductance per cell (CONDUCTANCE) and h_g the boundary head (HEAD, or a :BoundaryHead time series).

5.7.3 Fixed-head boundaries (CHD)

A specified-head boundary holds the head of its cells at h_c ; MODFLOW calculates whatever flow is needed to do so.

5.7.4 Where boundaries go

Boundaries are drawn in a GeoJSON file: polygons select the cells whose centre lies inside (tested with the even–odd ray-crossing rule, so holes work), lines select the cells they cross. With LAYERS TOP only the top active cell is used; with LAYERS ALL, every active layer. A boundary is placed only in cells whose bottom is below its head (MODFLOW requires this), and time-varying heads are kept at least 0.01 m above the cell bottom.

5.7.5 Monitoring (observation) wells

Monitoring wells are listed in :ObservationWells with a location and a screen elevation. Each is attached to the active cell of its column whose layer contains the screen elevation (or the nearest active layer). Their simulated heads are written to GWHeads.csv and can be compared with measurements through :ObservationData or :IrregularObservations GW_HEAD (matched to the observation times like any Raven observation, within :EvaluationTime) in the .rvt file; Raven then computes the usual goodness-of-fit statistics (NSE, RMSE, KGE, ...) in Diagnostics.csv.

Reference card: wells, boundaries and monitoring wells

Sources/sinks. Wells: between the aquifer (MODFLOW WEL package WEL_RAVEN) and the outside. General-head boundaries (GHB GHB_RAVEN): between the aquifer and an external water body. Fixed heads (CHD CHD_RAVEN): whatever flow holds the head. Monitoring wells move no water.

Constraints/notes. Pumping is reduced automatically as a cell runs dry (AUTO_FLOW_REDUCE); a well in a fixed-head cell has no effect (warning). Boundary heads are kept above cell bottoms. Duplicate IDs and names are rejected.

Command syntax

```
# .rvg
:Wells
  :Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_TOP, SCREEN_BOT
  1, PW_1, 59.27309, -124.73461, 1199.7, 1188.7
:EndWells
:ObservationWells
  :Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_ELEV
  101, OW_1, 59.27310, -124.73060, 1192.7
:EndObservationWells
:GeneralHeadBoundary G1 edge.geojson HEAD 1146.9 CONDUCTANCE 5000 LAYERS TOP
:SpecifiedHeadBoundary L1 lake.geojson HEAD 1226.5 LAYERS TOP
# .rvt
:WellRate 1 m3/d # negative = pumping
  1985-10-01 00:00:00 1.0 3
  -1000.0
  -1000.0
  -1200.0
:EndWellRate
:ObservationData GW_HEAD 101 m # observed heads
  ...
:EndObservationData
```

Parameters

Symbol	Parameter	Class (where it is set)	Units	Reference
Q_w	:WellRate	time series (.rvt)	m ³ /d	Sec. A.6
z_{st}, z_{sb}	SCREEN_TOP, SCREEN_BOT	:Wells table (.rvg)	m	Sec. A.6
K_ℓ	K_HORIZ	aquifer class of layer ℓ	m/d	Sec. A.2
C_g	CONDUCTANCE	:GeneralHeadBoundary (.rvg)	m ² /d	Sec. A.6
h_g, h_c	HEAD or :BoundaryHead	.rvg command; time series (.rvt)	m	Sec. A.6
–	SCREEN_ELEV	:ObservationWells (.rvg)	m	Sec. A.6

Outputs. GWBudget.csv: *wells specified, wells actual, GHB, CHD*; GWHeads.csv; GW_HEAD statistics in Diagnostics.csv.

5.7.6 Where does pumped water come from?

At first mostly from aquifer storage (a cone of depression forms); as the cone reaches rivers and seepage areas, more and more comes from reduced discharge to streams (“capture”). In the Liard pumping test 91% of the pumping was captured from reduced discharge to rivers in the first year and 98% in the fifth, and the outflow downstream fell by 98% of the pumping rate.

CHAPTER 6

Water accounting and time stepping

This chapter explains how the two models keep their water accounts consistent, what exactly happens in each time step, and how runs are started, restarted and repeated.

In plain words

Every day the two models take turns: Raven works out how much water goes down; MODFLOW moves it through the aquifer and reports what comes back up; Raven then books the returned water. Both keep a running account, and the two accounts are compared every day. This chapter also explains how to stop a run and carry on later (a restart), and how to run many versions of a model at once (an ensemble).

6.1 Keeping the books: water balance

6.1.1 MODFLOW's balance

For every step Raven computes all boundary flows from MODFLOW's own matrix terms ($q = \text{HCOF} \cdot h - \text{RHS}$) and the storage change from MODFLOW's storage package, and reports the imbalance

$$\varepsilon = Q^{\text{rch}} + Q^{\text{sto}} - Q^{\text{seep}} + Q_{\text{in}}^{\text{riv}} - Q_{\text{out}}^{\text{riv}} - Q^{\text{evt}} + Q^{\text{wel}} + Q^{\text{ghb}} + Q^{\text{chd}} + Q_{\text{in}}^{\text{swx}} - Q_{\text{out}}^{\text{swx}} + Q^{\text{res}}, \quad (6.1)$$

where Q^{sto} is water released from storage. It is written in m^3/d and as a percentage of the gross flow (the sum of the absolute values of all terms). In the tests of Chapter 9 the worst single-day error is about $5 \times 10^{-6}\%$ and the cumulative error about 10^{-10} .

6.1.2 Raven's balance

Raven tracks cumulative input I and output O and reports $\text{MB} = (S - S_0) + (O - I)$ in mm over the watershed. The coupling adds:

- **to output:** recharge (the end-of-day GROUNDWATER value of coupled HRUs, as Raven already does), and reservoir seepage (already counted by Raven as a reservoir loss);
- **to input:** every volume the coupling adds to Raven in the step — seepage returned to HRU stores, the net river exchange applied to reaches, and the net surface-store exchange. A net loss (for example a losing river) is a negative input.

With these, Raven's own balance closes exactly as without groundwater (0.43–0.52 mm maximum over 20 years in the Liard example, the same as uncoupled).

6.1.3 The whole system

The water that leaves Raven as recharge enters MODFLOW (directly or via the delay store); the water MODFLOW sends back enters Raven. Reservoir seepage enters MODFLOW one day after leaving Raven. Therefore

$$\sum V_{\text{rch}}^{\text{Raven}} = \sum V_{\text{rch}}^{\text{MF6}} + \Delta S^{\text{delay}} + \sum V_{\text{rch}}^{\text{CHD}}, \quad (6.2)$$

and the audit tool checks this identity, the river, seepage and store bookkeeping, and MODFLOW's balance for every run.

6.2 A real day of water accounts

Worked example: 15 June 1996 in the Liard run

From `GWBudget.csv` (all in m^3/d):

Inflows to the aquifer	
recharge from Raven	21 365 037
river leakage	2 978 677
Outflows from the aquifer	
seepage to Raven	19 087 047
discharge to rivers	1 331 456
storage release (negative = stored)	−3 925 212

Inflows – outflows + release = $24\,343\,714 - 20\,418\,502 - 3\,925\,212 = 0.00003 \text{ m}^3/\text{d}$, i.e. a relative error of about 10^{-10} . Raven recorded the same numbers on its side: 21.4 million m^3 of recharge leaving its stores, 19.1 million m^3 of seepage entering them, and a net river exchange of -1.65 million m^3 applied to its reaches (the rivers lost more than they gained that day). The aquifer stored the difference: a water-table rise of about 1 mm on average.

6.3 How to read the balance outputs

1. *MF6 balance error [%]* should stay well below 0.1% every step; larger values point to a non-converged step or a too-loose head tolerance (:`SolverHeadTolerance`, default 0.00001 m).
2. *Raven recharge* and *MF6 recharge* are equal unless a recharge delay or fixed-head cells are used.
3. *returned to Raven* equals *seepage to Raven*.
4. *river loss carried to next step* is normally zero; when not, it is taken from the river the next day.
5. Raven's *WatershedStorage.csv* MB error should be as small as without groundwater.

The tool `tools_audit.py` checks all of these automatically.

6.4 The coupled time step, start-up and restarts

6.4.1 The algorithm

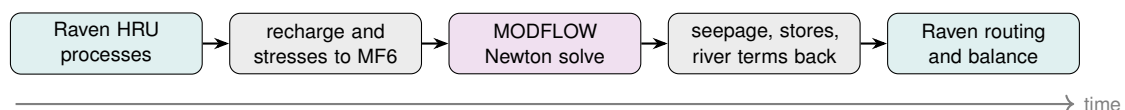


Figure 6.1: Order of work within one time step.

Algorithm 1 lists exactly what happens in one Raven time step of a coupled model.

Algorithm 1 One coupled time step

-
- 1: Raven: vertical and lateral processes of all HRUs; coupled HRUs' GROUNDWATER now holds the day's recharge
 - 2: Complete yesterday's line of GWBudget.csv (river and reservoir terms are final only after routing)
 - 3: Recharge: delay reservoirs (Ch. 5.1); spread over cells; skip fixed-head cells
 - 4: MODFLOW prepare_time_step; set RCH rates
 - 5: Set river stages and conductances, well rates, GHB/CHD heads (Ch. 5.3,5.7)
 - 6: Set water-table ET rates from PET left unmet by Raven (Ch. 5.4)
 - 7: Deliver yesterday's reservoir seepage (Ch. 5.6); set surface-store levels and limited conductances (Ch. 5.5)
 - 8: MODFLOW prepare_solve; **repeat** solve **until** converged or the iteration limit; finalize_solve; finalize_time_step
 - 9: Compute every package flow as $HCOF \cdot h - RHS$; reservoir local heads for today's routing
 - 10: Surface-store exchange applied to Raven stores; seepage and river flows computed
 - 11: **if** the step did not converge **then** limit seepage and river gains returned to Raven to the step's total inflow; warn
 - 12: **end if**
 - 13: Seepage added to Raven stores; balance, heads, connectivity and NetCDF output written
 - 14: Raven: before routing, river gains added to / losses taken from each subbasin's in-catchment inflow
 - 15: Raven routing; losses not covered are taken from the flow entering each reach; reservoirs compute seepage
 - 16: Raven: water balance (recharge out, all returned water in); diagnostics including GW_HEAD
-

6.4.2 Starting conditions

Initial heads are chosen in this order: (1) a hotstart block :GWHeads in the .rvc file; (2) :InitialGWHeads in the .rvc file (DEPTH_BELOW_SURFACE d or ELEVATION z); (3) each profile's INITIAL_HEAD_DEPTH. Guessed initial heads can be far from reality; :GWInitialization STEADY_STATE r first solves a steady state before Raven's first day, with recharge r [mm/d] over the area covered by coupled HRUs (cell i receives $r a_i / 1000$ m³/d) and the initial river stages, well rates and boundary heads. This is skipped on a hotstart. A steady state may not exist — for example when wells pump more than the aquifer can supply — and then the solve does not converge; Raven warns and starts from the initial heads instead, as if :GWInitialization had not been given. The groundwater model is built after *all* input files, including the .rvc, have been read.

6.4.3 Restarting a run (hotstart)

At the end of a run Raven writes solution.rvc. Besides Raven's own states it contains :GWHeads (the head in every cell), :GWRechargeStore (recharge delay stores, if used), :GWRiverLossCarry (river loss carried, if any) and :GWReservoirSeepage (reservoir seepage in transit, if used). Using this file as the .rvc of a new run — with :StartDate set to the end of the first run — continues exactly: in the tests the restarted run matched a continuous run to within 6×10^{-6} in all flows.

6.4.4 Ensembles

Raven’s ensemble modes (Monte Carlo, DDS, EnKF) run the model several times in one program run. For every member the groundwater model is rebuilt from scratch, MODFLOW is restarted, and its files go to that member’s output folder. Raven itself was corrected so that each member starts with a fresh water balance and, unless its own `.rvc` says otherwise, from the routing memory and reservoir states of the first run (stage, outflow, loss terms). These defaults are restored before the member’s `.rvc` is read, so member-specific initial conditions (as used by EnKF forecasting) still apply; groundwater hotstart values are likewise cleared, so each member uses only its own `.rvc`, and the steady-state start is decided for each member separately (Section 10.4).

6.4.5 Safeguards

- **Non-converged steps.** All water returned to Raven in such a step (seepage plus river gains) is limited to the water that entered the aquifer in that step (recharge, river and store leakage, reservoir injection and boundary inflows); discharge into wetland and lake stores is limited the same way. The step is flagged and a warning written.
- **Rivers and stores** can never lose more water than they hold (Sections 5.3,5.5).
- **Wells** cannot pump from a dry cell (AUTO_FLOW_REDUCE).
- **Step count:** Raven stops if it would run past MODFLOW’s last step.
- **MODFLOW folder:** if Raven cannot enter the MODFLOW folder, the call fails with a clear message instead of running MODFLOW in the wrong place.

CHAPTER 7

Inputs and outputs

This chapter is the reference for every command and every output, followed by a complete example set-up.

In plain words

This is the look-up chapter: every command you can write in the input files, what it does and its default, and every output file with its columns. There is no need to read it from start to finish; come here when you need the exact spelling of a command or the meaning of an output column.

7.1 Input reference

This chapter lists every command. Commands are case sensitive; separators may be commas or spaces, as in all Raven files.

7.1.1 .rvi (main input file)

Table 7.1: Commands in the .rvi file.

Command	Meaning
:GroundwaterModel MODFLOW6	switches the coupling on (NONE: off, the default)
:rvg_Filename file	name of the groundwater file, relative to the .rvi (default <model>.rvg)
processes into GROUNDWATER	any process moving water into GROUNDWATER in a coupled HRU produces recharge; use :->Conditional HRU_GROUP so uncoupled HRUs keep their own baseflow

7.1.2 .rvh (HRUs and subbasins)

The existing AQUIFER_PROFILE column of :HRUs holds a profile name, or [NONE]. Reservoirs to be coupled need :SeepageParameters and a lake HRU (:HRUID) with an AQUIFER_PROFILE.

7.1.3 .rvp (parameters)

```
:AquiferClasses
:Attributes, K_HORIZ, K_VERT, SPEC_STORAGE, SPEC_YIELD, POROSITY
:Units, m/d, m/d, 1/m, none, none
GRAVEL, 50.0, 5.0, 1.0E-5, 0.25, 0.30
CLAY_TILL, 0.001, 0.0001, 1.0E-4, 0.03, 0.40
:EndAquiferClasses
:AquiferProfiles
# name, number of layers, then {class, thickness [m] or TO_BEDROCK, layer type} for each layer
VALLEY_ALLUVIUM, 3, GRAVEL, 15.0, AQUIFER, CLAY_TILL, 5.0, AQUITARD, SAND, TO_BEDROCK,
    CONFINED_AQUIFER
UPLAND_TILL, 1, UPLAND_TILL, TO_BEDROCK, AQUIFER
:EndAquiferProfiles
:AquiferProfileParameters
:Attributes, INITIAL_HEAD_DEPTH, SEEPAGE_LEAKANCE, RECHARGE_DELAY
:Units, m, 1/d, d
```

```

    VALLEY_ALLUVIUM, 3.0, 1.0, 0.0
:EndAquiferProfileParameters

```

Table 7.2: Aquifer class attributes (.rvp).

Aquifer class attribute	Default	Meaning
K_HORIZ	required	horizontal hydraulic conductivity [m/d]
K_VERT	$K_h/10$	vertical hydraulic conductivity [m/d]
SPEC_STORAGE	10^{-5}	specific storage [1/m]
SPEC_YIELD	0.1	specific yield [–]
POROSITY	0.3	total porosity [–] (reserved for transport)

Layer types: AQUIFER (or UNCONFINED_AQUIFER), CONFINED_AQUIFER, AQUITARD (or CONFINING_LAYER), AQUICLUDE (not as top layer). Classes must be defined before the profiles that use them.

Table 7.3: Aquifer profile parameters (.rvp).

Profile parameter	Default	Meaning
INITIAL_HEAD_DEPTH	5 m	initial depth to water below land surface
DEFAULT_BEDROCK_DEPTH	50 m	bedrock depth where no bedrock map is given
SOIL_ZONE_DEPTH	0 m	model top below land surface
SEEPAGE_LEAKANCE	1 d^{-1}	seepage conductance per m^2 of cell
SEEPAGE_SMOOTHING_DEPTH	0.5 m	depth below the top where seepage starts
RIVERBED_K	0.5 m/d	riverbed hydraulic conductivity
RIVERBED_THICKNESS	1 m	riverbed thickness
EXTINCTION_DEPTH	0 m	water-table ET extinction depth (0 = off)
RECHARGE_DELAY	0 d	recharge delay time constant (0 = off)

7.1.4 .rvg (groundwater file)

Table 7.4: Commands in the .rvg file.

Command	Meaning (default)
:HRUGeometry file {ID field}	HRU polygons, GeoJSON in lon/lat; ID field (HRU_ID). Required.
:RiverGeometry file {subbasin field}	river lines; without a field, subbasins are assigned automatically
:LandSurface file	DEM, ESRI ASCII grid in lon/lat (HRU ELEVATION)
:BedrockSurface file	bedrock elevation, same format (DEFAULT_BEDROCK_DEPTH)
:GridCellSize m	cell size (1000 m)
:MinCellCoverage f	fraction of a cell that coupled HRUs must cover (0.25)
:GridType REGULAR QUADTREE HRU_MESH FILE	kind of grid (REGULAR); Section 3.4
:MF6Simulation file	couple an existing MODFLOW 6 simulation (Chapter 4)
:MF6Model, :MF6StartDate	model to couple (if several); date of the model's time zero

Table 7.4: Commands in the .rvg file (continued).

Command	Meaning (default)
:MF6CRS, :MF6GridFile, :OverlapWeights	how HRUs are linked to the model's cells (one of them)
:RavenTakesOver, :KeepPackages	recharge/ET/UZF packages replaced by Raven or kept
:StreamPackage p {AUX a DOMINANT_HRU}, :SeepageToRaven ON OFF	stream package to Raven's reaches; seepage package
:GridRotation θ	rotation of regular and quadtree grids, degrees counterclockwise (0)
:GridRefinement RIVERS WELLS LAKES s, LINES POLYGONS file s	finer cells (size s [m]) along rivers, around wells, in and around reservoir lakes, or along/around features in a file
:GridFile file {ID}	imported cell polygons, with :GridType FILE (ID field CELL_ID)
:QuadtreeMaxLevel n	maximum halvings of the base cell (from the refinement sizes)
:FlowCorrection AUTO XT3D XT3D_RHS NONE	XT3D conductance tensor (AUTO: off; XT3D_RHS or XT3D to switch it on)
:LayerConnection INDEX ELEVATION	connect layers by number or by overlapping elevation (INDEX)
:MF6Library path	MODFLOW 6 library (else the environment variable RAVEN_MF6_LIB, next to the Raven executable, then lib/mf6/, then the system path; Section 10.6)
:SeepageReturnTo SV	Raven store receiving seepage (lowest soil layer; soil-less HRUs: SURFACE_WATER)
:RiverBedDepth m REFERENCE_FLOW	riverbed depth below the DEM (REFERENCE_FLOW)
:GWInitialization STEADY_STATE r	steady-state start with recharge r mm/d (off)
:SurfaceWaterExchange SV group LEAKANCE L	Raven store of an HRU group $\leftrightarrow$ aquifer
:ReservoirExchange	Raven reservoirs $\leftrightarrow$ aquifer
:Wells... :EndWells	table: ID, NAME, LATITUDE, LONGITUDE, SCREEN_TOP, SCREEN_BOT
:ObservationWells...	table: ID, NAME, LATITUDE, LONGITUDE, SCREEN_ELEV
:GeneralHeadBoundary name file HEAD h CONDUCTANCE c {LAYERS ALL TOP}	head-dependent boundary; c per cell [m^2/d]
:SpecifiedHeadBoundary name file HEAD h {LAYERS ALL TOP}	fixed-head boundary
:SolverMaxOuterIterations n	Newton iterations per step (500)
:SolverHeadTolerance m	Newton head-change tolerance (0.00001 m)
:HeadSaveFrequency n	steps between saves of the full head field (30)
:WriteNetCDFHeads	write GWHeads.nc (Raven compiled with NetCDF)
:LinkageCache file	linkage cache location (<output>/mf6/linkage.cache)

The block commands take one row per item, for example:

```
:Wells
:Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_TOP, SCREEN_BOT
1, PW_1, 59.27310, -124.73460, 1199.7, 1188.7
:EndWells
:ObservationWells
```

```
:Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_ELEV
101, OW_NEAR, 59.27310, -124.73060, 1192.7
:EndObservationWells
:GeneralHeadBoundary GHB1 boundary_line.geojson HEAD 1146.85 CONDUCTANCE 5000.0 LAYERS TOP
:SpecifiedHeadBoundary LAKE1 lake.geojson HEAD 1226.5 LAYERS TOP
:SurfaceWaterExchange DEPRESSION WetlandHRUs LEAKANCE 0.01
:ReservoirExchange
```

7.1.5 .rvt (time series)

```
:WellRate 1 m3/d # well ID; negative = pumping
1985-10-01 00:00:00 1.0 1826
0.0
...
:EndWellRate
:BoundaryHead GHB1 m # replaces HEAD of boundary GHB1
1985-10-01 00:00:00 1.0 1826
...
:EndBoundaryHead
:ObservationData GW_HEAD 101 m # observation-well ID; head AT each date
1985-10-01 00:00:00 1.0 1826
...
:EndObservationData
```

7.1.6 .rvc (initial conditions)

```
:InitialGWHeads DEPTH_BELOW_SURFACE 5.0 # or FROM_PROFILE (default) or ELEVATION z
```

:GWHeads nlay nrow ncol, :GWRechargeStore, :GWRiverLossCarry and :GWReservoirSeepage are written by Raven into solution.rvc for hotstart; they are read back automatically.

7.2 Output reference

All rows are stamped with the date at the *end* of the step (the moment the values refer to).

7.2.1 GWBudget.csv — one row per step

One row of GWBudget.csv from the 20-year Liard run (selected columns):

Column	Value
time [d], date	3910, 1996-06-15
Raven recharge [m3/d]	21 365 037.27
MF6 recharge [m3/d]	21 365 037.27
seepage to Raven [m3/d]	19 087 056.98
storage release [m3/d]	−3 925 200.44

Recharge leaving Raven equals recharge entering MODFLOW; on this wet June day the aquifer stores about 3.9 million m³.

7.2.2 Other files

Table 7.5: Columns of GWBudget.csv.

Column	Meaning
time, date	model time [d] and date at the end of the step
Raven recharge [m^3/d]	recharge leaving Raven (before any delay)
MF6 recharge [m^3/d]	recharge entering MODFLOW
seepage to Raven [m^3/d]	drain discharge at the model top
storage release [m^3/d]	water released from aquifer storage (negative: stored)
MF6 balance error [m^3/d], [%]	ε of Section 6.1, and relative to gross flow
returned to Raven [m^3]	seepage volume added to Raven stores
outer iterations, converged	Newton iterations used; 1 target tolerance reached, 2 accepted at the relaxed tolerance, 0 not converged
recharge delay storage [m^3]	water in delay reservoirs
recharge onto CHD cells [m^3/d]	recharge of HRUs lying entirely on fixed-head cells
river leakage to aquifer, aquifer discharge to rivers [m^3/d]	river exchange, each direction
river exchange applied to Raven [m^3]	net volume actually added to (or taken from) Raven reaches
river loss carried to next step [m^3]	river loss not supplied in the step, taken from the river the next step (normally 0)
water-table ET [m^3/d]	evaporation from the water table
wells specified, wells actual [m^3/d]	requested and delivered well rates
GHB, CHD [m^3/d]	boundary flows (positive into the aquifer)
surface-store leakage / discharge [m^3/d], applied [m^3]	wetland and lake exchange
reservoir seepage sent, delivered [m^3/d], awaiting delivery [m^3]	reservoir exchange
reservoir gain not supplied by MODFLOW [m^3]	a gain the lake received that a drying cell could not supply; taken back from the water reaching the subbasin in the same step (normally 0)

Table 7.6: Other output files.

File	Content
GWHRUState.csv	per coupled HRU: overlap-weighted mean water table of its columns and mean depth to water
GWHeads.csv	head at each monitoring well, from $t = 0$ (missing value -1.2345 when the screened cell is dry)
GWConnectivity.csv	number of separate wet areas; wet fraction of each profile
mf6/grid_cells.geojson	every cell as a lon/lat polygon: number, active, profile, dominant HRU, area
GWBudget.csv, last column	with an existing model: net flow of the model's own packages (wells, boundaries, ...) [m^3/d]
GWModelSummary.txt	MODFLOW 6 library and its version, grid type and size, MODFLOW format, HRU fidelity of the cells, active and pass-through cells, profiles, river/well/boundary/store/reservoir cells, linkage per HRU (areas, fraction linked, cells), whether the linkage came from the cache
GWHeads.nc	(optional) head(time, layer, y, x) with 2-D latitude and longitude, CF conventions
mf6/	the generated MODFLOW model, its list file (gwf.lst, mfsim.lst) and binary heads (gwf.hds)
Diagnostics.csv	Raven's usual file, now also with GW_HEAD statistics
WatershedStorage.csv	Raven's balance; includes all exchanged water
solution.rvc	includes :GWHeads, :GWRechargeStore, :GWReservoirSeepage

7.3 A complete example set-up

The following .rvg uses every option; lines not needed can be deleted.

```
# ---- geometry ----
:HRUGeometry my_hrus.geojson HRU_ID
:RiverGeometry my_rivers.geojson SubId
:LandSurface dem_lonlat.asc
:BedrockSurface bedrock_lonlat.asc
# ---- grid and solver ----
:GridCellSize 1000.0
:MinCellCoverage 0.25
:SolverHeadTolerance 0.00001
:SolverMaxOuterIterations 500
:HeadSaveFrequency 30
:WriteNetCDFHeads
:LinkageCache cache/linkage.cache
# ---- coupling options ----
:MF6Library /opt/mf6/libmf6.so
:SeepageReturnTo SOIL[2]
:RiverBedDepth REFERENCE_FLOW
:GWInitialization STEADY_STATE 0.5
:SurfaceWaterExchange DEPRESSION WetlandHRUs LEAKANCE 0.01
:ReservoirExchange
# ---- stresses and observations ----
:Wells
:Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_TOP, SCREEN_BOT
1, TownWell, 49.123, -80.456, 245.0, 230.0
:EndWells
:ObservationWells
:Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_ELEV
101, OW_1, 49.120, -80.450, 240.0
:EndObservationWells
```

```
:GeneralHeadBoundary East east_edge.geojson HEAD 210.0 CONDUCTANCE 2000.0 LAYERS ALL
:SpecifiedHeadBoundary Lake lake.geojson HEAD 255.0 LAYERS TOP
```

with, in the .rvt,

```
:WellRate 1 m3/d
  2000-01-01 00:00:00 1.0 3
  -1500.0
  -1500.0
  -1800.0
:EndWellRate
:ObservationData GW_HEAD 101 m
  2000-01-01 00:00:00 1.0 3
  241.20
  241.18
  -1.2345
:EndObservationData
```

(-1.2345 marks a missing observation, as elsewhere in Raven). Only three days are shown: like every Raven forcing series, a :WellRate series must cover the whole simulation, and its time interval must divide evenly into one day (Raven stops with a message otherwise). Observation series may be shorter.

CHAPTER 8

Applying the coupling

This chapter takes the user from data to a calibrated coupled model: a complete worked example with the Liard River model, how to prepare data for another basin, a checklist and typical parameter values, calibration, and troubleshooting.

In plain words

This chapter walks through a real example — the Liard River in northern Canada — from input files to results. It then shows how to set up your own watershed, which values to start from, how to calibrate, and what to do when something goes wrong.

8.1 Tutorial: the Liard River model

This chapter walks through the example supplied in `examples/Liard_groundwater`: the Liard River model (65 subbasins, 1 107 HRUs, northern British Columbia and Yukon) with an aquifer added under three nested subbasins (52, 48 and 43; about 13 800 km², 85 coupled HRUs). The other 62 subbasins run exactly as before.

8.1.1 Step 0: prepare

1. Build Raven from this source (CMake: `mkdir build; cd build; cmake ..; make; or make in src/`; on Windows open `src/Raven.sln`).
2. Get the MODFLOW 6 library (6.5 or newer; tested with 6.8.1 and 6.6.3; 6.8 or newer recommended): run `python tools/get_mf6.py` in the source folder, or build with CMake, which fetches it (Section 10.6). Raven then finds it without any setting.
3. The original Liard package was made for Raven 2.9.1; for Raven 4.x its Windows paths use `/` and the parameter `DEP_THRESHOLD` is spelled `DEP_THRESHOLD`. The example files already include these changes.

8.1.2 Step 1: mark the HRUs with groundwater (.rvh)

In the `:HRUs` table, the `AQUIFER_PROFILE` column was filled for the HRUs of subbasins 52, 48 and 43: `VALLEY_ALLUVIUM` for forest, shrub and wetland HRUs; `UPLAND_TILL` for barren HRUs; glaciers keep `[NONE]`. A group listing the same HRUs was added at the end of the file:

```
:HRUGroup AquiferHRUs
  2736, 2745, 2747, ...
:EndHRUGroup
```

8.1.3 Step 2: describe the aquifer (.rvp)

Added at the end of `Liard.rvp`:

```
:AquiferClasses
:Attributes, K_HORIZ, K_VERT, SPEC_STORAGE, SPEC_YIELD, POROSITY
:Units, m/d, m/d, 1/m, none, none
GRAVEL, 50.0, 5.0, 1.0E-5, 0.25, 0.30
CLAY_TILL, 0.001, 0.0001, 1.0E-4, 0.03, 0.40
SAND, 10.0, 1.0, 1.0E-5, 0.20, 0.35
UPLAND_TILL, 0.2, 0.02, 1.0E-4, 0.08, 0.30
:EndAquiferClasses
```

```
:AquiferProfiles
  VALLEY_ALLUVIUM, 3, GRAVEL, 15.0, AQUIFER, CLAY_TILL, 5.0, AQUITARD, SAND, TO_BEDROCK,
    CONFINED_AQUIFER
  UPLAND_TILL, 1, UPLAND_TILL, TO_BEDROCK, AQUIFER
:EndAquiferProfiles

:AquiferProfileParameters
  :Attributes, INITIAL_HEAD_DEPTH, DEFAULT_BEDROCK_DEPTH, SEEPAGE_LEAKANCE, SOIL_ZONE_DEPTH
  :Units, m, m, 1/d, m
  VALLEY_ALLUVIUM, 3.0, 60.0, 1.0, 0.0
  UPLAND_TILL, 8.0, 15.0, 1.0, 0.0
:EndAquiferProfileParameters
```

The valley aquifer is a gravel layer (15 m) over a clay-till aquitard (5 m) over sand down to bedrock; the uplands have one till layer to shallow bedrock.

8.1.4 Step 3: switch the coupling on and send recharge to it (.rvi)

```
:GroundwaterModel MODFLOW6
:DefineHRUGroups AquiferHRUs
...
:Percolation PERC_CONSTANT FAST_RESERVOIR SLOW_RESERVOIR
  :-->Conditional HRU_GROUP IS_NOT AquiferHRUs # existing process, now only for uncoupled HRUs
:Percolation PERC_CONSTANT FAST_RESERVOIR GROUNDWATER
  :-->Conditional HRU_GROUP IS AquiferHRUs # new: this water becomes recharge
```

Seepage returns to the lowest soil layer (SLOW_RESERVOIR), whose existing baseflow process carries it to the rivers; aquifer–river exchange goes directly to the reaches.

8.1.5 Step 4: the groundwater file (.rvg)

```
#####
# Liard.rvg - Raven-MODFLOW 6 groundwater file (Liard River test case)
# Groundwater under Central Liard subbasins 52 -> 48 -> 43 (85 HRUs).
# Hydrogeology (aquifer classes, profiles) is in Liard.rvp;
# which HRUs have groundwater is set by AQUIFER_PROFILE in Liard.rvh.
#####

# --- geometry (stand-in data built by tools_build_standin_inputs.py; replace with real data) ---
:HRUGeometry Liard_HRUs_standin.geojson HRU_ID # HRU polygons, lon/lat
:RiverGeometry Liard_rivers.geojson # river lines (subbasin assigned automatically)
:LandSurface Liard_dem_standin.asc # DEM, ESRI ASCII, lon/lat
:BedrockSurface Liard_bedrock_standin.asc # bedrock elevation, same format

# --- grid and coupling settings ---
:GridCellSize 2000.0 # m
:MinCellCoverage 0.25 # fraction of a cell covered by coupled HRUs to activate it
:HeadSaveFrequency 365 # steps between saves of the full head field (mf6/gwf.hds)
# :MF6Library /path/to/libmf6.so # only for a library outside lib/mf6/ (or RAVEN_MF6_LIB)

# --- optional features (uncomment to use; see the user guide) ---
# :GWInitialization STEADY_STATE 0.45 # steady-state start, uniform recharge [mm/d]
# :SurfaceWaterExchange DEPRESSION WetlandHRUs LEAKANCE 0.01 # wetlands <-> aquifer
# :WriteNetCDFHeads # GWHeads.nc (Raven built with -Dnetcdf)
# :ReservoirExchange # Raven reservoirs (with :SeepageParameters) <-> aquifer
# :Wells
# :Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_TOP, SCREEN_BOT
# 1, PW_1, 59.27310, -124.73460, 1199.7, 1188.7
# :EndWells # rates: :WellRate 1 m3/d ... in Liard.rvt
# :ObservationWells
# :Attributes, NAME, LATITUDE, LONGITUDE, SCREEN_ELEV
```

```
# 101, OW_NEAR, 59.27310, -124.73060, 1192.7
# :EndObservationWells # data: :ObservationData GW_HEAD 101 m ... in Liard.rvt
# :GeneralHeadBoundary GHB1 boundary_line.geojson HEAD 1146.85 CONDUCTANCE 5000.0 LAYERS TOP
# :SpecifiedHeadBoundary LAKE1 lake.geojson HEAD 1226.5 LAYERS TOP
```

The HRU polygons, DEM and bedrock of this example are *stand-ins*: the Liard package has no HRU polygons, DEM or bedrock map, so `tools_build_standin_inputs.py` made Voronoi polygons around the HRU centroids clipped to each subbasin, a DEM interpolated from HRU elevations, and bedrock 60 m (valley), 15 m (upland) and 5 m (glacier) below it. For a real study, replace them with real data.

8.1.6 Step 5: run

```
cd examples/Liard_groundwater
../../Raven.exe Liard -o output/
python3 tools_audit.py output # optional automated check
```

The 20-year daily run takes about 15 minutes on one processor core (35 seconds without groundwater). Raven builds a grid of 3 layers $\times$ 85 $\times$ 54 cells of 2 km, 3 252 active columns, 178 river cells (Figure 3.1).

8.1.7 Step 6: look at the results

- `GWModelSummary.txt`: check the grid, the number of active cells and that every coupled HRU is linked.
- `GBudget.csv`: the daily groundwater balance (Figure 8.1); the error column should stay tiny.
- `GWHRUState.csv`: depth to water under each HRU (Figure 8.2 shows the map from `mf6/gwf.hds`).
- `Hydrographs.csv`: streamflow now includes groundwater discharge (Figure 8.3).

Without recalibration and with stand-in geometry, the Nash–Sutcliffe efficiency at gauges 10BE004, 10BE005 and 10BE010 changes from 0.867, 0.835, 0.681 to 0.850, 0.821, 0.634. Recalibrating with flow and head data is the natural next step.

8.2 Preparing the input data

8.2.1 HRU polygons

Raven’s HRUs usually come from a GIS overlay (for example BasinMaker): subbasins intersected with land-use, soil and terrain classes. Export that polygon layer, keeping the HRU ID used in the `.rvh`:

```
ogr2ogr -f GeoJSON -t_srs EPSG:4326 my_hrus.geojson my_hrus.shp
```

(or, in QGIS, *Export* $\rightarrow$ *Save Features As*, format GeoJSON, CRS EPSG:4326). Check that every HRU with an `AQUIFER_PROFILE` appears in the file; `GWModelSummary.txt` lists, per HRU, its polygon area, its area in the `.rvh` and the fraction linked to active cells.

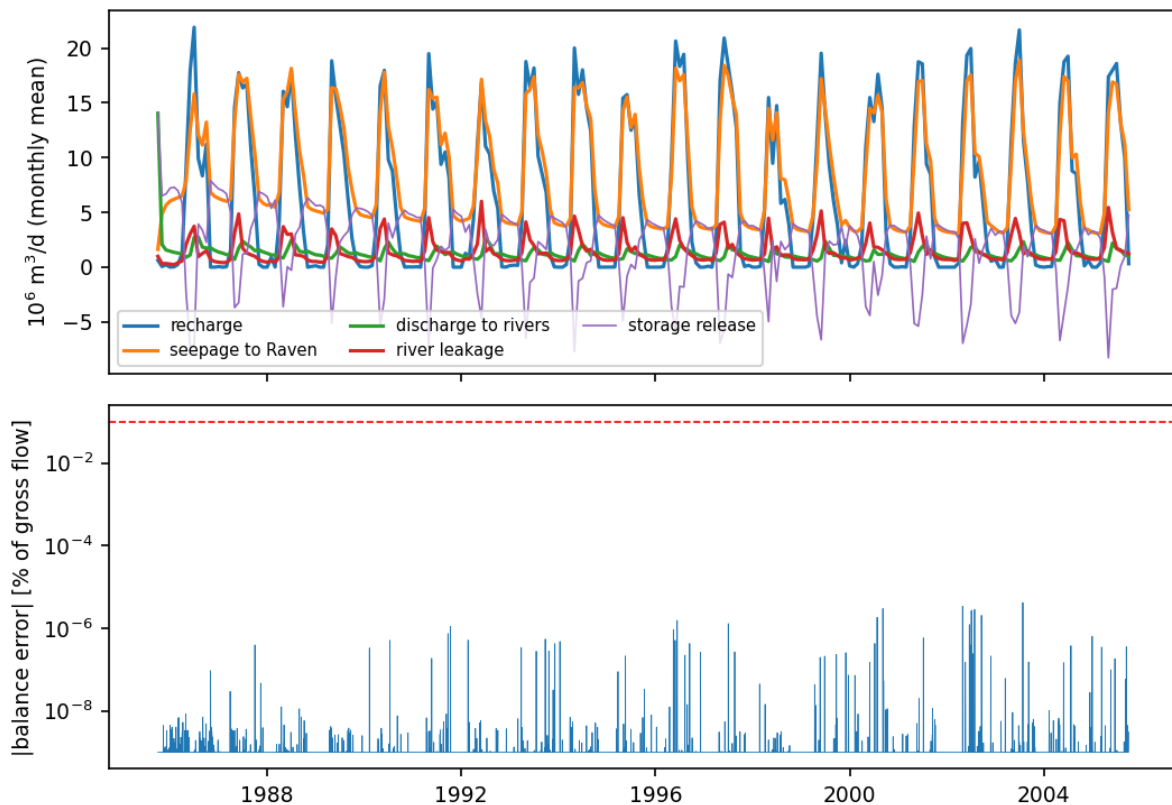


Figure 8.1: Groundwater budget of the 20-year Liard run (monthly means) and daily balance error; the dashed line is 0.1%.

8.2.2 Maps

Convert the DEM and the bedrock (or aquifer base) surface to ESRI ASCII grids in longitude/latitude:

```
gdalwarp -t_srs EPSG:4326 -tr 0.005 0.005 -r bilinear dem.tif dem_lonlat.tif
gdal_translate -of AAIGrid dem_lonlat.tif dem_lonlat.asc
```

Use a raster resolution finer than the model cells (each model cell averages 25 samples). Both maps must use the same vertical datum, as must well screens, monitoring wells, boundary heads and reservoir stages.

8.2.3 River lines

Any line layer of the river network in longitude/latitude. A field with the Raven subbasin ID is optional; without it, each river piece is assigned to the subbasin of the HRU that dominates its grid cell.

8.2.4 Aquifer information

Layer thicknesses and materials come from borehole logs, geological maps or regional hydrogeological studies. Start with one or two profiles and typical values (Section 8.4); refine only where data or calibration demand it.

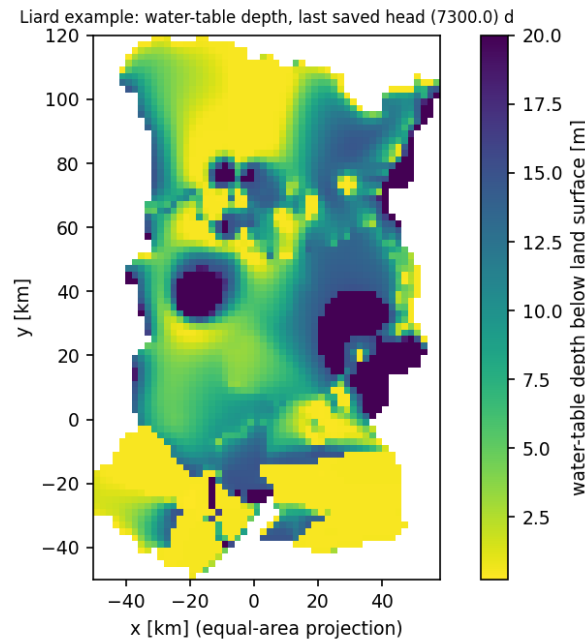


Figure 8.2: Depth to water at the last saved head.

8.3 Choosing a grid for the Liard model

The Liard model was run for two years on each grid option (Table 8.1); every run keeps the water accounts exact (MODFLOW residual about 10^{-11} , no failed step, Raven’s balance unchanged). Streamflow hardly changes: total seepage is nearly the same on every grid and the river exchange

Table 8.1: The Liard model (2 years) on each grid: active cells, run time on one core, Nash–Sutcliffe efficiency at 10BE004 / 10BE005 / 10BE010, mean seepage to Raven and mean net aquifer discharge to rivers [million m^3/d].

Grid	Cells	Time [s]	NSE	Seepage	Rivers
Regular 2 km (default)	8 840	40	0.728 / 0.861 / 0.635	9.48	0.82
Regular, rotated 20°, 1 km along rivers	35 058	220	0.726 / 0.861 / 0.638	9.92	0.66
Quadtree 2 km, 500 m along rivers	16 276	108	0.719 / 0.858 / 0.611	9.49	0.01
HRU mesh 2 km, 1 km along rivers	14 610	100	0.717 / 0.859 / 0.627	9.94	0.35
Regular 2 km, layers by elevation	8 840	41	0.666 / 0.859 / 0.606	4.11	0.49

Final code with MODFLOW 6.8.1 (tests/liard_grids.py); run times on one core of the test machine.

is a small part of the flow at the gauges. The net river exchange, however, depends on the size of the river cells — 0.82 million m^3/d with 2 km cells, 0.01 with 500 m cells — for two reasons: the channel bed is placed at the lowest DEM value along the river in each cell, which is lower in larger cells, and the head of a large cell averages the aquifer over a wider area around the river. Calibrate the riverbed parameters (RIVERBED_K, :RiverBedDepth) on the grid that will be used, and compare models only at the same river-cell size. Connecting layers by elevation halves the seepage here, because water in the upland till can now flow directly into the valley gravel where the two lie at the same elevation — a different conceptual model rather than a refinement. A regular grid refined to 1 km along rivers has more than twice the cells of a quadtree with 500 m river cells, because its refinement spans whole rows and columns.

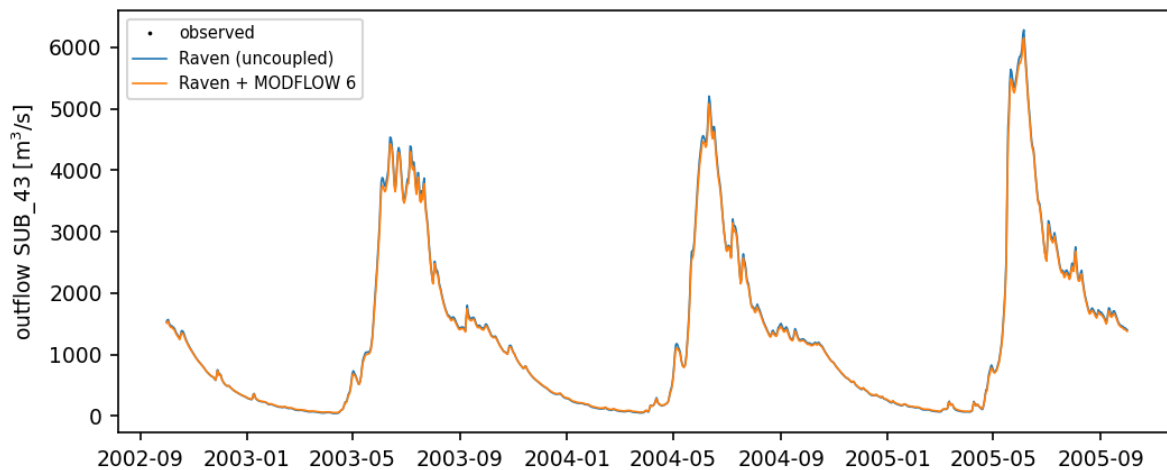


Figure 8.3: Outflow of subbasin 43, 2003–2005, without and with groundwater (not recalibrated).

8.4 Setting up your own model

8.4.1 Checklist

1. **HRU polygons.** Export the HRU polygons from your GIS or BasinMaker as GeoJSON in longitude/latitude (EPSG:4326), with a field holding the HRU ID used in the .rvh file.
2. **Maps.** A DEM and a bedrock (or aquifer-base) elevation map as ESRI ASCII grids in longitude/latitude, in the same vertical datum. River lines as GeoJSON.
3. **Profiles.** Group your HRUs by the aquifer beneath them and write one profile per group; set AQUIFER_PROFILE in the .rvh.
4. **Recharge.** Decide which Raven flux is recharge (usually percolation from the deepest soil store) and send it to GROUNDWATER for coupled HRUs only. Remove or restrict any process that drains GROUNDWATER there, and any conceptual baseflow that would double-count groundwater.
5. **Grid size.** Choose Δ so that the smallest features you care about span a few cells, but the number of active cells stays manageable (a few thousand to a few hundred thousand per layer).
6. **Start.** Use :GWInitialization STEADY_STATE with the long-term average recharge, or a warm-up period.
7. **Check.** Read GWModelSummary.txt and run tools_audit.py.
8. **Calibrate.** Use streamflow and monitoring-well heads (Section 8.6).

8.5 Typical parameter values

Typical values for aquifer materials are listed in Section A.3, and typical ranges for every aquifer class and profile parameter in Sections A.2 and A.4.

8.5.1 How much of the water balance should groundwater take?

In a coupled HRU, Raven’s conceptual baseflow from deep stores is replaced by MODFLOW. Check after a first run that the long-term baseflow (seepage plus aquifer discharge to rivers) is plausible, and that the aquifer is not steadily filling or emptying (storage release near zero on average once spin-up is over).

8.6 Calibration

Aquifer class and profile parameters are ordinary entries in the .rvp file, so they can be calibrated with Ostrich exactly like soil parameters: make a template of the .rvp with parameter names in place of values.

```
GRAVEL, par_K_gravel, 5.0, 1.0E-5, par_Sy_gravel, 0.30
UPLAND_TILL, par_K_till, 0.02, 1.0E-4, 0.08, 0.30
```

Monitoring-well heads (:ObservationData GW_HEAD) are scored in Diagnostics.csv next to streamflow, so an objective can combine both. Calibrate conductivities on a logarithmic scale. docs/ostrich_example/ contains an ostIn.txt using DDS [11], the template and run scripts. Because the linkage is cached, repeated runs skip the geometry work.

8.6.1 A synthetic test

Groundwater under subbasin 52 only (545 active columns), 2 years, 7 s per run. Heads from a run with known parameters, plus 2 cm of random noise, at four monitoring wells were the observations. The search started from values 2.5 to 10 times off. Table 8.2 shows the result. The

Table 8.2: Synthetic calibration: the parameters used to make the “observations”, the starting values, and the values DDS recovered.

Parameter	True	Start	After 150 runs	Error
Gravel K [m/d]	50	5	47.7	−4.6%
Gravel S_y	0.25	0.10	0.239	−4.5%
Upland-till K [m/d]	0.2	2.0	0.149	−25%

objective (sum of RMSE over the four wells) fell from 14.4 m to 0.17 m (it is 0.08 m at the true values, so more runs would refine it further; Figure 8.4). The upland-till conductivity is the hardest to identify because the upland heads respond weakly to it.

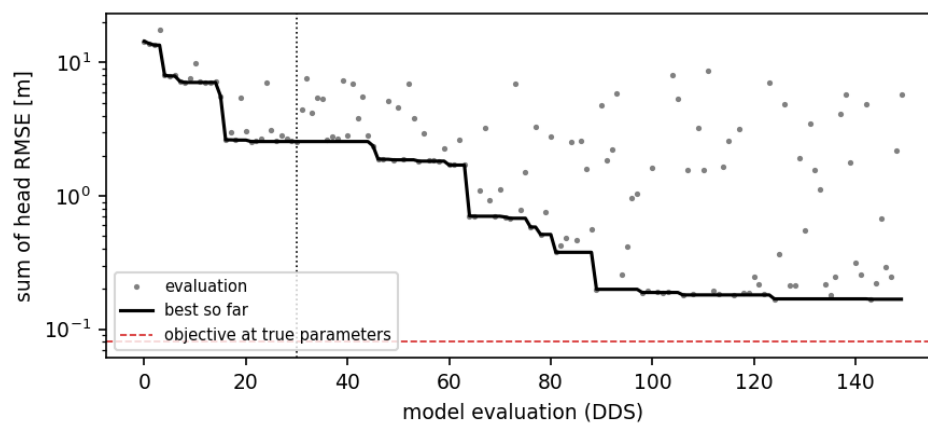


Figure 8.4: Synthetic calibration: objective against model runs.

8.7 The Ostrich input file, line by line

```

ProgramType DDS # search algorithm (dynamically dimensioned search)
ObjectiveFunction GCOP # general constrained optimization
ModelExecutable ./Ost-Raven.sh # script that runs Raven in ./model
BeginFilePairs
  Liard.rvp.tpl ; model/Liard.rvp # template -> model file with values filled in
EndFilePairs
BeginParams # name, start, lower, upper, transformations
  par_K_gravel 5.0 1.0 500.0 none log10 none
  par_Sy_gravel 0.10 0.05 0.35 none none none
  par_K_till 2.0 0.01 5.0 none log10 none
EndParams
BeginResponseVars # values read from Diagnostics.csv
  RMSE1 model/output/Diagnostics.csv ; OST_NULL 1 5 ','
  ...
EndResponseVars
BeginTiedRespVars # objective = sum of the RMSEs
  SUM_RMSE 4 RMSE1 RMSE2 RMSE3 RMSE4 wsum 1.0 1.0 1.0 1.0
EndTiedRespVars
BeginGCOP
  CostFunction SUM_RMSE
EndGCOP
BeginDDSAAlg
  MaxIterations 150
EndDDSAAlg

```

Check the row and column of each response in your own `Diagnostics.csv`; they depend on the order of observations and on `:EvaluationMetrics`. To combine heads and streamflow, add NSE or KGE responses of the hydrographs with suitable weights.

8.8 Troubleshooting

8.9 Frequently asked questions

Do I need MODFLOW experience? No. Raven writes and runs the MODFLOW model. It helps to know the concepts of Chapter 2; the generated files can be inspected by anyone who knows MODFLOW.

Can only part of my watershed have groundwater? Yes. Only HRUs with an `AQUIFER_PROFILE` are coupled; all others run exactly as before.

Can two aquifer systems exchange water? Yes, when their cells touch: lateral flow between neighbouring profiles and leakage between layers are computed by MODFLOW like any other flow.

What happens where the water table reaches the surface? The seepage drains return the water to Raven, where it follows Raven's processes (for example baseflow to the river).

Can the aquifer take more water than a river carries? No: the supply limit caps each river cell's largest possible loss at its share of the reach's flow.

How do I restart a long run? Use `solution.rvc` of the finished run as the `.rvc` of the next, with `:StartDate` set to the end of the first run.

Table 8.3: Messages and symptoms, with their causes and remedies.

Message or symptom	Cause and remedy
“cannot load MODFLOW 6 library”	run <code>python tools/get_mf6.py</code> in the source folder, or set <code>:MF6Library</code> or <code>RAVEN_MF6_LIB</code> to the full path of <code>libmf6</code> ; the message lists the places searched
Raven stops during initialization without a Raven message	MODFLOW rejected its input; read <code><output>/mf6/mfsim.lst</code>
“HRU ... has an AQUIFER_PROFILE but no polygon”	HRU IDs in the GeoJSON do not match the <code>.rvh</code> ; check the ID field name
“does not overlap any active groundwater cell”	the HRU is too small for the grid or sits where bedrock is at the surface; reduce <code>:GridCellSize</code> or <code>:MinCellCoverage</code> , or check the bedrock map
“polygon area ... differs from <code>.rvh AREA</code> ”	information only; volumes are always consistent
“process ... removes water from GROUNDWATER”	a Raven process drains GROUNDWATER in a coupled HRU; restrict it with <code>:->Conditional</code>
“process ... sends water to GROUNDWATER in HRU ... which has no AQUIFER_PROFILE”	that water would leave the model; restrict the process
“MODFLOW 6 did not converge”	start from a steady state or realistic heads; raise <code>:SolverMaxOuterIterations</code> ; increase <code>SEEPAGE_SMOOTHING_DEPTH</code> ; check for extreme parameter contrasts
Well rate is zero (“no <code>:WellRate</code> time series”)	the <code>:WellRate</code> block is missing, or its ID does not match an ID in <code>:Wells</code>
“reservoir ... has no <code>:SeepageParameters</code> ” / “no coupled lake HRU”	add <code>:SeepageParameters</code> ; give the lake HRU an AQUIFER_PROFILE
“running more time steps than the MODFLOW 6 simulation holds”	<code>:Duration</code> or <code>:TimeStep</code> is inconsistent; rerun
Water table far above the land surface Aquifer steadily emptying for years	SEEPAGE_LEAKANCE too small, or K too low for the recharge initial heads too high; use a steady-state start or a longer spin-up

How fine should the grid be? Fine enough that valleys and well spacings span two or three cells (Section 3.1). Coarser grids run faster.

How long a spin-up do I need? Start from a steady state with the long-term mean recharge, then check that *storage release* in `GWBudget.csv` averages near zero over the first years; large, slow aquifers may need decades.

Can I see the head everywhere, not only at wells? Yes: `mf6/gwf.hds` (MODFLOW binary, readable with `FloPy`) every `:HeadSaveFrequency` steps, or `GWHeads.nc` with `:WriteNetCDFHeads`.

Does the coupling change results for my other subbasins? Only through the rivers: ground-water discharge or losses in coupled subbasins change the flow passed downstream.

Can I use it in an ensemble or with Ostrich? Yes; each ensemble member rebuilds and restarts MODFLOW, and aquifer parameters are ordinary `.rvp` values.

What if MODFLOW stops the program? Read `<output>/mf6/mfsim.lst`; it names the file and value MODFLOW rejected.

CHAPTER 9

Verification and code audit

This chapter reports how the coupling was tested and what was found. Nine complementary approaches were used (Table 9.1); every problem they uncovered is listed, with its correction, in Section 9.10.

In plain words

Before anyone relies on a model, it must be shown to be right. The coupling was tested in several independent ways: automatic checks that no water is created or lost, comparisons with exact textbook solutions, comparisons with independent software, hundreds of random test set-ups, and a line-by-line review of the code. Every problem found along the way is listed at the end of the chapter with how it was fixed.

Table 9.1: Verification approaches.

Approach	What it shows	Section
Regression against Raven 4.15	models without groundwater give identical results	9.1
Automated audit of every run	exact water accounting on both sides; MODFLOW's balance	9.2
Behaviour tests	pumping, drying, restarts, ensembles and every exchange behave as expected	9.3
Grid types	geometry, format cross-checks, analytical solution, restarts and randomized tests for every grid	9.6
Existing model	projections, units, round trip, overlap weights, restarts, safeguards, randomized tests	9.7
Independent checks	geometry against standard libraries; flows against MODFLOW's own budget; grid and time-step invariance; faulty input	9.4
Randomized testing	hundreds of random set-ups combining every option	9.5
Release audit	five-step audit of the final package; checks of every MODFLOW input file and of faulty input without MODFLOW; one command repeats every test	9.8
Code audit	line-by-line review, compiler, static analysis, memory and undefined-behaviour sanitizers	9.9

9.1 Models without groundwater are unchanged

The coupling was developed on Raven 4.1.1 and then carried over to Raven 4.15 (Section 9.1.1). Models without groundwater were run with the unmodified Raven 4.15 and with the coupled code, both built with NetCDF:

- the Liard model: all outputs identical byte for byte;
- the benchmark models distributed with Raven (benchmarking/_InputFiles): the eleven that run give identical outputs, except that Lake of the Woods, whose reservoir in basin 44 dries out on 45 days, books less evaporation on those days (the correction of Section 9.10).

Its hydrographs and reservoir stages are identical; its cumulative balance error falls from 5.44 to 4.97 mm, and its `solution.rvc` carries the reservoir stages with 12 digits. The other fourteen benchmark set-ups stop with the same input error in both builds (a missing `.rvt` file, wilting point above field capacity), as they do in the unmodified Raven 4.15.

The corrections made to Raven itself (Section 9.10) affect only ensembles, reservoirs with absolute stages or that dry out, restart files, constituent observations and the `:rvg_Filename` command.

9.1.1 Carrying the coupling over to Raven 4.15

Between 4.1.1 and 4.15, 157 of Raven’s source files changed. A three-way merge (Raven 4.1.1, the coupling, Raven 4.15) left 26 overlapping edits in 22 files; each was resolved by hand:

- copyright headers (13): the 4.15 year kept, with the line naming the modification;
- observation diagnostics: 4.15’s new rule for observations without a subbasin kept, with `GW_HEAD` handled first (its location is a monitoring well);
- initial conditions: the new groundwater commands of the `.rvc` file beside 4.15’s new management commands;
- ensembles: 4.15’s `RebootTimeVariables` (which resets the balance arrays between members) kept beside the restoring of routing memory and reservoir states; the history setters now take the time step, as in 4.15;
- reservoirs: 4.15 replaced the data-assimilation scale factor by an additive adjustment; the reservoir snapshot for ensemble members and the 12-digit `solution.rvc` values follow that change;
- BMI: 4.15 had already corrected the index in `SetValueAtIndices` in the same way, so its version was taken;
- the Visual Studio project files were rebuilt from 4.15’s, with the MODFLOW-USG files replaced by the new ones.

Raven 4.15 prints its closing message (*Exiting Gracefully*) to the error stream; the test scripts read both streams. The whole test suite was then repeated on the ported code with MODFLOW 6.8.1 and 6.6.3 (Section 9.8).

9.2 Automated audit of every run

The tool `tools_audit.py`, shipped with the example, checks any output folder for:

1. **recharge:** $(\sum V^{\text{Raven}} - \sum V^{\text{MF6}} - \Delta S^{\text{delay}} - \sum V^{\text{CHD}}) / \sum V^{\text{Raven}}$;
2. **MODFLOW’s balance:** cumulative imbalance relative to the gross flow, and the worst single step;
3. **convergence:** steps that failed to converge;
4. **seepage, rivers and surface stores:** the volume applied to Raven against MODFLOW’s flow, including carried river losses;
5. **reservoirs:** the seepage sent to MODFLOW each step against the volume that left the lake the step before, and the volume MODFLOW took against the volume sent;
6. **Raven’s balance:** the largest mass-balance error in `WatershedStorage.csv`;

7. **outputs:** finite heads, contiguous end-of-step dates, a $t = 0$ row for monitoring wells.

Table 9.2 gives the results. They were first obtained before the release audit (Section 9.8); with the final code the 20-year Liard run gives outputs identical to those runs, and the reservoir and ensemble runs were repeated (Section 9.3).

Table 9.2: Automated audit (`tools_audit.py`); the reservoir and ensemble rows are from the final code's repeat (`tests/behaviour_check.py`), the 20-year Liard run is identical with the final code. Relative errors are shares of the water moved. In every run the seepage returned to Raven equals MODFLOW's seepage exactly, the surface-store bookkeeping is exact, no river loss is carried at the end and the output dates are contiguous. The one failed step is the rewetting day of the deliberately extreme drying test (Section 9.3).

Run	Steps	Recharge	MODFLOW residual	Worst step [%]	Failed steps	River	Raven balance [mm]
20-year Liard	7 305	0	-1.0×10^{-10}	4.2×10^{-6}	0	1.1×10^{-12}	0.52
Pumping	1 826	0	-1.6×10^{-10}	7.1×10^{-6}	0	-9.6×10^{-12}	0.52
Fixed head, delay	730	-3.2×10^{-11}	-3.9×10^{-10}	5.1×10^{-6}	0	7.7×10^{-12}	0.52
Restart	365	-1.7×10^{-11}	-1.9×10^{-10}	3.2×10^{-6}	0	2.1×10^{-11}	0.06
Ensemble	731	0	-1.7×10^{-11}	1.6×10^{-7}	0	9.5×10^{-12}	0.52
Wetlands	730	0	-1.1×10^{-10}	1.5×10^{-6}	0	2.2×10^{-11}	0.52
Reservoir	731	0	-1.7×10^{-11}	1.6×10^{-7}	0	9.5×10^{-12}	0.52
Soil-less HRUs	365	0	-4.5×10^{-11}	7.4×10^{-7}	0	-1.2×10^{-12}	0.52
Drying (extreme)	1 826	0	-3.1×10^{-4}	43.2	1	8.1×10^{-12}	0.52

9.3 Behaviour tests

The tests on the regular-grid Liard model were first run before the release audit. With the final code (MODFLOW 6.8.1) the 20-year run gives outputs identical to that earlier run — budget, heads, hydrographs, storage and restart file — so these results and the figures drawn from them stand. The two rows the audit's corrections could change were repeated with the final code (`tests/behaviour_check.py`): a 50 km² lake with seepage on the largest coupled HRU of subbasin 52, with a pumping well, for two years — the seepage MODFLOW received each day equals the volume that left the lake the day before (difference 0), all of it delivered, MODFLOW residual 1.7×10^{-11} ; and the same set-up as a two-member ensemble — both members identical to the single run (difference 0).

9.4 Independent checks

9.4.1 Geometry against standard libraries

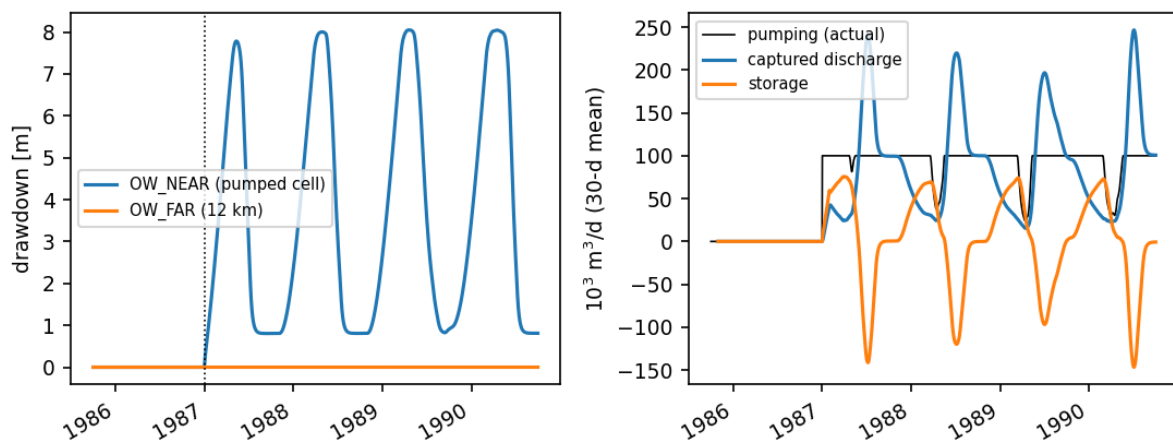
The geometry functions were compiled into a stand-alone test and compared with widely used libraries (Table 9.4).

9.4.2 Exchanged flows against MODFLOW's own budget

MODFLOW writes its own volume budget in its list file. For runs covering every package, every flow the coupling exchanged was compared with MODFLOW's figure for the same step (Table 9.5); the differences are at the printing precision of the list file.

Table 9.3: Behaviour tests.

Test	Set-up	Result
Full Liard model	20 years, 85 coupled HRUs, 3 252 active columns	recharge exact; MODFLOW residual 10^{-10} ; no failed step; Raven balance 0.52 mm
Pumping	100 000 m ³ /d well, 5 years	93% delivered; seasonal drawdown 0.8–8 m in the pumped cell, none 12 km away; 91–98% of the water captured from rivers (Figure 9.1)
Monitoring wells	synthetic heads with 5 cm noise	RMSE 4.9 cm; NSE 0.991–0.9997
Fixed-head lake	9 cells	heads held exactly; recharge on them redistributed
Restart	1 + 1 years against 2 years	flows within 6×10^{-6} (Figure 9.2)
Ensembles	2 Monte Carlo members, every option on	members identical to each other and to a single run
Drying and rewetting	valley aquifer starting dry	26 wet areas merge into 2 and split with the seasons (Figure 9.3); one rewetting step of 1 826 does not converge and is flagged
Wetlands and lakes	1 903 exchange cells	water removed from Raven equals MODFLOW's leakage exactly
Reservoir	50 km ² lake on a coupled HRU	seepage delivered each day equals Raven's seepage of the day before
Soil-less HRUs	glacier HRUs coupled	seepage goes to surface water; balance unchanged
Other features	:rvg_Filename, NetCDF, linkage cache, calibration	file found; NetCDF equals the binary heads within 10^{-4} m; cached run identical; DDS recovers gravel K and S_y within 5%

**Figure 9.1:** Pumping test: drawdown at the monitoring wells (left) and where the pumped water comes from (right).

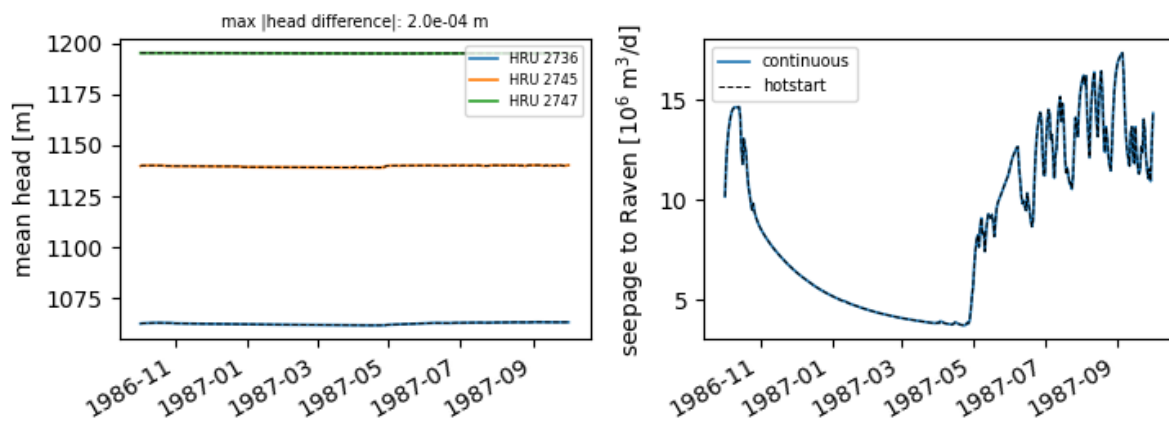


Figure 9.2: Restart: the second year of a continuous run (solid) and of a run restarted from solution.rvc (dashed).

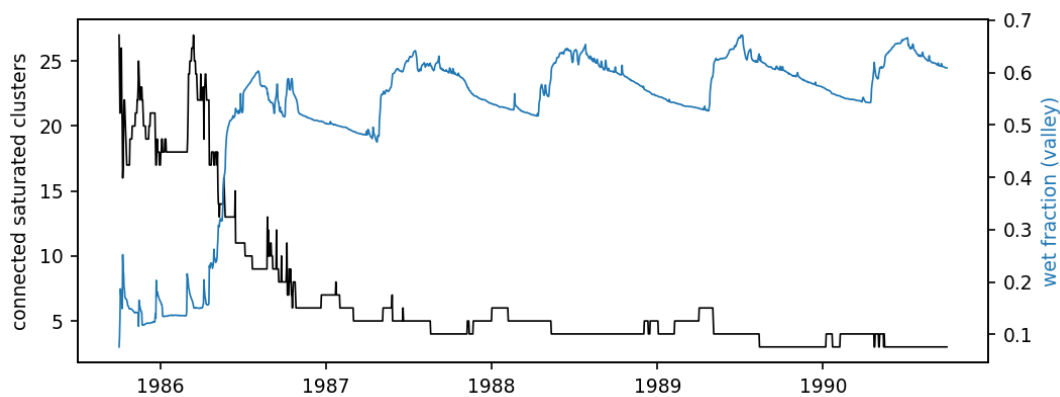


Figure 9.3: Drying and rewetting: number of separate wet areas in the top layer (black) and wet fraction of the valley aquifer (blue).

Table 9.4: Geometry functions against independent libraries.

Function	Reference	Largest difference
Polygon clipping and area (400 random, mostly concave polygons)	shapely	1.3×10^{-10}
Equal-area projection (300 points over the Liard region)	pyproj (same sphere)	5×10^{-7} m
Projection round trip	–	exact
Bilinear raster sampling with a no-data cell (200 points)	independent code	4×10^{-7}
GeoJSON and ASCII-grid variants (3-D coordinates, string IDs, holes, cell-centre headers, ...)	shapely, reference values	read correctly

Table 9.5: Coupler flows against MODFLOW’s list-file budget.

Run	Packages compared	Largest difference
20-year Liard	recharge, drains, rivers, storage	6.2×10^{-7}
Pumping and boundaries	+ wells, general-head boundaries	6.9×10^{-7}
Fixed head and delay	+ fixed heads	6.1×10^{-8}
Wetland exchange	+ store exchange	1.1×10^{-7}
Reservoir	+ reservoir seepage	2.6×10^{-7}
Drying	+ water-table ET	4.1×10^{-8}

9.4.3 Grid size and time step

The accounts stay exact for any cell size and time step (Table 9.6; subbasin 52, one year). These runs used the earlier head tolerance of 0.01 m; the 0.47% step on 4 km cells is what led to the present default of 0.00001 m.

Table 9.6: Invariance to cell size and time step.

Set-up	Active columns	Recharge	Worst step	Raven balance
1 km cells	2 147	exact	0.029%	0.456 mm
2 km cells	545	exact	0.092%	0.456 mm
4 km cells	142	exact	0.47%	0.456 mm
4 km cells, tighter tolerance	142	exact	0.002%	0.456 mm
2 km cells, half-day steps	545	exact	0.059%	0.219 mm

9.4.4 Faulty input

Thirty-nine faulty or unusual set-ups were run. Every error stops the run with a message that names the problem — no crash — and every tolerated case runs (Table 9.7).

Table 9.7: Response to faulty or unusual input.

Input	Response
Missing HRU file or ID field; unknown profile or class; malformed class row	stops; names the file, field, profile or row
Aquiclude as top layer; zero layer thickness; value out of range ($K \leq 0$, $S_y > 1$, negative delay, ...)	stops; names the parameter
Process draining GROUNDWATER in a coupled HRU; no process feeding it	stops; names the process and HRU
Wrong library path; missing raster; well outside the model	stops; names the path, file or well
Duplicate IDs or names; two exchanges on one store; boundary without HEAD	stops; names the item
Malformed groundwater command in .rvg, .rvt or .rvc	stops; names the command
Hotstart heads for a different grid; cells larger than the HRUs	stops; explains the mismatch
Monitoring well outside the model; unknown .rvg command	runs with a warning
TO_BEDROCK in a middle layer; no DEM or bedrock map; coupling switched off	runs as documented

9.5 Randomized testing

A test driver builds random but valid set-ups of the subbasin-52 model, drawing every option at random: cell size (1–3 km), minimum coverage, one-day or half-day steps, conductivities, initial depth, seepage leakance and smoothing, water-table ET, recharge delay, steady-state start, wetland exchange, 0–2 pumping wells, monitoring wells, a general-head boundary, a fixed-head area and a coupled reservoir. Every run is checked independently of the audit tool: exact recharge, seepage, river, store and reservoir identities; MODFLOW's balance (every converged step below 0.1%, cumulative below 10^{-4}); Raven's balance below 1 mm; agreement with MODFLOW's list file; and, for about a third of the set-ups, a restart that must reproduce the continuous run (every flow within 10^{-4} of its largest value in the run).

Rounds of review and randomized testing were repeated until a complete round found no model error: nine rounds and 480 random set-ups in all. The final code, with MODFLOW 6.8.1, passes the rounds that tests/run_all.py repeats: 72 of 72 set-ups over every grid type and 24 of 24 with a reservoir and lake refinement (Section 9.6).

9.6 Grid types

In plain words

Every kind of grid was checked in four ways: its cells cover the area exactly with no gaps or overlaps; two different ways of handing the same cells to MODFLOW give the same answer; the answer matches an exact textbook solution and gets closer to it as the cells get smaller; and a restarted run matches a continuous one. Existing models are not affected by the new options.

Every grid option was verified the same way as the default grid, plus four checks specific to grids.

Existing models are unchanged. With the default grid every output of the coupled Liard model (budget, hydrographs, storage, HRU state, connectivity) and the generated MODFLOW files are identical byte for byte to those of the code before the grid options were added.

Geometry of the grids. A stand-alone test checks every generator: cells tile the domain exactly (area sums equal to 10^{-15}), neighbours are symmetric, each cell's perimeter equals its shared faces plus the domain edge, Voronoi connections are exactly orthogonal, quadtree neighbours differ by one level at most, clipping handles concave cells and lattices of cocircular seeds, and grids survive the linkage cache bit for bit.

Two formats, one answer. The quadtree cells were run once as DISV (MODFLOW computes the geometry) and once imported as DISU (the coupler computes connection lengths, face widths and directions): all flows agree to 6×10^{-7} and water tables to 0.3 mm. A regular grid written as DISU gives heads identical to DIS. Solving the coupler's DISU equations independently in Python reproduces MODFLOW's heads to 3×10^{-8} m.

Analytical solution. A textbook case has an exact answer to compare with: a single well pumping from an aquifer whose water level is held fixed all around, far away (the *Thiem* solution). Close to the well the water level falls in a known way with distance. A well pumping $5000 \text{ m}^3/\text{d}$ from a confined aquifer ($T = 500 \text{ m}^2/\text{d}$) inside a 20 km square with fixed heads on its edge should give the Thiem profile $h = A + \frac{Q}{2\pi T} \ln r$. The slope fitted to all cells between four cell sizes and 3 km from the well is compared with $Q/(2\pi T)$ in Table 9.8: every grid converges

to the analytical value as its cells shrink, and the solutions are symmetric around the well (slopes in the four directions within 0.6%).

Table 9.8: Thiem test: error of the fitted slope $\partial h / \partial \ln r$.

Grid	Coarse	Medium	Fine
Regular, 500 / 250 / 125 m	+0.60%	+0.04%	+0.01%
Regular 250 m, rotated 17°		+0.03%	
Regular 1 km refined to 125 m at the wells		+1.01%	
Quadtree, base 1 000 / 500 / 250 m, 62.5 m at the wells	+0.80%	+0.59%	+0.26%
Imported cells (the 250 m quadtree)			+0.25%
HRU mesh, 500 / 250 m (250 m with XT3D)	+0.57%	+0.29%	−0.06%

Final code with MODFLOW 6.8.1 (tests/thiem); every run audited, none with a failed step.

Lakes. A reservoir with seepage to the aquifer was placed on the largest coupled HRU of subbasin 52 and each refinable grid (3 km cells) was refined around it with LAKES. The median size of the cells touching the lake fell from 3 000 to 750 m (rotated regular grid), from 3 000 to 375 m (quadtree) and from 1 210 to 490 m (HRU mesh); a POLYGONS file with the same outline gives the same mesh. The water accounts stay exact (MODFLOW residual below 1.2×10^{-11}), and the reservoir’s seepage reaches MODFLOW exactly one step after it leaves the lake (sent = previous step’s volume in transit to 1.6×10^{-10} ; delivered = sent); MODFLOW’s own list-file budget shows the same reservoir seepage. Two randomized rounds — 20 set-ups over all grid types and 12 that always have a reservoir and lake refinement — all passed every check on the code before the release audit, including the reservoir identity, which the audit tool now reports for every run with a reservoir; the final code’s rounds are reported below.

Restarts. For every grid type a 60-day run was compared with a 30-day run restarted from its solution.rvc (final code, subbasin-52 model with 3 km cells, a pumping well and half-day steps): regular 9.9×10^{-7} , rotated and refined 1.9×10^{-7} , quadtree 4.4×10^{-7} , HRU mesh 1.9×10^{-7} , layers by elevation 8.3×10^{-8} , with exact water accounting in all.

Randomized testing. The randomized set-ups of Section 9.5 were extended with a random grid (regular, rotated and refined, quadtree, HRU mesh, imported cells, layers by elevation), random rotation, refinement and XT3D choice. With the final code and MODFLOW 6.8.1, three rounds of 24 (72 set-ups: 10 regular, 11 rotated and refined, 16 quadtree, 15 HRU mesh, 6 imported, 14 with layers connected by elevation; 49 with a reservoir; 20 with a restart comparison, largest deviation 3.1×10^{-6}) and two rounds of 12 that always have a reservoir and lake refinement (24 set-ups, 6 with a restart) all pass every check, including MODFLOW’s list-file budget and the reservoir identity. One HRU-mesh set-up first failed the restart test under the original criterion, which divided each day’s difference by that day’s flow plus 1 000 m³/d: its storage release on the first restarted day was 36.95 instead of 37.11 m³/d. The criterion now scales by the flow’s largest value in the run, as the existing-model tests always did. With this scale the largest deviation is 3.1×10^{-6} .

In an earlier round on the code before the release audit, of 48 set-ups (6 regular, 2 rotated and refined, 12 quadtree, 14 HRU mesh, 7 imported, 7 with layers connected by elevation; 21 with a restart comparison, largest deviation 6×10^{-6}), 46 passed every check. The two exceptions had drawn the optional fully implicit XT3D: one failed to converge on 38 of 45 days (its accounts on the converged days are exact; on the others the safeguard limits the exchange, as designed) and one exceeded the time limit. They confirmed the decision to leave XT3D off by default; the remaining set-ups of the round drew the default correction only.

9.7 Coupling an existing model

In plain words

The existing-model mode was tested on a MODFLOW model built independently of Raven. It was also tested on a copy of that model in feet and seconds, which must give the same answers; on a model that Raven built itself and then read back in, which must reproduce Raven’s own run; with restarts; with deliberately wrong input, which must be caught; and with dozens of random set-ups. In every run no water was created or lost.

The existing-model mode was tested on the synthetic Liard model of Section 4 (a rotated UTM grid, four layers, monthly stress periods, wells, rivers and a head boundary), built independently of Raven, and on a feet/seconds copy of it. Every run keeps the water accounts exact: Raven’s recharge reaches MODFLOW exactly, the MODFLOW residual including the model’s own packages is about 2×10^{-11} , seepage and river exchange balance to 10^{-11} , and no step fails.

Table 9.9: Existing-model mode: tests and results.

Test	Result
Built-in projections against PROJ (9 systems, 18 000 points)	largest difference 3 micrometres
Cell outlines from MODFLOW (:MF6CRS) against a cell file	recharge identical; other flows within 3.5×10^{-7}
Feet and seconds against metres and days	recharge identical; other flows within 4.6×10^{-6}
Round trip: a Raven-generated model re-imported	recharge identical; outlet flow within 1.7×10^{-6}
:OverlapWeights (computed in UTM) against cell outlines	recharge identical; seepage within 4.5×10^{-7} , rivers within 4×10^{-6}
Restarts (inside a stress period; before a change of well rates)	within 1.4×10^{-7} ; the renumbered well data switch on the right date
Safeguards (14 faulty set-ups)	every one stops with a message naming the problem
Randomized set-ups (units, link, stream mapping, seepage, ET kept or taken over, coverage, restarts); checked with the audit identities and MODFLOW’s own budget file	20 of 20 pass, 6 with a restart (within 2.7×10^{-7}); MODFLOW’s saved flows of Raven’s packages agree to 10^{-10}
Sanitizers (AddressSanitizer, UndefinedBehaviorSanitizer) on the coupling’s code: both units, all three links, seepage off, restarts	no error

One feet/seconds set-up failed to converge on its first step (the steady-state period) while its metres/days twin converged: the metres model needed 288 of the 300 iterations the test model allowed and the feet model a few more, and every later step took the same number of iterations in both — the limit of the test model, not a conversion error. With 500 iterations both converge on every step. The weights test shows why the overlaps are normalised per HRU: the weights were computed in the model’s projection, whose areas differ from Raven’s by about 0.5% at this latitude, and the recharge volumes still agree exactly.

The tests of Table 9.9 were first run before the release audit. With the final code (MODFLOW 6.8.1) the two-year example gives a groundwater budget and hydrographs identical to that earlier run, and the safeguards (14 of 14), the round trip (1.7×10^{-6}), the restarts (1.4×10^{-7}) and the randomized set-ups (20 of 20) were repeated with the same results. In the example the stream package loses about 3.8 million m^3/d to the aquifer; the small headwater reaches cannot

always supply their share, so up to 106 million m³ of river loss is carried from step to step at times — all of it supplied by the end of the run, as the accounting requires.

9.7.1 USGS example models

Two models of the USGS MODFLOW 6 example collection (mf6examples.zip) were coupled as existing models with `:OverlapWeights`. They are not georeferenced, so the Liard HRUs of subbasin 52 were laid over each model's active cells, with `.rvh` areas equal to their zones so that Raven's recharge depths become the same depths on the model (tests/existing/usgs_sagehen.py, usgs_capture.py).

Sagehen Creek (a real watershed in the Sierra Nevada: 73×81 cells of 90 m, one convertible layer, Newton, a steady-state day and 398 daily periods, an unsaturated-zone package (UZF) whose rejected infiltration and groundwater discharge the water mover (MVR) sends to streams (SFR), land-surface drains also moved to SFR, three specified heads). Raven took over the UZF package, so its recharge replaced the model's infiltration, and removed the 3 375 movers from the working copy that named it. The recharge identity, the river bookkeeping and the seepage returned are exact in every case. MODFLOW's cumulative residual is 8.5×10^{-7} with the model's own loose solver (OUTER_DVCLOSE 0.03 m, INNER_RCLOSE 1 000 m³/d), 2.8×10^{-10} with a tight solver, and 1.3×10^{-11} with Raven's seepage drains beside the model's own. A restart at day 200, inside the daily periods (the mover renumbered), reproduces the continuous run to 3.3×10^{-8} . Keeping the UZF package beside Raven's recharge (recharge counted twice, the user's choice) runs with exact accounts; one day of the snowmelt peak misses the tight tolerance and is flagged. The drains as Raven's stream package are refused, because the mover already sends their water to SFR.

Stream capture (a steady-state model: 40×20 cells of 250 m, time in seconds, one stress period of one second, a river, six pumping wells, specified heads). Raven recognises the steady-state model and stretches its period over the 60-day run, so each day is a steady-state solution; the storage column stays zero. The river carries its exchange to Raven's reaches (subbasin of the dominant HRU); recharge identity and river bookkeeping are exact, MODFLOW's residual is 2.8×10^{-9} , and a restart at day 30 reproduces the continuous run to 1.0×10^{-7} . With the model's own tolerance of 10^{-8} m and 100 outer iterations, four days do not converge; they are flagged and the accounts on the other days are exact. The Liard seepage leakance of 1/d let the water table of this thin, flat-topped aquifer reach the land surface on the wettest days, where MODFLOW's Newton solver did not converge on two of them; 0.01/d, suited to its conductivity of about 7×10^{-5} m/s, converges every day.

These models found four defects of the existing-model mode, all corrected (Table 9.11): the water the model's packages pass to the mover was missing from the balance check; a mover naming a taken-over package stopped MODFLOW; a stream package feeding the mover would have been counted twice; and steady-state models with one short stress period could not be coupled.

9.7.2 A second MODFLOW version

Every test of tests/run_all.py was run with MODFLOW 6.8.1 and 6.6.3. With 6.6.3 the default Liard grid first failed to converge on 295 of 344 days: once the thin clay-till aquitard under the gravel began to dry, the Newton change of that cell repeated unchanged at every iteration. The cause was the UNDER_RELAXATION keyword of the NEWTON option, which resets a head that falls below the cell bottom; Raven now writes NEWTON alone (Section 3.9). The results with 6.8.1 are identical byte for byte, and with 6.6.3 every step passes with the same numbers: the Liard grid table agrees to the last digit shown. For existing models Raven reads the library's version and warns when a model uses the keyword with a version before 6.8.

9.8 Release audit

In plain words

Before release, the whole package was checked once more, in five steps: three independent readings of the code; builds with two compilers and for Windows; a check of every MODFLOW input file Raven writes; a battery of wrong and unusual set-ups; and a final comparison with Raven 4.1.1 under memory checkers. The problems found are listed at the end of this chapter under *Release audit*. The first four steps ran without the MODFLOW library. Once it was available, every coupled test of this chapter was repeated on the final code with one command, `tests/run_all.py`, and all of them pass.

Raven writes every MODFLOW input file, and `grid_cells.geojson`, before it loads the MODFLOW library; in the existing-model mode it also copies the model and prepares the simulation, time and name files first. Almost everything except the solve itself can therefore be checked without MODFLOW, by pointing `RAVEN_MF6_LIB` at a file that does not exist: the run then stops, with a message, just after the files are written (Table 9.10).

All results of this chapter that the corrections could change were repeated on the final code: the HRU-mesh rows (the corrected mesh has 14 610 instead of 14 398 active cells on the Liard model), the lake, reservoir and ensemble runs, the restarts, the randomized rounds and the existing-model tests. The 20-year Liard run and the two-year existing-model example give outputs identical to the runs before the audit, and the other grids of the Liard comparison reproduce their earlier numbers.

The coupled runs found one more defect and three faults in the test scripts, all corrected:

- **Raven 4.1.1 crash.** A reservoir whose `:HRUID` is an HRU of another subbasin crashed Raven. It now stops with a message, and `tests/edge.py` has a case for it.
- **Randomized harness on the wrong model.** The portable harness ran on the full three-subbasin model instead of the subbasin-52 model and placed reservoirs outside subbasin 52. This exposed the crash above; the new datum check also correctly refused lakes set below the aquifer.
- **Budget cross-check compared nothing.** Newer FloPy names list-file budget lines by package type, so the check matched no line and still passed. It now reads MODFLOW's list file by package name and fails if nothing is compared. The worst difference over 45 package budgets is 1.1×10^{-7} , at the list file's printing precision.
- **Existing-model harness.** It stored its pass flag as text, and it counted a restart comparison as run when only one of its two runs had finished.

Reservoirs that run dry. In a 20-year test with a reservoir deliberately drawn empty by a minimum flow, the reservoir's balance error is now 13.7 million m³ (158 with Raven 4.1.1, 114 with the code before the audit): 1 072 of its 1 086 dry days balance exactly. Almost all of the rest falls on the day after the reservoir empties, when the outflow still averages in the release rate at the end of the previous day although the lake is empty. This lies in Raven 4.1.1's reservoir routine, not in the coupling, and is left to the Raven developers.

Repeating the coupled tests. `python3 tests/run_all.py` runs every test of this chapter (the unit tests, the file and input checks, the Thiem, restart, lake and grid tests, the budget cross-check, both randomized harnesses and the existing-model tests) and writes one report; `--nolib` runs only the checks that need no MODFLOW library, and `--quick` uses smaller randomized rounds.

Table 9.10: Release audit: steps and results.

Step	What was checked	Result
Code review	three independent line-by-line reviews of the coupling and of every changed Raven file	every defect corrected (Table 9.11, <i>Release audit</i>)
Builds	GCC and Clang, unoptimised and optimised; cppcheck and clang-tidy; a Windows build (MinGW) run under Wine	no warnings in the coupling's files; no defects from static analysis; Windows results equal to Linux to rounding
MODFLOW input files	tests/mf6files_check.py: 16 set-ups over every grid type, rotation, refinement, layers by elevation, aquicludes, boundaries, wells, lakes and imported cells; FloPy reads every model; cells valid and not overlapping; grid origin and rotation against the cell file (0.5 m); DISV and DISU vertices; DISU connections symmetric; face angles against the vertices (1°); no vertical connection through an aquiclude; drains inside the top cell; boundary polygons select the right cells	127 of 127 checks pass (the code before the audit fails five of the set-ups)
Wrong and unusual input	tests/edge.py: 36 generated-model and 17 existing-model set-ups, judged without and with MODFLOW; the valid existing-model preparations checked in detail (stress periods split at the start date, restart renumbering, the original model untouched)	53 of 53 behave as required
Regression and memory	the 20-year Liard model without groundwater against Raven 4.1.1; AddressSanitizer and UndefinedBehaviorSanitizer on the two steps above, on seven reservoir and ensemble cases and on three years of the Liard model	outputs identical byte for byte; one uninitialised value (in 4.1.1's reservoir code) corrected; otherwise no error
Coupled tests	tests/run_all.py with MODFLOW 6.8.1 and 6.6.3: unit tests, file and input checks, Thiem, restarts, lakes, the Liard grids, reservoir and ensemble behaviour, MODFLOW's list-file budget, randomized rounds, the existing-model tests and two USGS example models	every step passes with both versions (details in the sections above)

9.9 Code audit

- **Review:** line-by-line reading of the new files and of every changed Raven file — units, signs, indices, order of operations, and that each volume is applied once on each side — and a search for orphaned code and commands.
- **Compiler and static analysis:** no warnings with `-Wall -Wextra -Wshadow` (with and without NetCDF) and none from `cppcheck` in the new files.
- **Sanitizers:** complete builds with AddressSanitizer and UndefinedBehaviorSanitizer, run on random set-ups covering every option, an ensemble and a restart, and on one random set-up of every grid type: no memory error and no undefined behaviour. One use of freed memory at program exit was found this way early on and corrected. The only reported leaks are allocated inside the MODFLOW library.

9.10 Findings and corrections

Every problem found during development, verification and the release audit was corrected (Table 9.11); corrections from the release audit were re-tested with the checks of Section 9.8.

Table 9.11: Findings and corrections.

Finding	Correction
Raven itself (present before this work)	
<code>:rvg_Filename</code> : the path was taken relative to the working folder, and a line without a file name was not caught	path relative to the <code>.rvi</code> ; a missing name stops the run
Constituent name read after the token buffer was reused (<code>:ObservationData</code> , <code>:ObservationWeights</code>)	read before parsing
Ensemble members inherited balance, routing memory and reservoir state	first-run state restored before each member's <code>.rvc</code>
Reservoirs with absolute stages started at 0 m (209 mm balance error)	start at the crest; also on ensemble reset
A reservoir that dried out booked full evaporation and seepage (64–135 mm)	books only the water available
Reservoir stage and flows written to <code>solution.rvc</code> with 5 decimals	full precision
BMI <code>SetValueAtIndices</code> read the wrong element (Raven 4.1.1)	corrected; Raven 4.15 corrects it the same way
The old MODFLOW-USG coupling was compiled in but could not run in a standard build	moved unchanged to <code>legacy/mfusg/</code> ; its <code>.rvg</code> commands give a clear error
Water accounting	
Recharge on fixed-head cells lost (0.09%)	redistributed or reported
River losses could exceed the river's flow; the limit used the reservoir release	conductance limited by the channel flow
Unsupplied river loss was created water	carried to the next step and across restarts
Wetland and lake stores could be over-drawn	river-type boundary; shares fixed before the solve
Water-table ET spread over part of the cell and counted ET-free HRUs	rate over the whole cell; ET-enabled HRUs only
Reservoir seepage lost on restart and leaked between ensemble members	saved; nothing in transit at a fresh start

Table 9.11: Findings and corrections (continued).

Finding	Correction
Rivers of disabled subbasins exchanged water not applied to Raven	no river cells there
Heads of dry cells (down to -5×10^8 m) used as water levels	physical water table; lake head never below its bed
Numerical solution	
Head tolerance 0.01 m allowed 0.2–0.8% residuals and restart differences	default 0.00001 m; inner solver ten times tighter
The first two-tier rule accepted damped, unconverged steps (errors up to 99%)	MODFLOW's own convergence test at both tiers
Safeguard for failed steps missed some inflows and store discharge	all inflows and discharges included
Input checking	
Values that create water or break MODFLOW were accepted	range checks at start-up
Malformed groundwater commands were ignored	stop with a message
An explicit $K_VERT \leq 0$ was silently replaced by the default	rejected
Duplicate IDs, double store exchanges and boundaries without HEAD accepted	rejected
Disabled HRUs coupled; seepage sent to soil in soil-less HRUs	excluded; sent to SURFACE_WATER
Water sent to GROUNDWATER in uncoupled HRUs lost silently; wells in fixed-head cells	warnings
Grid options	
A boundary line drawn along the edge of the active area lost its boundary where the edge cells were inactive or where it lay exactly on a cell edge (also in the original regular grid)	attached to the nearest active cell
Voronoi cells of cocircular seeds had near-duplicate corners, which made clipping drop whole cells (holes in HRU meshes)	corners cleaned; degenerate clip edges ignored
Rivers and boundary lines were sampled at a twentieth of the smallest cell, very slow on refined grids	exact clipping of each segment by each cell
DISU connections unsorted (MODFLOW pairs them by position); vertices missing (needed for XT3D); a partial last line in 1-D arrays	sorted; vertices written; lines ended
Polygon refinement (:GridRefinement POLYGONS) was silently ignored by the HRU mesh, although documented for it	implemented (cells of the target size in and around the polygon); LAKES added
A steady-state start that did not converge was kept as the initial heads; on layers connected by elevation with strong pumping every later day then failed to converge (found by the randomized tests)	the run falls back to the initial heads, with a warning; every day then converges
XT3D, first on by default for quadtree and imported grids, slowed Newton badly where cells dry and rewet (41 of 61 days failed in one random set-up; on the Liard quadtree 32 instead of 7 iterations per day, even in its right-hand-side form)	off by default (optional); no loss of accuracy in the Thiem test

Table 9.11: Findings and corrections (continued).

Finding	Correction
Existing-model mode	
A stream-cell subbasin that did not exist was not caught (Raven’s “not found” code differs from the one checked) and crashed the run	any negative index stops the run with a message
Auxiliary values (subbasin IDs) of MODFLOW 6.5+ are kept in its input context, so the package’s array read zero	read from the input context when present
Recharge rates divided by Raven’s cell areas, but MODFLOW multiplies by its own; the two measures differ by 0.5% at 58°N	rates use MODFLOW’s cell areas
MODFLOW’s recharge flow was not converted to m ³ /d in the budget (found by the feet/seconds test)	converted
With :OverlapWeights the connectivity diagnostic had no neighbours and crashed	neighbours from MODFLOW’s own connections
With :SeepageToRaven OFF the seepage package’s arrays were still requested (found by the randomized tests)	requested only when the package exists
The hard-step tolerance was relaxed to ten times Raven’s target in metres, but applied in the model’s units: a model in feet got a threefold relaxation only (found by the randomized tests)	relaxed tenfold from the model’s own tolerance, in its units
Build files	
The Visual Studio filters file had been malformed since the first coupling files were added (entries without closing tags), and the two newest files were missing from the project	filters regenerated from 4.1.1, listing the same files as the project; new files added (4.1.1’s own quirks — FrozenGround.h listed but absent, RavenMain.h not listed — are left as they were)
Outputs, restarts and program safety	
Output dates at the start of steps; duplicate sub-daily labels	end-of-step dates with time of day
Hotstart data leaked between ensemble members	cleared before each member’s .rvc
Recharge delay parsed but not used; linkage cache written with 15 digits	implemented with restart; 17 digits
Use of freed memory at exit; unchecked folder change; uninitialized pointers	corrected
Release audit (Section 9.8)	
A reservoir drying under a minimum or override flow booked no evaporation (114 million m ³ unbalanced in a 20-year test; 158 with Raven 4.1.1)	books the evaporation the outflow formula removed, and no seepage
Reservoir constraint read before the first step without a value (Raven 4.1.1; found by the sanitizer)	initialised; outputs unchanged
A reservoir whose :HRUID is an HRU of another subbasin crashed the run (Raven 4.1.1; found by the randomized tests with MODFLOW)	stops with a message
Existing models with a water mover (MVR): the water that the model’s packages pass to the mover was left out of the balance check (residual 0.1% on the Sagehen model, up to 30% with its UZF kept; found with the USGS examples)	counted with the package’s other flows

Table 9.11: Findings and corrections (continued).

Finding	Correction
Taking over a UZF package named in the model's mover stopped MODFLOW, and with it Raven, without a message	movers from or to taken-over packages removed from the working copy (reported)
A stream package that passes water to the mover would have been counted twice	refused with a message
Steady-state models with one stress period (often one second long) could not be coupled; without a storage package MODFLOW's error was reported as storage	the period is stretched over Raven's run; the residual is reported as MODFLOW's error
Restarts refused for models with a mover or advanced packages	MVR renumbered like list packages; SFR, LAK, MAW, UZF like arrays
With MODFLOW 6.6, the Newton option's UNDER_RELAXATION keyword made steps with a drying cell stall (the default Liard grid failed on 295 of 344 days)	NEWTON written without it (identical results with 6.8.1); the library's version is reported, and existing models using the keyword with a version before 6.8 get a warning
GW_HEAD observations bypassed Raven's observation matching (irregular series, :EvaluationTime)	matched like every other observation
Ensemble snapshot lost each subbasin's last lateral inflow	kept
Rotated grids without refinement: cell centres and grid origin in the unrotated frame (a boundary polygon then selected no cell)	rotated frame
HRU mesh: a well outside the mesh area crashed the run; the closing edge of each HRU outline got no seeds	skipped; seeded
DISU: vertical connections bridged aquicludes; face angles had the grid rotation added twice (30° off on a rotated grid)	no vertical flow through an aquiclude; angles in the vertex frame
Imported cells: repeated vertices, and faces between re-projected cells with T-junctions were lost	vertices de-duplicated; faces found with a tolerance relative to their length
Drain smoothing could reach below the top cell (2794 of 3252 drains with a thin top layer)	limited to the top layer
Reservoir exchange: the lake bed was set at the model top even when the lake stood below it (false gains); gains a drying cell could not supply created water; a lake below the aquifer bottom was accepted	bed at the lake level; unsupplied gains taken back from the reach (new budget column); stops with a message
Quadtree refinement slow on large grids; linkage cache ignored monitoring wells; GeoJSON reader could read past the end of a value	7–68 times faster (identical cells); wells in the cache key; bounds checked
Existing models: ensembles and calibration with :MF6CRS or :MF6GridFile failed at the second initialisation	corrected
mfsim.nam written without CONTINUE and not under the name MODFLOW opens; simulations with several solutions accepted	corrected; refused
Recharge could fall on the model's own specified-head cells	kept off them (read from IBOUND every step)
Restarts: time series in the first period, adaptive time stepping, TVK/TVS and multi-model simulations were accepted although they cannot be shifted	refused with a message

Table 9.11: Findings and corrections (continued).

Finding	Correction
Restarts: START_DATE_TIME and the time of day not set; calendars other than Gregorian accepted; monitoring wells below inactive cells	set; Gregorian required; corrected
:MF6GridFile check compared areas only	common scale allowed; cell layout checked against MODFLOW's connections (order, mirror image)
Settings of generated models, NetCDF forcing with overlap weights and copies inside the working folder were accepted with an existing model; file writes unchecked	refused; checked
Windows build failed (GetCurrentTime clashed with a windows.h macro); path checks not normalised	renamed; normalised

CHAPTER 10

Developer guide and limitations

This chapter is for those who maintain or extend the code.

In plain words

This chapter is for programmers: which file does what, how one time step passes through the code, how to add a new exchange or grid type, and how to test changes. Users of the coupling can skip it.

10.1 Source files

Table 10.1: Source files.

File	Content
GroundwaterModel.h/.cpp	class CGroundwaterModel: parse-time setters, model builder, per-step exchange, outputs, hotstart
GWExternal.cpp	coupling to an existing MODFLOW 6 model: reads the simulation through the MF6 API, copies it, rewrites the time discretization (and renumbers periods on restarts), links HRUs (cell outlines with :MF6CRS, polygon file, or overlap weights), writes RCH_RAVEN/DRN_RAVEN, maps the stream package to reaches, collects the other packages' flows
GWUtil.h	small helpers shared by the coupling's files (absolute paths, MODFLOW array writer, number formatting)
legacy/mfusg/	L. Scantlebury's MODFLOW-USG coupling as in Raven 4.15, preserved unchanged outside the build
GWGrid.h/.cpp	CGWGrid: cells of one layer as polygons with areas, centres, neighbours (face length, centre-to-face distances, face direction); generators for regular (rotation, variable spacing), quadtree, Voronoi and imported grids; point location, candidate search, exact clipping of polygons and segments; cache serialization
GWGeometry.h/.cpp	CGWProjection (equal-area projection), CGWCRS (projections of existing models: Transverse Mercator, Albers, Lambert conformal conic), CGWRaster (ESRI ASCII, bilinear sampling), minimal JSON/GeoJSON reader, polygon clipping and areas
MF6Engine.h/.cpp	CMF6Engine: loads libmf6 at run time (dlopen/LoadLibrary); initialize, time step, Newton loop, memory access; switches to the MODFLOW folder around every call
ParseGWFile.cpp	.rvg parser

10.2 Order of calls

1. CModel constructor creates an empty CGroundwaterModel. The .rvp parser fills aquifer classes and profiles; the .rvh parser stores AQUIFER_PROFILE on each HRU; the .rvt parser stores well-rate and boundary-head series; the .rvc parser stores initial or hotstart heads; ParseGWFile reads the .rvg.
2. CModel::CalculateInitialWaterStorage (called once, and again for each ensemble member) calls CGroundwaterModel::Initialize: CheckProcesses, BuildGeometry (grid generation by CGWGrid and HRU–cell overlaps, with cache), BuildVertical, BuildLinks, BuildRivers, BuildStresses (wells, monitoring wells, GHB/CHD, surface stores, reservoirs), WriteMF6Files, ConnectEngine, OpenOutputs, WriteSummary, RunSteadyState.
3. Every step, MassEnergyBalance (Solvers.cpp) calls Exchange after the lateral processes, then ApplyRiverExchange before routing, and TakeRiverLossFromInflow inside the routing loop. IncrementCumulInput adds GetStepReturnVolume; UpdateDiagnostics asks GetObservationWellHead.
4. WriteMajorOutput calls WriteHotstart; the destructor closes outputs and finalizes MODFLOW.

10.3 MODFLOW 6 memory used

All are read or written through `get_value_ptr` after `prepare_time_step` (pointers are refreshed every step). For MODFLOW versions before 6.5 the values are written to the columns of BOUND

Table 10.2: MODFLOW 6 memory used.

Package (name)	Arrays	Use
GWF	X	heads
STO	STRGSS, STRGSY	storage flows
SLN_1	MXITER	iteration limit
RCH (RCH_RAVEN)	RECHARGE, HCOF, RHS	recharge
DRN (DRN_RAVEN)	HCOF, RHS (aux DDRN in file)	seepage
RIV (RIV_RAVEN)	STAGE, COND, RBOT, HCOF, RHS	rivers
EVT (EVT_RAVEN)	RATE, HCOF, RHS	water-table ET
WEL (WEL_RAVEN)	Q, HCOF, RHS	wells
GHB (GHB_RAVEN)	BHEAD, COND, HCOF, RHS	head boundaries
CHD (CHD_RAVEN)	HEAD, SIMVALS	fixed heads
RIV (SWX_RAVEN)	STAGE, COND, RBOT, HCOF, RHS	wetlands and lakes
WEL (RES_RAVEN)	Q, HCOF, RHS	reservoir seepage

instead.

10.4 Changes to existing Raven files

The source files are in `src/`; the changes are against Raven 4.15.

Table 10.3: Changes to existing Raven files.

File	Change
Solvers.cpp	groundwater hook after lateral processes; river exchange before routing; losses from reach inflow
Model.cpp/.h	always creates the groundwater object; returned water counted as input; GW_HEAD diagnostics; stored first routing state
ModelInitialize.cpp	groundwater initialized after all inputs; cumulative inputs and outputs reset per ensemble member (Raven 4.15's RebootTimeVariables resets the balance arrays); routing memory and reservoir states of the first run snapshotted and restored before each member's .rvc is read (RestoreInitialDynamicState); per ensemble member
StandardOutput.cpp	hotstart blocks in solution.rvc
ParseInput.cpp	:GroundwaterModel; :rvg_Filename relative to the .rvi; obsolete USG commands give clear errors
ParsePropertyFile.cpp	:AquiferClasses, :AquiferProfiles, :AquiferProfile Parameters
ParseHRUFile.cpp, HydroUnits.h	AQUIFER_PROFILE stored on each HRU
ParseTimeSeriesFile.cpp	:WellRate, :BoundaryHead; constituent name captured before parsing
ParseInitialConditionFile.cpp	:InitialGWHeads, :GWHeads, :GWRechargeStore, :GWReservoirSeepage
SubBasin.cpp/.h	channel depth at a given flow
Reservoir.h/.cpp	accessors for the seepage constant and local head; reservoirs with absolute stages start at their crest when no initial condition is given; dynamic-state get/set for ensemble members; a drying reservoir books only the water available; constraint initialised
RavenInclude.h, CommonFunctions.cpp	references to the MODFLOW-USG coupling removed; RC_UNSET (reservoir constraint before the first step)
CMakeLists.txt, src/Makefile, src/Raven.vcxproj(.filters)	new files; link -ldl (CMake: CMAKE_DL_LIBS)
Drain.cpp, GWRecharge.cpp, GWRiverConnection.*, GWSWProcessABC.cpp, GWSWProcesses.h, MFUSGpp.h, RiverReach.*	moved unchanged to legacy/mfusg/ (with the old GroundwaterModel.* and ParseGWFile.cpp); not part of the build

10.5 Building

The source files are in `src/`. Linux/macOS: CMake (`mkdir build; cd build; cmake ..; make; links -ldl` and NetCDF when found) or `make` in `src/` (`links -ldl`; for NetCDF head output add `-Dnetcdf` to `CXXFLAGS` and `-lnetcdf` to `LDLIBS`, as the Makefile’s commented lines show). Windows: Visual Studio 2022 with `src/Raven.sln`. MODFLOW is not needed to build; it is loaded only when a model uses `:GroundwaterModel MODFLOW6`.

10.6 Getting MODFLOW 6

The MODFLOW 6 shared library (`libmf6.so`, `libmf6.dll` or `libmf6.dylib`, about 15–55 MB) is not part of the repository. One command in the source folder downloads it:

```
python tools/get_mf6.py # MODFLOW 6.8.1, the tested version, for this computer
python tools/get_mf6.py --latest # the newest release instead (or --version 6.x.y)
python tools/get_mf6.py --zip mf6.8.1_win64.zip # from a release zip downloaded by hand (offline)
```

The script (standard Python only) picks the release zip for the computer from <https://github.com/MODFLOW-ORG/modflow6/releases> (`mf6.8.1_linux.zip`, `mf6.8.1_win64.zip`, `mf6.8.1_macarm.zip`; `_mac.zip` for Intel Macs where one is released), keeps only the library in `lib/mf6/`, and records its version, source and SHA-256 in `lib/mf6/libmf6_version.txt`. It does nothing when the library is already there (`-force` replaces it). The default is the tested version rather than the newest release, so that a new MODFLOW release cannot change a model’s results unnoticed; with MODFLOW 6.6, for example, the Newton option behaved differently (Section 9.8).

A CMake build does the same while configuring (option `RAVEN_GET_MF6`, on by default; version `RAVEN_MF6_VERSION`, default 6.8.1) and copies the library next to the executable. Without internet access the build continues with a warning.

Raven looks for the library in this order: `:MF6Library` in the `.rvg` file; the environment variable `RAVEN_MF6_LIB`; next to the Raven executable; `lib/mf6/` beside it or one or two folders up (so `src/Raven.exe`, `build/Raven` and `build/Release/Raven.exe` of a source checkout all find the checkout’s `lib/mf6/`); finally the platform’s name on the system library path. `GWModelSummary.txt` names the library used and its version; when none loads, the message lists the places searched.

10.7 Adding a new exchange

1. Choose the MODFLOW package (a new package name, e.g. `XYZ_RAVEN`) and decide which Raven quantity drives it.
2. Parse its inputs in `ParseGWFile.cpp` (or the `.rvp/.rvt` parsers) and store them in `CGroundwaterModel`.
3. Select its cells in `BuildStresses`; write the package file and its line in `gwf.nam` in `WriteMF6Files`.
4. Fetch its arrays in `RefreshPointers` (named arrays first, `BOUND` as fallback).
5. Set its values in `Exchange` (or `SetBoundaryArrays`) after `prepare_time_step`.
6. After the solve, compute its flow as $HCOF \cdot h - RHS$; apply the matching volume on the Raven side and add it to `GetStepReturnVolume` if it changes Raven’s storage.
7. Add the flow to the balance error ε , the gross-flow scale, the `GWBudget.csv` header and line, and the non-convergence cap if it is an inflow.
8. Add a test, extend `tools_audit.py` and the list-file cross-check.

10.8 Adding a grid type

A grid type only has to produce cells: add a generator to `CGWGrid` that fills the cell polygons (counterclockwise, in the grid’s local frame) and, if it knows them exactly, the neighbours; `Finish` computes areas, centres, bounding boxes, spatial bins and welded vertices, and finds neighbours from shared edges otherwise. Choose the MODFLOW format in `Initialize` (`DISU` works for any cells) and the XT3D default in `UseXT3D`. Everything else — overlaps, rivers, boundaries, wells, outputs, restarts — works through the cell polygons and needs no change. Add the new type to the grid unit test (`tests/gridtest.cpp`), the Thiem test (`tests/thiem/`) and the randomized tests.

10.9 Testing procedure

1. Uncoupled regression: run a model without groundwater with the new build and compare `Hydrographs.csv` and `WatershedStorage.csv` byte for byte with the previous version.
2. Build with `-Wall -Wextra -Wshadow` and run `cppcheck` on the new files.
3. Run the feature tests and `tools_audit.py` on each output folder.
4. Compare with MODFLOW’s list-file budget.
5. Build with `-fsanitize=address,undefined` and run the feature tests.

10.10 Limitations and future work

Current limitations.

- Refinement of a regular grid spans whole rows and columns; use a quadtree or HRU mesh for local refinement.
- XT3D (optional) makes each iteration more expensive and can slow the Newton method badly where cells dry and rewet; it is off by default.
- Imported grid files must hold one single-ring polygon per cell in longitude/latitude.
- Maps must be ESRI ASCII grids in longitude/latitude; the same vertical datum must be used for DEM, bedrock, reservoir stages and wells.
- Layer ℓ connects sideways only to layer ℓ of neighbouring columns.
- River stage uses Raven’s channel depth at the start of the step; reservoir seepage reaches the aquifer one step later (both exactly accounted).
- A layer that starts completely dry can cause isolated non-converged steps; start from a steady state or realistic heads.
- MODFLOW stops the program on invalid input (Fortran stop); the reason is in `mf6/mfsim.lst`.
- Raven’s channel-storage bookkeeping (`:ChannelStorage` in `solution.rvc`) can be negative in some subbasins; this already happens in Raven 4.1.1 without groundwater (17 subbasins of the Liard model) and is left unchanged here, as it belongs to Raven’s routing, not to the coupling.

Planned extensions. Groundwater transport (MODFLOW 6 GWT) linked to Raven constituents; particle tracking (PRT) for well capture zones and time-of-travel zones; wells driven by Raven water demands (irrigation); frozen-ground control of recharge from Raven’s soil temperature; importing an existing hand-built MODFLOW 6 model instead of generating one.

APPENDIX A

Reference tables

This appendix collects, in the style of the Raven manual, every parameter, command, option, algorithm and output of the groundwater coupling in one place. *Default* is the value used when the item is not given; *checked* is the range Raven enforces at start-up (values outside it stop the run with a message); *typical* ranges are guidance from the literature [4, 3, 6, 1], not limits.

In plain words

The tables here repeat, in one place, every parameter, command, option and output with its unit, default and allowed range — the same information as the chapters, arranged for looking up rather than reading.

A.1 Parameter classes

As in Raven, every parameter belongs to a class, and the class decides where it is set and which part of the model it applies to.

Table A.1: Parameter classes.

Class	File	Applies to	Examples
Aquifer class	.rvp	every layer made of that material	K_HORIZ, SPEC_YIELD
Aquifer profile	.rvp	every column whose dominant HRU has the profile; its layering	SEEPAGE_LEAKANCE, RIVERBED_K, layers
HRU	.rvh	one HRU	AQUIFER_PROFILE, AREA, ELEVATION
Subbasin, channel	.rvh, .rvp	a subbasin's reach	channel profile, Q_REFERENCE
Reservoir	.rvh	one reservoir	:SeepageParameters, :AbsoluteCrestHeight
Groundwater option	.rvg	the whole groundwater model	:GridCellSize, :SolverHeadTolerance
Stress or observation	.rvg, .rvt	a well, boundary or monitoring well	:Wells, :WellRate, :BoundaryHead
Initial condition	.rvc	the start of a run	:InitialGWHeads, hotstart blocks
Raven state variable	(computed)	an HRU store or flux	GROUNDWATER, AET, DEPRESSION

Table A.2: Aquifer class parameters (:AquiferClasses, .rvp) (continued)

Parameter	Meaning	Units	Default	Checked	Typical range
-----------	---------	-------	---------	---------	---------------

A.2 Aquifer class parameters (:AquiferClasses, .rvp)

Table A.2: Aquifer class parameters (:AquiferClasses, .rvp).

Parameter	Meaning	Units	Default	Checked	Typical range
K_HORIZ	horizontal hydraulic conductivity	m/d	required	> 0	10^{-7} (clay) – 10^3 (gravel)
K_VERT	vertical hydraulic conductivity	m/d	$K_h/10$	> 0	$K_h/100 - K_h$
SPEC_STORAGE	specific storage S_s	1/m	10^{-5}	≥ 0	10^{-6} (rock) – 10^{-3} (soft clay)
SPEC_YIELD	specific yield S_y	–	0.1	$[0, 1]$	0.01 (clay) – 0.35 (gravel)
POROSITY	total porosity (reserved for transport)	–	0.3	$[0, 1]$	0.01 (rock) – 0.6 (clay)

A.3 Typical values by material

Table A.3: Typical values by material.

Material	K [m/d]	S_y [–]	S_s [1/m]	Porosity [–]
Gravel	10^1 – 10^3	0.13–0.35	10^{-6} – 10^{-5}	0.25–0.40
Sand, clean	10^0 – 10^2	0.10–0.35	10^{-5} – 10^{-4}	0.25–0.50
Sand, silty	10^{-2} – 10^0	0.08–0.25	10^{-5} – 10^{-4}	0.30–0.45
Silt, loess	10^{-3} – 10^0	0.03–0.20	10^{-4}	0.35–0.50
Glacial till	10^{-6} – 10^{-1}	0.02–0.15	10^{-4} – 10^{-3}	0.10–0.30
Clay	10^{-7} – 10^{-3}	0.00–0.05	10^{-4} – 10^{-2}	0.40–0.70
Sandstone	10^{-4} – 10^0	0.02–0.30	10^{-6} – 10^{-5}	0.05–0.30
Limestone, karst	10^{-1} – 10^3	0.01–0.15	10^{-6}	0.05–0.50
Fractured crystalline rock	10^{-3} – 10^1	0.001–0.05	10^{-6}	0.00–0.10

A.4 Aquifer profile parameters (:AquiferProfileParameters, .rvp)

Table A.4: Aquifer profile parameters (:AquiferProfileParameters, .rvp).

Parameter	Meaning	Units	Default	Checked	Typical range
INITIAL_HEAD_DEPTH	initial depth to water below land	m	5	any	site data
DEFAULT_BEDROCK_DEPTH	bedrock depth where no bedrock map is given	m	50	> 0	5–300

Table A.4: Aquifer profile parameters (:AquiferProfileParameters, .rvp) (continued)

Parameter	Meaning	Units	Default	Checked	Typical range
SOIL_ZONE_DEPTH	model top below land surface	m	0	≥ 0	0–3
SEEPAGE_LEAKANCE	seepage conductance per m ² of cell	1/d	1	≥ 0	0.1–10
SEEPAGE_SMOOTHING_DEPTH	depth below the top where seepage starts	m	0.5	≥ 0	0.1–1
RIVERBED_K	riverbed hydraulic conductivity	m/d	0.5	≥ 0	10 ⁻³ –10
RIVERBED_THICKNESS	riverbed thickness	m	1	> 0	0.3–3
EXTINCTION_DEPTH	water-table ET extinction depth (0 = off)	m	0	≥ 0	0.5–5 (root depth)
RECHARGE_DELAY	recharge delay time constant (0 = off)	d	0	≥ 0	0–365

A.5 Layer types (:AquiferProfiles, .rvp)

Table A.5: Layer types (:AquiferProfiles, .rvp).

Keyword	MODFLOW 6 setting	Behaviour
AQUIFER, UNCONFINED_AQUIFER	ICELLTYPE 1, ICONVERT 1	convertible: S_y when unconfined, S_s when full; transmissivity from saturated thickness
CONFINED_AQUIFER	ICELLTYPE 0, ICONVERT 0	always full: constant transmissivity, storage by S_s
AQUITARD, CONFINING_LAYER	as AQUIFER	leaky layer with the (low) K of its class
AQUICLUDGE thickness TO_BEDROCK	IDOMAIN 0 layer bottom = bedrock	no flow; not allowed as top layer layers below it become pass-through cells
(automatic) pass-through	IDOMAIN -1	layer thinner than 0.5 m or missing in the profile: connects the cells above and below

Table A.6: Commands of the groundwater coupling, by input file (continued).

File	Command	Meaning (default)
------	---------	-------------------

A.6 Commands

Table A.6: Commands of the groundwater coupling, by input file.

File	Command	Meaning (default)
.rvi	:GroundwaterModel NONE MODFLOW6	switch the coupling on (NONE)
.rvi	:rvg_Filename file	groundwater file, relative to the .rvi (<model>.rvg)
.rvh	AQUIFER_PROFILE column of :HRUs	profile name or [NONE]
.rvh	:SeepageParameters k h in :Reservoir	reservoir seepage constant; h is replaced by the aquifer head under :ReservoirExchange
.rvp	:AquiferClasses ... :EndAquiferClasses	material table (Section A.2)
.rvp	:AquiferProfiles ... :EndAquiferProfiles	name, number of layers, {class, thickness or TO_BEDROCK, type}
.rvp	:AquiferProfileParameters ...	per-profile parameters (Section A.4)
.rvg	:HRUGeometry file {field}	HRU polygons, GeoJSON FeatureCollection, lon/lat (HRU_ID); required
.rvg	:RiverGeometry file {field}	river lines; subbasin field optional
.rvg	:LandSurface file	DEM, ESRI ASCII grid in lon/lat (HRU elevations)
.rvg	:BedrockSurface file	bedrock elevation grid (DEFAULT_BEDROCK_DEPTH)
.rvg	:GridCellSize m	cell size, > 0 (1000)
.rvg	:MinCellCoverage f	coverage for an active column, (0, 1] (0.25)
.rvg	:GridType REGULAR QUADTREE HRU_MESH FILE	kind of grid (REGULAR)
.rvg	:MF6Simulation file, :MF6Model name	existing model; which GWF model
.rvg	:MF6StartDate yyyy-mm-dd	date of the model's time zero (required)
.rvg	:MF6CRS EPSG:code TMERC ALBERS LCC ...	projection of the model's coordinates
.rvg	:MF6GridFile file {ID}, :OverlapWeights	cell polygons; or HRU–cell weights
.rvg	:RavenTakesOver p ... , :KeepPackages p ...	declared recharge/ET/UZF packages
.rvg	:StreamPackage p {AUX a DOMINANT_HRU}	RIV/DRN/GHB package to Raven's reaches
.rvg	:SeepageToRaven ON OFF	DRN_RAVEN at the model top (ON)
.rvg	:GridRotation θ	degrees counterclockwise, $ \theta \leq 360$; regular and quadtree grids (0)
.rvg	:GridRefinement RIVERS WELLS LAKES s or LINES POLYGONS file s	target cell size $0 < s < :GridCellSize$; not for FILE; LAKES: every reservoir's lake HRU
.rvg	:GridFile file {ID}	cell polygons for :GridType FILE (CELL_ID)
.rvg	:QuadtreeMaxLevel n	$0 \leq n \leq 12$; QUADTREE only (from refinement sizes)
.rvg	:FlowCorrection AUTO XT3D XT3D_RHS NONE	XT3D (AUTO: off)

Table A.6: Commands of the groundwater coupling, by input file (continued).

File	Command	Meaning (default)
.rvg	:LayerConnection INDEX ELEVATION	layers connected by number or by elevation (INDEX)
.rvg	:MF6Library path	MODFLOW 6 library (else RAVEN_MF6_LIB, next to the Raven executable, then lib/mf6/, then the system path)
.rvg	:SeepageReturnTo SV	store receiving seepage (lowest soil layer; SURFACE_WATER for soil-less HRUs)
.rvg	:RiverBedDepth m REFERENCE_FLOW	riverbed depth below the DEM (REFERENCE_FLOW)
.rvg	:GWInitialization STEADY_STATE r	steady-state start, recharge $r \geq 0$ mm/d (off); if it does not converge, the initial heads are used
.rvg	:SurfaceWaterExchange SV group LEAKANCE L	store–aquifer exchange, $L > 0$ 1/d
.rvg	:ReservoirExchange	reservoir–aquifer exchange (off)
.rvg	:Wells... :EndWells	ID, NAME, LATITUDE, LONGITUDE, SCREEN_TOP, SCREEN_BOT
.rvg	:ObservationWells...	ID, NAME, LATITUDE, LONGITUDE, SCREEN_ELEV
.rvg	:GeneralHeadBoundary n file HEAD h CONDUCTANCE c {LAYERS ALL TOP}	head-dependent boundary; $c > 0$ m ² /d per cell (ALL)
.rvg	:SpecifiedHeadBoundary n file HEAD h {LAYERS ALL TOP}	fixed-head boundary (ALL)
.rvg	:SolverHeadTolerance m	Newton target tolerance, > 0 (0.00001)
.rvg	:SolverMaxOuterIterations n	Newton iteration limit, ≥ 1 (500)
.rvg	:HeadSaveFrequency n	steps between saved head fields, ≥ 1 (30)
.rvg	:WriteNetCDFHeads	write GWHeads.nc (off; needs a NetCDF build)
.rvg	:LinkageCache file	linkage cache (<output>/mf6/linkage.cache)
.rvt	:WellRate id units... :EndWellRate	well rate series, m ³ /d, negative = pumping
.rvt	:BoundaryHead name units...	time-varying boundary head
.rvt	:ObservationData GW_HEAD id units...	observed heads for diagnostics
.rvc	:InitialGWHeads FROM_PROFILE DEPTH_BELOW_SURFACE d ELEVATION z	initial heads (FROM_PROFILE)
.rvc	:GWHeads, :GWRechargeStore, :GWReservoirSeepage, :GWRiverLossCarry	hotstart blocks written by Raven (Section A.11)

A.7 Options

Table A.7: Options of the groundwater commands.

Command	Option	Effect
:GroundwaterModel	NONE	Raven as before (identical results to Raven 4.15)
	MODFLOW6	coupled HRUs exchange water with MODFLOW 6
:InitialGWHeads	FROM_PROFILE	$z_{\text{land}} - \text{INITIAL_HEAD_DEPTH}$ of the column's profile

Table A.7: Options of the groundwater commands (continued)

Command	Option	Effect
:GWInitialization	DEPTH_BELOW_SURFACE d	$z_{\text{land}} - d$ everywhere
	ELEVATION z	z everywhere (at least 0.1 m above the lowest active bottom)
	STEADY_STATE r	steady state with recharge r over covered areas before day 1; skipped on hotstart
:RiverBedDepth	value	bed = lowest DEM along the river – value
	REFERENCE_FLOW	bed = lowest DEM – Raven channel depth at the reference flow (max 20 m)
LAYERS (boundaries)	ALL	every active layer whose bottom is below the head
	TOP	the first such layer only
:SeepageReturnTo	any storage state variable	e.g. SOIL[2], SURFACE_WATER, DEPRESSION

A.8 Exchange algorithms

Table A.8: Exchange algorithms and their safeguards.

Exchange	MODFLOW package (name)	Formulation	Safeguard
Recharge	RCH (RCH_RAVEN)	$V_k = r_k A_k / 1000$; optional linear delay; spread by overlap	fixed-head cells skipped; exact per HRU
Seepage	DRN (DRN_RAVEN)	$Cf(h)(h - z_{\text{top}} + d_s)$, $C = L_s \Delta^2$	returned by overlap; soil-less HRUs to SURFACE_WATER
Rivers	RIV (RIV_RAVEN)	$C(s - h)$, or $C(s - z_{\text{bed}})$ below the bed; $C = K_b WL / M_b$	C limited by channel flow; unsupplied loss carried
Water-table ET	EVT (EVT_RAVEN)	unmet PET over the cell, linear to d_x	only ET-enabled HRUs; never above unmet demand
Wetlands, lakes	RIV (SWX_RAVEN)	bed = top, stage = top + store depth	C limited by stored water; one exchange per store
Reservoirs	WEL (RES_RAVEN)	Raven seepage law, $h_{\text{loc}} = \sum w_i \max(z_i^{\text{wt}}, z_{\text{top},i})$; one-step lag	in-transit volume reported and saved
Wells	WEL (WEL_RAVEN)	rate split by transmissivity of screened layers	AUTO_FLOW_REDUCE
Head boundary	GHB (GHB_RAVEN)	$C_g(h_g - h)$	heads kept above cell bottoms
Fixed head	CHD (CHD_RAVEN)	head held at h_c	heads kept above cell bottoms

A.9 Solver settings (sim.ims)

Table A.9: Solver settings written to sim.ims.

Setting	Value	Note
Formulation	Newton	in gwf.nam, without its UNDER_RELAXATION keyword (Section 3.9)
OUTER_DVCLOSE	0.00001 m	target; raised to 10× for the rest of a step after 100 iterations (or half the limit)
OUTER_MAXIMUM	500	:SolverMaxOuterIterations
UNDER_RELAXATION	DBD	theta 0.7, kappa 0.07, gamma 0.1
BACKTRACKING_NUMBER	20	tolerance 1.05, reduction 0.1, residual limit 0.002
INNER_MAXIMUM	500	linear iterations
INNER_DVCLOSE, INNER_RCLOSE	0.1 × outer, 1 m ³ /d	
LINEAR_ACCELERATION	BICGSTAB	ILU, 5 levels, drop tolerance 10 ⁻⁴

A.10 Output: GWBudget.csv columns

Table A.10: Columns of GWBudget.csv.

Column	Meaning
time [d], date	end of the step (with time of day for sub-daily steps)
Raven recharge [m3/d]	recharge leaving Raven, before any delay
MF6 recharge [m3/d]	recharge entering MODFLOW
seepage to Raven [m3/d]	drain discharge at the model top
storage release [m3/d]	water released from aquifer storage (negative: stored)
MF6 balance error [m3/d], [%]	MODFLOW imbalance; % of gross flow
returned to Raven [m3]	seepage added to Raven stores
outer iterations, converged	Newton iterations; 1 target, 2 relaxed, 0 not converged
recharge delay storage [m3]	water in delay reservoirs
recharge onto CHD cells [m3/d]	recharge of HRUs lying wholly on fixed-head cells
river leakage to aquifer [m3/d], aquifer discharge to rivers [m3/d]	river exchange by direction
river exchange applied to Raven [m3]	net volume applied to Raven reaches
river loss carried to next step [m3]	unsupplied loss taken next step (normally 0)
water-table ET [m3/d]	evaporation from the water table
wells specified [m3/d], wells actual [m3/d]	requested and delivered well rates
GHB [m3/d], CHD [m3/d]	boundary flows (positive into the aquifer)
surface-store leakage to aquifer [m3/d], aquifer discharge to surface stores [m3/d]	wetland/lake exchange
surface-store exchange applied to Raven [m3]	net volume applied to Raven stores
reservoir seepage sent [m3/d], delivered [m3/d], awaiting delivery [m3]	reservoir exchange

Table A.10: Columns of GWBudget.csv (continued).

Column	Meaning
reservoir gain not supplied by MODFLOW [m3]	lake gain a drying cell could not supply; taken back from the reach (normally 0)
other model packages [m3/d]	existing models only: net flow of the model's own packages (wells, boundaries, ...), positive into the aquifer

Other output files: GWHRUState.csv (water table and depth per coupled HRU), GWHeads.csv (monitoring wells; -1.2345 when dry), GWConnectivity.csv (wet areas, wet fraction per profile), GWModelSummary.txt, GWHeads.nc (optional), mf6/ (MODFLOW model, list files, binary heads).

A.11 Hotstart blocks (solution.rvc)

Table A.11: Hotstart blocks written to solution.rvc.

Block	Content
:GWHeads nlay nrow ncol	MODFLOW head of every cell (raw values, 12 digits); checked against the grid
:GWRechargeStore	HRU ID, water in the recharge-delay reservoir [m ³]
:GWReservoirSeepage	subbasin ID, reservoir seepage in transit [m ³]
:GWRiverLossCarry	subbasin ID, unsupplied river loss to take next step [m ³] (only when non-zero)

Each block ends with the matching :End line (:EndGWHeads, :EndGWRechargeStore, :EndGWReservoirSeepage, :EndGWRiverLossCarry).

A.12 Error messages

All stop the run with the message shown (abbreviated) in Raven_errors.txt.

Table A.12: Error messages and remedies.

Message	Remedy
:HRUGeometry is required in the .rvg file	add the HRU polygon file
cannot open GeoJSON / raster file; missing ID field	check paths and the ID field name
HRU ... has an AQUIFER_PROFILE but no polygon	polygon IDs must match .rvh IDs
HRU ... references unknown AQUIFER_PROFILE	define the profile in the .rvp
HRU ... does not overlap any active groundwater cell	smaller :GridCellSize or :MinCellCoverage; check bedrock
no HRU has an AQUIFER_PROFILE; coupled model requires :AquiferProfiles	fill the .rvh column; define profiles
no process moves water into GROUNDWATER	route recharge to GROUNDWATER
process # removes water from GROUNDWATER in HRU ...	restrict the process with :->Conditional
aquifer class/profile ... must be positive / may not be negative / between 0 and 1	correct the value (Sections A.2, A.4)
top layer cannot be an AQUICLUDE; layer thickness must be positive	correct the profile

Table A.12: Error messages and remedies (continued).

Message	Remedy
:MinCellCoverage, :GridCellSize, :SolverHead Tolerance, :SolverMaxOuterIterations, :HeadSaveFrequency out of range	correct the value (Section A.6)
improper format of command ...	correct the command syntax
boundary ... needs a HEAD value / positive CONDUCTANCE	complete the boundary command
two wells / observation wells / boundaries have the same ID or name	make IDs unique
HRU ... is in two :SurfaceWaterExchange groups for the same store	one exchange per store and HRU
well ... is outside the active groundwater model	move the well or enlarge the coupled area
:GWHeads does not match the groundwater grid	hotstart file from a different grid
the model's <i>type</i> package <i>name</i> would add recharge or ET beside Raven's	list it in :RavenTakesOver or :KeepPackages
:MF6StartDate ... / the model ends before Raven's run	correct the dates or :Duration
stress period ... is not a whole number of Raven time steps	choose a Raven time step that divides the periods
cells of the grid file differ from the model's cell area	wrong projection, length unit or cell numbering in the file
give one of :MF6CRS, :MF6GridFile or :OverlapWeights	link HRUs in exactly one way
the period data of <i>package</i> cannot be renumbered safely	start Raven at :MF6StartDate
:GridType FILE needs :GridFile (and the reverse)	give both, or neither
:GridRotation applies only to REGULAR and QUADTREE grids	remove it, or change the grid type
:GridRefinement size must be positive and smaller than :GridCellSize; RIVERS needs :RiverGeometry; WELLS needs wells	correct the refinement
:GridRefinement LAKES found no reservoir whose lake HRU has a polygon	give the reservoir an :HRUID whose polygon is in :HRUGeometry
grid cell ... must be a single polygon without holes	fix the cell file
HRU mesh exceeds 5 million cells per layer	increase :GridCellSize or the refinement sizes
grid exceeds 5 million cells per layer	increase :GridCellSize
cannot load MODFLOW 6 library	run python tools/get_mf6.py, or set :MF6Library or RAVEN_MF6_LIB
cannot enter MODFLOW 6 folder	output folder removed or not writable during the run
Raven is running more time steps than the MODFLOW 6 simulation holds	check :Duration and :TimeStep

A.13 Parameter index

Every parameter, its class, and where it is used.

Table A.13: Parameter index: every parameter, its class and where it is used.

Parameter	Class	Used in
K_HORIZ	aquifer class	flow between cells (Sec. 2); well split (Sec. 5.7)
K_VERT	aquifer class	vertical flow between layers (Sec. 3.5)
SPEC_STORAGE, SPEC_YIELD	aquifer class	storage (Sec. 2, 3.5)
POROSITY	aquifer class	reserved for transport
layers, thickness, type	aquifer profile	layer construction (Sec. 3.5)
INITIAL_HEAD_DEPTH	aquifer profile	initial heads (Sec. 3.5)
DEFAULT_BEDROCK_DEPTH	aquifer profile	bedrock without a map (Sec. 3.5)
SOIL_ZONE_DEPTH	aquifer profile	model top (Sec. 3.5); seepage (Sec. 5.2)
SEEPAGE_LEAKANCE, SEEPAGE_SMOOTHING_DEPTH	aquifer profile	seepage (Sec. 5.2)
RIVERBED_K, RIVERBED_THICKNESS	aquifer profile	river conductance (Sec. 5.3)
EXTINCTION_DEPTH	aquifer profile	water-table ET (Sec. 5.4)
RECHARGE_DELAY	aquifer profile	recharge delay (Sec. 5.1)
AQUIFER_PROFILE	HRU	coupling of HRUs (Sec. 3.1)
:SeepageParameters	reservoir	reservoir seepage (Sec. 5.6)
LEAKANCE	:SurfaceWater Exchange	wetland and lake exchange (Sec. 5.5)
CONDUCTANCE, HEAD	boundary	GHB and CHD (Sec. 5.7)
SCREEN_TOP, SCREEN_BOT, :WellRate	well	pumping (Sec. 5.7)
SCREEN_ELEV, GW_HEAD	monitoring well	heads and diagnostics (Sec. 5.7)
:GridCellSize, :MinCellCoverage	groundwater option	grid and active cells (Sec. 3.1)
:GridType, :GridRotation, :GridRefinement, :GridFile, :QuadtreeMaxLevel	groundwater option	grid types (Sec. 3.4)
:FlowCorrection, :LayerConnection	groundwater option	XT3D; layer connections (Sec. 3.4)
:MF6Simulation, :MF6CRS, :MF6GridFile, :OverlapWeights, :RavenTakesOver, :KeepPackages, :StreamPackage, :SeepageToRaven, :MF6StartDate, :MF6Model	existing-model option	Chapter 4
:RiverBedDepth, :SeepageReturnTo	groundwater option	rivers, seepage (Sec. 5.3, 5.2)
:SolverHeadTolerance, :SolverMax OuterIterations	groundwater option	Newton solution (Sec. 3.9)

Table A.14: Notation (continued)

Symbol	Meaning	Unit
--------	---------	------

A.14 Notation**Table A.14:** Notation.

Symbol	Meaning	Unit
$A_k, A_{k,i}$	HRU area (.rvh); overlap of HRU k and column i	m^2
C	conductance	m^2/d
d_s, d_x, d_0	seepage smoothing depth; ET extinction depth; initial depth to water	m
$\Delta, \Delta t$	cell size; time step	m, d
h, z^{wt}	head; water table of a column (highest wet cell)	m
K, K_h, K_v, K_b	hydraulic conductivity; horizontal; vertical; riverbed	m/d
k	reservoir seepage constant	$\text{m}^3/\text{s}/\text{m}$
L, L_s	leakance (stores); seepage leakance	$1/\text{d}$
Q, q	flow (positive into the aquifer for boundaries)	m^3/d
r_k	recharge depth of HRU k in a step	mm
s, s_k	river stage; store depth	m
S_s, S_y	specific storage; specific yield	$1/\text{m}, -$
T	transmissivity	m^2/d
τ	recharge delay time constant	d
V	volume	m^3
$z_{\text{land}}, z_{\text{top}}, z_{\text{br}}, z_{\text{bed}}$	land surface, model top, bedrock, riverbed elevations	m

Bibliography

- [1] Ann Calver. Riverbed permeabilities: information from pooled data. *Ground Water*, 39(4):546–553, 2001.
- [2] J. R. Craig, G. Brown, R. Chlumsky, et al. Flexible watershed simulation with the Raven hydrological modelling framework. *Environmental Modelling & Software*, 129:104728, 2020.
- [3] Patrick A. Domenico and Franklin W. Schwartz. *Physical and Chemical Hydrogeology*. Wiley, New York, 2nd edition, 1998.
- [4] R. Allan Freeze and John A. Cherry. *Groundwater*. Prentice-Hall, Englewood Cliffs, NJ, 1979.
- [5] J. D. Hughes, M. J. Russcher, C. D. Langevin, E. D. Morway, and R. R. McDonald. The MODFLOW application programming interface for simulation control and software interoperability. *Environmental Modelling & Software*, 148:105257, 2022.
- [6] A. I. Johnson. Specific yield — compilation of specific yields for various materials. Water-Supply Paper 1662-D, U.S. Geological Survey, 1967.
- [7] C. D. Langevin, J. D. Hughes, E. R. Banta, R. G. Niswonger, S. Panday, and A. M. Provost. Documentation for the MODFLOW 6 groundwater flow model. Techniques and Methods 6-A55, U.S. Geological Survey, 2017.
- [8] S. L. Markstrom, R. G. Niswonger, R. S. Regan, D. E. Prudic, and P. M. Barlow. GSFLOW: Coupled ground-water and surface-water flow model. Techniques and Methods 6-D1, U.S. Geological Survey, 2008.
- [9] J. P. Snyder. *Map Projections: A Working Manual*. U.S. Geological Survey Professional Paper 1395, 1987.
- [10] I. E. Sutherland and G. W. Hodgman. Reentrant polygon clipping. *Communications of the ACM*, 17(1):32–42, 1974.
- [11] B. A. Tolson and C. A. Shoemaker. Dynamically dimensioned search algorithm for computationally efficient watershed model calibration. *Water Resources Research*, 43:W01413, 2007.